

Универсальное пособие * полный исходный код

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 04.09.2026, 22:04:09
Файлов исходного кода: 77
Суммарный объем кода: 981.0 KB
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: seg-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: pref-sum]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пратта (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

```
.....  
22. src/data/tickets/ticket1_5.ts  
23. src/data/tickets/ticket14_20.ts  
24. src/data/tickets/ticket21_24.ts  
25. src/data/tickets/ticket6_13.ts
```

Интерактивные компоненты и симуляторы

```
26. src/components/ArticulationPointsViz.tsx  
27. src/components/BellmanFordViz.tsx  
28. src/components/BfsViz.tsx  
29. src/components/ChapterImage.tsx  
30. src/components/ChapterNav.tsx  
31. src/components/ChapterView.tsx  
32. src/components/DfsBridgesSimulator.tsx  
33. src/components/DijkstraViz.tsx  
34. src/components/DPVisualizer.tsx  
35. src/components/EulerSimulator.tsx  
36. src/components/ExpressQuiz.tsx  
37. src/components/FloydViz.tsx  
38. src/components/GraphTraversalViz.tsx  
39. src/components/HeapViz.tsx  
40. src/components/hints/demoKit.tsx  
41. src/components/hints/demosExtra.tsx  
42. src/components/hints/demosGraph.tsx  
43. src/components/hints/demosStruct.tsx  
44. src/components/hints/MiniDemo.tsx  
45. src/components/hints/SimulatorModal.tsx  
46. src/components/hints/TermPopover.tsx  
47. src/components/JohnsonViz.tsx  
48. src/components/KosarajuViz.tsx  
49. src/components/KruskalSimulator.tsx  
50. src/components/MnemonicCards.tsx  
51. src/components/MobileToc.tsx  
52. src/components/Navbar.tsx  
53. src/components/PlanarityDemo.tsx  
54. src/components/QueueViz.tsx  
55. src/components/SalmonAutomatonWidget.tsx  
56. src/components/SegmentTreeVisualizer.tsx  
57. src/components/Sidebar.tsx
```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7

AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

****CRITICAL DIRECTIVE****:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `ai_skills/universal_guide_authoring/SKILL.md`

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

ФАЙЛ 2/77: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

Gemini App Canvas Instructions

Ты – AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas). При запросе пользователя создать или сгенерировать новую страницу, ты ****строго**** придерживаться готовых шаблонов разметки, описанных ниже.

Правила для малых страниц (Билетов / Глав)

- **Единый стиль:**** Всегда используй Tailwind CSS и HTML.
- **Структура JSON/TS:**** Каждая новая малая страница должна быть описана в формате `{ "id": "...", "content": ... }` для вставки в `src/data/content.ts`.
- **Строгая структура контента:****
 - Заголовок с цветным бейджем (например, `bg-indigo-100`).
 - Определение/правило в центре (`border-blue-500` или `border-purple-500`).
 - Обязательная сетка аналогий (2 колонки: неформальный язык / строгий язык).

ФАЙЛ 5/77: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 41 | РАЗМЕР: 1

```
{
  "name": "algv0-exam-guide",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "audit:content": "node scripts/audit-coverage.mjs",
    "audit:terms": "node scripts/audit-terms.mjs",
    "export:txt": "node scripts/export/collect-code.mjs",
    "export:png": "node scripts/export/render-pages.mjs",
    "export:pdf": "node scripts/export/build-pdf.mjs",
    "export": "npm run export:txt && npm run export:png",
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
    "tailwind-merge": "3.4.0"
  },
  "devDependencies": {
    "@tailwindcss/vite": "4.1.17",
    "@types/node": "22.19.17",
    "@types/pdfkit": "^0.17.6",
    "@types/react": "19.2.7",
    "@types/react-dom": "19.2.3",
    "@vitejs/plugin-react": "5.1.1",
  }
}
```

Универсальное пособие • полный исходный код

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash
sudo apt-get install -y imagemagick fonts-dejavu-core
```
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` — DPI растеризации (по умолчанию)
- * `EXPORT\_GRAYSCALE=1` — оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **на каждом коммите** и пересобирает TXT → PNG → PDF, кладёт PNG-страницы и документ PDF в папку `public/export/full\_code\_all\_pages` и коммитит обновлённые `public/export/full\_code\_all\_pages` (с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code\_all\_pages`

Аудит контента

```
```bash
node scripts/audit-coverage.mjs # какие темы без
node scripts/audit-coverage.mjs --md # markdown-таблицы
```
```

Актуальный отчёт: [`TOPICS\_AUDIT.md`](./TOPICS_AUDIT.md)

Как устроен интерфейс

- * **Содержание** — на десктопе боковая панель с поиском, на телефоне — липкая строка «Содержание · N из M» и вкладки «Содержание» и «Поиск»
- * **Переходы между темами** — кнопки «Назад / Вперёд» в шапке, «Предыдущая / Следующая тема» под текстом; на клавиатуре — стрелки
- * **Всплывающие подсказки по терминам** — текст главы автоматически подсвечивается, известные термины (BFS, куча, бор, суффиксная ссылка, ...)

Универсальное пособие • полный исходный код

.....
* Таким образом, при извлечении текстового содержимого технически остаются в валидном Markdown-формате латеха (и

Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- * Они помещены внутрь оберток с классом `<div class="img">`
- * Все отрисовки находятся внутри векторного тега `<svg>`
- * Подпись к картинке, если она есть, идет сразу после SVG

Списки

- * Стандартные списки оформлены через `<ul class="list-n">`

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `{ Иллюстрация }`
4. Элементы списка `<li>` получают маркер `-``
5. Результат извлекается в виде текста и чистится от лишнего

.....
ФАЙЛ 7/77: TOPICS_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

.....
Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.m`

> Критерии:

> * **Аналогия** — в HTML главы есть блок «Аналогия ...»

> * **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

Универсальное пособие • полный исходный код

```
.....
| 16. Флойд (Флойд–Уоршелл) | `floyd` | Есть матричный
| 17. Алгоритм Джонсона | `johnson-algo` | Переавзвешива
| Обзор: Кратчайшие пути и Графы | `intro` | Вводная стр
| Эйлер: Путь ≠ Цикл | `euler-path-vs-cycle` | Ключевая
```

```
## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации (
```

```
> Замкнуто: «5. Двумерная матрица префиксных сумм» пол
```

```
Глава	id	Чего не хватает
3. Декартово дерево (Trear)	`trear`	Нет ни визуал
8. Графы. Компоненты связности	`graph-components`	
24. Классы сложности, сведение задач	`complexity-cl	
```

```
## Сводная таблица по всем главам
```

| # | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|-----|---|----------|---------|-----|----------|
| --- | --- | --- | --- | --- | --- |
| 2 | 2. Двумерная разреженная матрица (Sparse Table) | | | | |
| 3 | 3. Декартово дерево (Trear) | 2 | – | – | – |
| 4 | 4. Splay-дерево | 2 | | – | – |
| 5 | 5. Двумерная матрица префиксных сумм | 1 | | | – |
| – | Планарность: K5, K3,3 и формула Эйлера | – | | | – |
| 6 | 6. Графы. Представление, DFS, BFS | 4 | | – | – |
| 8 | 8. Графы. Компоненты связности | 1 | – | – | – |
| 9 | 9. Графы. Топологическая сортировка | 1 | | | – |
| 10 | 10. Графы. Компоненты сильной связности (SCC) | | | | – |
| 12 | 12. Графы. Точки сочленения | 1 | | – | – |
| 14 | 14. Графы. Кратчайший путь. Дейкстра | 3 | | | – |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 | | | – |
| 16 | 16. Графы. Кратчайший путь. Флойд | – | | | – |
| 17 | 17. Графы. Алгоритм Джонсона | – | | – | – |

```
.....
  /* Bundler mode */
  "moduleResolution": "bundler",
  "allowImportingTsExtensions": true,
  "resolveJsonModule": true,
  "isolatedModules": true,
  "noEmit": true,
  "jsx": "react-jsx",

  /* Path mapping */
  "baseUrl": ".",
  "paths": {
    "@/*": ["src/*"]
  },

  /* Linting */
  "strict": true,
  "noUnusedLocals": true,
  "noUnusedParameters": true,
  "noFallthroughCasesInSwitch": true
},
"include": ["src", "vite.config.ts"]
}
```

.....
ФАЙЛ 9/77: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 33 | РАЗМЕР: 812

```
.....
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(  __filename);
```



```

.....
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";

export default function App() {
  const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");
  const [activeChapterId, setActiveChapterId] = useState<string>("");
  const [modalVizId, setModalVizId] = useState<string>("");

  const index = Math.max(
    0,
    chapters.findIndex((c) => c.id === activeChapterId)
  );
  const activeChapter = chapters[index] ?? chapters[0];
  const prev = index > 0 ? chapters[index - 1] : undefined;
  const next = index < chapters.length - 1 ? chapters[index + 1] : undefined;

  const goTo = useCallback((id: string) => {
    setActiveChapterId(id);
    window.scrollTo({ top: 0, behavior: "smooth" });
  }, []);

  // Стрелки ← → листают темы (как на обучающих сайтах)
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      const target = e.target as HTMLElement | null;
      if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName)) return;
      if (e.metaKey || e.ctrlKey || e.altKey) return;
      if (e.key === "ArrowRight" && next) goTo(next.id);
      if (e.key === "ArrowLeft" && prev) goTo(prev.id);
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
  }, [goTo, next, prev]);

  const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters.length]);

```

```

.....
        style={{ width: `${progress}%` }}
      />
    </div>

    <ChapterView
      key={activeChapter.id}
      chapter={activeChapter}
      prev={prev}
      next={next}
      index={index}
      total={chapters.length}
      onGo={goTo}
      onOpenSimulator={setModalVizId}
    />
  </main>
</div>
</>
) : (
  <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
    <SimulatorHub
      onOpen={setModalVizId}
      onGoChapter={({id}) => {
        setActiveTab("guide");
        goTo(id);
      }}
    />
  </main>
)}

<SimulatorModal vizId={modalVizId} onClose={() => {}} />

<footer className="p-8 text-center text-slate-600">
  © 2026 Universal Educational Guide. Интерактивный
</footer>
</div>
);
}

```

```

.....
    const parent = (node as Text).parentElement;
    if (!parent) return NodeFilter.FILTER_REJECT;
    if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
    if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
    return NodeFilter.FILTER_ACCEPT;
  },
});

const targets: Text[] = [];
let current = walker.nextNode();
while (current) {
  targets.push(current as Text);
  current = walker.nextNode();
}

for (const textNode of targets) {
  const text = textNode.nodeValue ?? "";
  re.lastIndex = 0;
  if (!re.test(text)) continue;
  re.lastIndex = 0;

  const frag = document.createDocumentFragment();
  let last = 0;
  let m: RegExpExecArray | null;

  while ((m = re.exec(text)) !== null) {
    const id = labelToId.get(m[1].toLowerCase());
    if (!id) continue;
    const used = counts.get(id) ?? 0;
    if (used >= MAX_PER_TERM) continue;
    counts.set(id, used + 1);

    if (m.index > last) frag.appendChild(document.createElement("span"));
    const span = document.createElement("span");
    span.className = "term-hint";
    span.dataset.termId = id;
    span.tabIndex = 0;
    span.setAttribute("role", "button");
  }
}

```

```
.....
.neon-glow-emerald {
  box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
  box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
  box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
  from {
    transform: translateX(-100%);
  }
}
```

```
.....
@media (hover: none) {
  .term-hint {
    border-bottom-style: solid;
    background: rgba(99, 102, 241, 0.12);
  }
}

/* Контент главы приходит готовым HTML — чиним его под
.chapter-body {
  max-width: 100%;
  overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
  min-width: 0;
}

.chapter-body :where(pre, code) {
  white-space: pre-wrap;
  word-break: break-word;
}

.chapter-body pre {
  overflow-x: auto;
  max-width: 100%;
  font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
  display: block;
  width: 100%;
  overflow-x: auto;
  border-collapse: collapse;
  font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
  cursor: pointer;
```

```
.....
    background: #334155;
    border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
    background: #475569;
}
```

.....
ФАЙЛ 13/77: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
.....
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

.....
ФАЙЛ 14/77: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
.....
export interface Chapter {
  id: string;
  title: string;
  type: "html" | "markdown";
  content: string;
  description?: string;
  category?: string;
}
```

```
.....
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-blue-300 italic">
              <p class="text-xl font-mono text-white">
                yx отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
            </p>
          </div>
          <div class="grid grid-cols-1 xl:grid-cols-2">
            <div class="bg-slate-800 p-6 rounded-lg">
              <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                Аналогия 1: Школьные линейки
              </div>
              <p class="text-slate-300 text-sm mb-4">
                зок длины 7. Ты берешь две заготовленные за
                ь отрезок длины 7 (перекрывая друг друга по
                ничего складывать, просто пересекаем куски.
              </div>
            <div class="bg-slate-900 p-6 rounded-lg">
              <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                Аналогия 2: Ступени
              </div>
              <p class="text-slate-300 text-sm mb-4">
```

```
.....
    <p class="text-xl font-mono text-white">
        метода: Split и Merge.</p>
    </div>
    <div class="grid grid-cols-1 xl:grid-cols-2"
        <div class="bg-slate-800 p-6 rounded-lg"
            <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
                Аналогия 1: Удар катаной (Sp
            </div>
            <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
            </div>
            <div class="bg-slate-900 p-6 rounded-lg"
                <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
                    Аналогия 2: Слияние капель (M
                </div>
                <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
            </div>
        </div>
    <details class="bg-slate-900/40 rounded-lg"
        <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300"
            <pre class="bg-slate-950 p-4 rounded
pair&lt;Node*, Node*&gt; split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val &lt;= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
```



```

.....
    </div>
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
        Аналогия 2: Балансировка чере
      </div>
      <p class="text-slate-300 text-sm mb
и становиться кривым как бамбук. Но как тол
    </p>
  </div>
</div>
<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300"
    <pre class="bg-slate-950 p-4 rounded
// Zig, Zag
function rotate(x) {
  let p = x.parent;
  if (p.left === x) { // Right rotate
    p.left = x.right;
    x.right = p;
  } else { // Left rotate
    p.right = x.left;
    x.left = p;
  }
}</pre>
    </div>
  </details>
</div>
</div>
</section>',
},
{
  id: "prefix-sums-2d",
  title: "5. Двумерная матрица префиксных сумм",

```

```

.....
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 r
        <h2 class="text-2xl sm:text-3xl font-bold text-
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-blue-400
            <div class="bg-slate-900 p-4 rounded-lg mb-
                <p class="text-lg text-blue-300 italic
                <p class="text-xl font-mono text-white"
            ый граф можно раскрасить 4 цветами.</p>
        </div>
        <div class="grid grid-cols-1 gap-6">
            <div class="bg-slate-800 p-6 rounded-lg
                <div class="absolute -top-3 left-4
                rder border-emerald-500 shadow-md">
                    Аналогия 1: Карта мира
                </div>
                <p class="text-slate-300 text-sm mb
            . Границы не пересекаются в одной точке по-
            ы не сливались цветами, Всегда можно всего
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "top-sort",
        title: "9. Графы. Топологическая сортировка",
        type: "html",
        description: "Разложение задач по порядку выполнения",
        category: "Графы",
        content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

.....
  description: "Разбиение орграфа на макро-вершины. А
  category: "Графы. Продвинутые",
  content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
        нвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            БЫХ направлениях по односторонним улицам. Как это выглядит
            у на карте.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
},
{
  id: "graph-bridges",
  title: "11. Графы. Мосты",
  type: "html",
  description: "Рёбра, удаление которых расхреначивает граф",
  category: "Графы. Продвинутые",
  content: `

```

```

</div>
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-4"
      <p class="text-lg text-rose-300 italic"
      <p class="text-xl font-mono text-white">
тей > 1).</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 b
        rder border-emerald-500 shadow-md">
          Аналогия 1: Центральный роутер
        </div>
        <p class="text-slate-300 text-sm mb-4"
правильный роутер - и два сегмента офиса б
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-indigo-400

```

```

</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 rder border-emerald-500 shadow-md">
      Аналогия: Копаем туннель под .
    </div>
    <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-emerald-4
      <div class="bg-slate-900 p-4 rounded-lg mb-1
        <p class="text-lg text-emerald-300 ital
        <p class="text-xl font-mono text-white"
        (проверяем через DSU) - берем в ответ.</p>
      </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg

```

```

</div>
<p class="text-slate-300 text-sm mb-6">
    ли вирус). Мы стартуем из одного города, и
    Сеть растет только из одного связного куска
  </div>
</div>
</div>
</div>
</section>`,
  },
  {
    id: "mst-boruvka",
    title: "20. Графы. Остовное дерево. Борувка",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-emerald-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">
              <p class="text-xl font-mono text-white">
                ается с соседом.</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 i
                  rder border-emerald-500 shadow-md">
                    Аналогия: Племена
                </div>
                <p class="text-slate-300 text-sm mb-6">
                  изкому племени для объединения. К вечеру пл

```

```

        </div>
    </div>
    <details class="bg-slate-900/40 rounded-lg"
        <summary class="p-4 font-bold text-slate-300"
        -b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300"
        <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</section>',
{
    id: "string-z-func",
    title: "22. Z-функция",
    type: "html",
    description: "",
    category: "Строки",
    content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-blue-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-1

```

```
.....
    description: "",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded-md">Сложность
    <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Сложность
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">Сведение (Reduction)
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Сведение (Reduction)
        <p class="text-xl font-mono text-white">Сведение (Reduction) A->B: Если я умею реша
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
          </div>
          <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс <b>NP</b> — это не-
          енную сетку (ответ), он БЫСТРО (за класс P)
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md">
            Аналогия 2: Машина-переводчик
          </div>
          <p class="text-slate-300 text-sm mb-4">Это отличный переводчик на английский (Сведен
          аче B, то B — это <b>NP-Полная</b> (сосредото
        </div>
      </div>
    </div>
  </div>
`
```


.....

ин раз. Вершины повторять можно.</p>

<p><strong class="text-white">Эйлер

<div class="p-3 bg-slate-950 rounded

<p class="text-sm text-blue-300

<p><strong class="text-white">П

<p><strong class="text-white">Ц

</div>

</div>

</div>

<p class="text-slate-300 text-sm">

Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font-
финиш).

</p>

</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap

<div class="bg-slate-800 p-6 rounded-lg bor

<div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">

Путь: Нормальная прогулка

</div>

<p class="text-slate-300 text-sm mb-4">

Ты вышел из дома (нечётная)-
талый пришёл в бар (вторая нечётная
Домой вернуться не смог, потому что

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg bor

<div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md bg-slate-950">

Цикл: Гамильтонов синдром (болезнь

</div>

<p class="text-slate-300 text-sm mb-4">

```
.....
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-4">
    <h2 class="text-3xl font-bold text-white">Мосты
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500">
    <h3 class="text-xl font-bold text-emerald-400">Мост: Единственная веревка

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-emerald-300 italic">Мост: Единственная веревка
      <div class="text-left font-mono text-sm">
        <p><strong class="text-white">tin{v
        <p><strong class="text-white">low{v
        <div class="p-3 bg-slate-950 rounded">
          <span class="text-rose-400 font-bold">Мост: Единственная веревка
          <p class="text-xl font-bold text-white">Мост: Единственная веревка
          <p class="text-xs text-slate-400">Мост: Единственная веревка
        </div>
      </div>
    </div>
  </div>
```

```
<p class="text-slate-300 text-sm">
  Если из сына и и всех его потомков <strong>Мост: Единственная веревка
  Единственная ниточка. Пораётся – граф р
</p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500">
    <div class="absolute -top-3 left-4 bg-slate-900 border-emerald-500 shadow-md">
      Мост: Единственная веревка
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь, что ты альпинист. Ты спу
      Из пещеры и нет других выходов – то
      Если верёвка (ребро v-u) оборвётся
```

```

.....
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;           // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue;         // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v);                  // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) dfs(i);
    }
}
}

```

</div>

<p class="text-xs text-slate-400">Занес

</div>

```
.....
    order-green-500 shadow-md">
```

К5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У К5: `class="text-white">V=5, -white">10 > 9</code> – БАХ, непланарен!`

Все 5 вершин соединены со всеми. На

Теорема Куратовского: К5 – эталон не

Если внутри графа спрятан К5 – забудь,

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border
```

```
    <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

К3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя

Нарисовать без пересечений невозможно

У него `class="text-white">V=6 xt-white">9 ≤ 12</code> – проходит, но он в`

Почему? Потому что у двудольного графа

Отсюда более строгое ограничение: `<`

`class="text-white">9 > 8</code>`

```
</p>
```

```
</div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg border
```

```
    <summary class="p-4 font-bold text-slate-400
order-slate-700/50">
```

Доказательство непланарности К5 (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 space
```

```
    <p class="text-blue-400">Доказательство
```

```
    <p>
```

Пусть К5 планарен. $V = 5$, $E = 10$.

```
.....
    <p class="text-slate-300 text-sm">
      <strong>Важно:</strong> Это работает то
      епятся к точкам сочленения. То есть, <strong>
      ег - это тупик со степенью 1).
    </p>
  </div>

  <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
    <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
      <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
        Аналогия: Главный перекресток
      </div>
      <p class="text-slate-300 text-sm mb-4">
        Представь город, где два района соединены
        её перекроют (удалят вершину) – районы будут
        <strong>хрупкость</strong> системы.
      </p>
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
      <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
        Телеком: Центральный свитч
      </div>
      <p class="text-slate-300 text-sm mb-4">
        У тебя несколько компьютеров в одной
        абелем (мостом). Сами свитчи – это точки со
      </p>
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg border border-slate-800">
    <summary class="p-4 font-bold text-slate-400">
      Душный мод: Код и корень DFS (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-2">
```

```
.....
    <li><strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
    <li><strong class="text-emerald-400">SCC
ависимые "конгломераты".</li>
    <li><strong>Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S
</ul>
</div>
</div>
</section>`,
  },
};
```

ФАЙЛ 18/77: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

```
.....
import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp

// We want to combine all of them, but overwrite 7, 11,
const merged = [
  ...graphChapters,
  ...additionalTickets,
  ...tickets6to13,
  ...tickets14to20,
  ...tickets21to24
].filter(c => c && c.id);
```

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
  id: string;
```

```
  /** Варианты написания в тексте (регистр не важен). */
```

```
  labels: string[];
```

```
  title: string;
```

```
  /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
  short: string;
```

```
  /** Строгая формулировка / что важно не забыть. */
```

```
  formal?: string;
```

```
  complexity?: string;
```

```
  /** Какая мини-анимация показывается в карточке. */
```

```
  demo?: DemoKind;
```

```
  /** id полного симулятора (см. vizRegistry) – кнопка
```

```
  simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
  {
```

```
    id: "bfs",
```

```
    labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
    title: "BFS – обход в ширину",
```

```
    short:
```

```
      "Волна от источника: сначала все соседи, потом со
```

```
    formal:
```

```
      "Очередь (FIFO). Кладём старт, достаём вершину –
```

```
        у рёбер.",
```

```

.....
    id: "queue",
    labels: ["очередь", "очереди", "очередью", "FIFO"],
    title: "Очередь (FIFO)",
    short: "Очередь в столовой: кто первым встал, тот и ест",
    formal: "push_back / pop_front за  $O(1)$ . Основа BFS",
    demo: "queue",
    simulator: "queue-bfs",
},
{
    id: "dsu",
    labels: ["DSU", "СНМ", "система непересекающихся множеств"],
    title: "DSU – система непересекающихся множеств",
    short:
        "«Кто твой староста?» Каждый элемент знает своего старосту",
    formal:
        "find со сжатием путей + union по рангу/размеру. Обратная функция Аккермана",
    complexity: "почти  $O(1)$  – обратная функция Аккермана",
    demo: "dsu",
    simulator: "mst-kruskal",
},
{
    id: "trie",
    labels: ["бор", "бора", "боре", "бору", "бором", "trie"],
    title: "Бор (trie, префиксное дерево)",
    short:
        "Дерево, где путь от корня по буквам и есть слово",
    formal: "Из корня по символам; в вершине хранится ф. слово",
    complexity: "поиск слова –  $O(|\text{слова}|)$ ",
    demo: "trie",
    simulator: "aho-corasick",
},
{
    id: "bfs-on-trie",
    labels: [
        "обход дерева",
        "обход бора",
        "обход в ширину по бору",
        "какой-то обход",
    ],

```



```

.....
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
        ,
    formal: "if  $d[v] > d[u] + w(u,v) \rightarrow d[v] = d[u] + w(u,v)$ 
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "префикс"],
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набежало от начала»
    formal: " $P[0]=0, P[i]=P[i-1]+a[i-1]$ . Сумма  $a[l..r] = P[r]-P[l-1]$ .",
    complexity: "препроцессинг  $O(n)$ , запрос  $O(1)$ ",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "sparse-table",
    labels: [
        "разреженная таблица", "разреженная таблица", "sparse-table",
        "разреженная", "двоичные подъёмы", "двоичный подъём",
    ],
    title: "Разреженная таблица (метод двоичных подъёмов)",
    short:
        "Заранее считаем ответ для всех отрезков длины 1,
        M.",
    formal: " $ST[l][j] = \text{op}(ST[l][j-1], ST[l+2^{(j-1)}][j-1])$ ",
    complexity: "препроцессинг  $O(n \log n)$ , запрос  $O(1)$ ",
    demo: "sparse-table",
}

```

Универсальное пособие * полный исходный код

```
.....
    "остовное дерево", "остовного дерева", "остовное"
    "минимальное остовное дерево", "MST",
},
title: "Минимальное остовное дерево (MST)",
short:
    "Связать все точки проводами так, чтобы суммарная
formal: "V-1 ребро, никаких циклов. Краскал (жадно
    но).",
demo: "mst",
simulator: "mst-kruskal",
},
{
    id: "amortized",
    labels: [
        "амортизированная", "амортизированный", "амортизи
        "амортизированное", "амортизированного", "амортизи
    ],
    title: "Амортизированная сложность",
    short:
        "Средняя цена операции «по абонементу»: одна опер
    formal: "Метод потенциалов: дорогая операция оплачи
        ассив.",
    demo: "amortized",
},
{
    id: "np",
    labels: [
        "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
        "класс P", "полином", "полиномиальное",
    ],
    title: "Класс NP и NP-полнота",
    short:
        "NP — «решение проверить легко, найти трудно» (су
        ые в NP.",
    formal: "Задача NP-полна, если она в NP и к ней полн
    demo: "np",
},
{
```

```

},
{
  id: "z-function",
  labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функция"],
  title: "Z-функция",
  short:
    "Для каждой позиции: сколько символов подряд, начинающихся с этой буквы.",
  formal:
    " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i..n]$ ",
  complexity: " $O(n)$ ",
  demo: "prefix-function",
  simulator: "string-z-func",
},
{
  id: "dp",
  labels: [
    "динамическое программирование", "динамики", "динамическое",
    "мемоизация", "мемоизацию", "подзадач", "подзадачи"
  ],
  title: "Динамическое программирование",
  short:
    "Решаем задачу через ответы на её меньшие копии и сохраняем их.",
  formal:
    "Нужны: оптимальная подструктура + перекрывающиеся подзадачи.",
  demo: "dp",
  simulator: "dynamic-programming",
},
{
  id: "shortest-path",
  labels: ["кратчайший путь", "кратчайшие пути", "кратчайшие"],
  title: "Кратчайший путь",
  short:
    "Самый дешёвый маршрут из A в B. «Дешёвый» – по стоимости ребра.",
  formal:
    "Дейкстра – неотрицательные веса,  $O(E \log V)$ . Фор-Мора – произвольные веса,  $O(V^3)$ .",

```

```

.....
{
  id: "planarity",
  labels: ["планарный", "планарность", "планарного",
  title: "Планарность",
  short:
    "Граф планарен, если его можно нарисовать на листе",
  formal:
    "V - E + F = 2 для связного планарного графа. Курсив",
  simulator: "planarity-euler-formula",
},
{
  id: "splay",
  labels: [
    "splay", "splay-дерево", "поворот", "повороты", "всплытие",
    "zig-zag", "zig-zig", "AVL", "AVL-дерево", "всплытие",
  ],
  title: "Splay и повороты",
  short:
    "Каждый раз, когда к узлу обратились, его «вытягивают»",
  formal:
    "Три случая: zig (родитель – корень), zig-zig (узлы в одну сторону), zig-zag (узлы в разные стороны).",
  demo: "amortized",
  simulator: "splay-tree",
},
{
  id: "adjacency",
  labels: ["матрица смежности", "список смежности", "матрица смежности", "список смежности"],
  title: "Способы хранить граф",
  short:
    "Матрица смежности – таблица «есть ли ребро», быстрый доступ, экономно для разреженных графов.",
  formal: "Матрица: проверка O(1), память O(V^2). Список: проверка O(V), память O(V).",
},
{
  id: "dijkstra",

```

```

.....
    complexity: "O(V^3) времени, O(V^2) памяти",
    demo: "floyd",
    simulator: "floyd",
},
{
    id: "prim",
    labels: ["Прима", "Прим", "Prim"],
    title: "Алгоритм Прима",
    short:
        "Остов растёт как плесень из одной точки: на каждо
    formal:
        "Множество посещённых + куча рёбер на границе. До
        усок.",
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-prim",
},
{
    id: "boruvka",
    labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
    title: "Алгоритм Борувки",
    short:
        "Все компоненты одновременно шлют «гонца» к ближайш
    formal:
        "Фаза: для каждой компоненты ищем минимальное исх
    complexity: "O(E log V)",
    demo: "mst",
    simulator: "mst-boruvka",
},
{
    id: "kruskal",
    labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
    title: "Алгоритм Краскала",
    short:
        "Сортируем все рёбра по весу и берём подряд те, ч
    formal:
        "Сортировка O(E log E) + E операций union/find. C
    complexity: "O(E log E)",

```

```
.....
    "Координаты бывают гигантские и дробные, а нам ну-
      рядковым номером.",
  formal:
    "sort + unique + binary search: значение → индекс
      ,
  complexity: "O(n log n)",
  demo: "coord-compress",
},
{
  id: "treap",
  labels: ["Treap", "декартово дерево", "декартова де
  title: "Treap = Tree + Heap",
  short:
    "Одно дерево живёт по двум правилам сразу: по ключу
      ча. Случайность и держит его сбалансированным."
  formal:
    "Базис – split(t, x) (разрезать по ключу) и merge
      я через них. Ожидаемая высота O(log n).",
  complexity: "ожидаемо O(log n)",
  demo: "heap",
},
{
  id: "bst",
  labels: ["бинарное дерево поиска", "дерево поиска",
  title: "Бинарное дерево поиска",
  short:
    "Слева всё меньше корня, справа – всё больше. Пои
  formal:
    "Без балансировки дерево может вырождаться в списо
  complexity: "O(высоты): от O(log n) до O(n)",
  demo: "segment-tree",
},
{
  id: "inclusion-exclusion",
  labels: [
    "включений-исключений", "включения-исключения", "
    "вырезание прямоугольников",
  ],
}
```

Универсальное пособие • полный исходный код

```
.....
    "Строковые алгоритмы держатся на трёх вопросах: где (Z-функция) и где встречаются слова словаря (бордерформал:
    "Наивный поиск подстроки –  $O(N \cdot M)$ . Префикс-функция.
    demo: "prefix-function",
    simulator: "string-kmp",
  },
};
```

```
/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (человек любит читать с конца)
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  GLOSSARY.map((e) => ({ label: e.label, id: e.id }))
    .sort((a, b) => b.label.length - a.label.length);
```

.....
ФАЙЛ 20/77: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРок: 275 | РАЗМЕР: 1.5 КБ

```
.....
import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
  {
    id: "intro",
    title: "Обзор: Кратчайшие пути и Графы",
    type: "html",
    category: "Графы",
    content: `<section id="intro-content" class="mb-12">
      <div class="flex items-center mb-6">
        <span class="bg-indigo-600 text-white px-4 py-1">Обзор
        <h2 class="text-3xl font-bold text-white">Обзор
      </div>
      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border`
  }
];
```

```

.....
    {нельзя поехать и заработать). Он всегда в
      <div id="slot-chapter-image-dijkstra"
    </div>

    <!-- Аналогия 2 -->
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
        Ограничения
      </div>
      <p class="text-slate-300 text-sm mb-6">
охождение улицы), Дейкстра сломается, так к
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg"
    <summary class="p-4 font-bold text-slate-300"
-b border-slate-700/50">
      Строгое доказательство (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300"
      <p>Алгоритм использует приоритетную
g V). Гарантирует корректность только для н
    </div>
  </details>

  <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
  id: "bellman-ford",
  title: "Билет 15. Форд-Беллман",
  type: "html",
  category: "Графы",
  content: `<section id="bellman-ford-content" class="
<div class="flex items-center mb-6">

```



```

</div>
</section>`
},
{
  id: "floyd",
  title: "Билет 16. Флойд-Уоршелл",
  type: "html",
  category: "Графы",
  content: `<section id="floyd-content" class="mb-12">
<div class="flex items-center mb-6">
  <span class="bg-blue-600 text-white px-4 py-1 rounded">
  <h2 class="text-3xl font-bold text-white">Флойд
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-indigo-400">
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-indigo-300 italic">
      <p class="text-xl font-mono text-white">
    </div>

    <div class="grid grid-cols-1 xl:grid-cols-2">
      <!-- Аналогия 1 -->
      <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
          Аналогия 1: Автомагистрали
        </div>
        <p class="text-slate-300 text-sm mb-4">
евле долететь из Москвы в Париж, если мы сд
        <div id="slot-chapter-image-floyd">
        </div>

      <!-- Аналогия 2 -->
      <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4

```

```

.....
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
        Аналогия 1: Паутина (Бор)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        мы движемся по веткам. Если нужного продолж
        ("нити страховки") в самый длинный из извест
      </div>

  <div class="bg-slate-900 p-6 rounded-lg border-2
    <div class="absolute -top-3 left-4 bg-rose-900
      -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
        терминальные ссылки</b> (term_link) – прямые мос
      <p>
    </div>
  </div>

  <details class="bg-slate-900/40 rounded-lg border b
    <summary class="p-4 font-bold text-slate-400 outli
      pen:border-slate-700/50">
        Внутренняя структура автомата и код BFS (Скры
      </summary>
      <div class="p-5 text-sm text-slate-300 space-y-4">
        <p>Каждая вершина имеет таблицу переходов <code>
          ссылку <code>term_link</code> (ближайшая мен
        <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
  map<char, int> go;          // Предавычисленные пер
  int link = 0;              // Суффиксная ссылка: куда
  int term_link = 0;         // Терминальная ссылка: мат
  bool is_terminal = false;
  vector<string> words;

```



```

.....
    "Нет, low[u] должно быть меньше или равно",
    "Только если граф связный и неориентированный"
  ],
  correctIndex: 0,
  explanation: "Условие моста: low[u] > tin[v]. В н
},
{
  question: "Для чего нужен массив low[v]?",
  options: [
    "Для того чтобы было больше переменных в коде",
    "Чтобы знать минимальный tin предка, до которого",
    "Чтобы хранить глубину рекурсии"
  ],
  correctIndex: 1,
  explanation: "low[v] хранит минимальное время входа
    поиска мостов и точек сочленения."
},
{
  question: "Какова временная сложность алгоритма п
  options: [
    "O(V^2) — ищем мосты в квадрате",
    "O(V + E) — каждый элемент один раз",
    "O(2^V) — полный перебор"
  ],
  correctIndex: 1,
  explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
  question: "Планарный граф имеет V=6, E=12. Возможен ли такой граф?",
  options: [
    "Да, если это двудольный граф",
    "Нет, нарушается неравенство E ≤ 3V - 6",
    "Да, если нет треугольных граней"
  ],
  correctIndex: 1,
  explanation: "E ≤ 3V - 6 ⇒ 12 ≤ 12 — на грани пр

```

```

{
  id: "segment-trees",
  title: "1. Дерево отрезков с операциями на отрезках",
  type: "html",
  description: "Дерево отрезков – структура данных для...",
  category: "Оптимизации и Продвинутое",
  content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>

  <div class="space-y-8">
    <div id="intro" class="bg-slate-700/50 p-6 rounded">
      <h3 class="text-xl font-bold text-blue-400">

      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        omise (lazy).</p>
      </div>

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 b
            rder border-emerald-500 shadow-md">
            Аналогия 1: Корпоративная пре
          </div>
          <p class="text-slate-300 text-sm mb">
            ы рассылать 10 000 писем, он пишет одно пис
            счетах сотрудников только тогда, когда сот
            женное обновление).</p>
        </div>

        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 b
            border-rose-500 shadow-md">

```

```
.....
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        
$$y \times \text{отрезков: } \min(\text{ST}[L][k], \text{ST}[R - 2^k + 1][k])$$

      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные заготовки
            ь отрезок длины 7 (перекрывая друг друга пополам)
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
              Аналогия 2: Ступени
            </div>
            <p class="text-slate-300 text-sm mb-4">
              ужно покрыть отрезок, мы берем две самые большие
            </div>
          </div>
        <div class="bg-slate-900/40 rounded-lg p-6">
          <summary class="p-4 font-bold text-slate-300">
            -b border-slate-700/50">
              Код (Скрыто)
            </summary>
          </div>
        </div>
      </div>
    <div class="bg-slate-900/40 rounded-lg p-6">
      <summary class="p-4 font-bold text-slate-300">
        -b border-slate-700/50">
          Код (Скрыто)
        </summary>
      </div>
    </div>
  </div>
```

```

</div>
<div class="bg-slate-900 p-6 rounded-lg">
  <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
    Аналогия 2: Слияние капель (М
  </div>
  <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
</div>
</div>
<details class="bg-slate-900/40 rounded-lg">
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <pre class="bg-slate-950 p-4 rounded
pair&lt;Node*, Node*&gt; split(Node* t, int x) {
  if (!t) return {nullptr, nullptr};
  if (t->val &lt;= x) {
    auto [L, R] = split(t->right, x);
    t->right = L;
    return {t, R};
  } else {
    auto [L, R] = split(t->left, x);
    t->left = R;
    return {L, t};
  }
}</pre>
  </div>
</details>
</div>
</div>
</section>
},
{
  id: "splay-tree",

```

```
.....
    <details class="bg-slate-900/40 rounded-lg
      <summary class="p-4 font-bold text-slate
        -b border-slate-700/50">
          Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300
          <pre class="bg-slate-950 p-4 rounded
// Zig, Zag
function rotate(x) {
  let p = x.parent;
  if (p.left === x) { // Right rotate
    p.left = x.right;
    x.right = p;
  } else { // Left rotate
    p.right = x.left;
    x.left = p;
  }
}</pre>

    </div>
  </details>

  <div class="bg-indigo-950/60 p-6 rounded-xl
    <h4 class="text-lg font-bold text-indig

      <div class="space-y-4 text-sm text-slate
        <div>
          <strong class="text-white block
корень?</strong>
          <p>Оно поднимет узел, на котором
ска. За счет сдвига этого листа в корень, д
/p>
        </div>

        <div>
          <strong class="text-white block
          <p>Если агент постоянно переключо
находящимися на разных концах дерева, его
ащая дерево в вытянутый бамбук для другого.
```


Универсальное пособие • полный исходный код

```
.....
        <div class="absolute -top-3 left-4
            rder border-emerald-500 shadow-md">
            * Аналогия 1: Вырезание прямоу
        </div>
        <p class="text-slate-300 text-sm mb
            з него маленький целевой прямоугольник в це
            ый верхний угол отрезается дважды! Поэтому
        </div>
    </div>
</div>
</section>`
    }
};
```

ФАЙЛ 23/77: src/data/tickets/ticket14_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

```
import { Chapter } from '../types';
```

```
export const tickets14to20: Chapter[] = [
```

```
    {
        id: "dijkstra",
        title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
        type: "html",
        category: "Графы. Пути",
        content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">14
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">14. Графы. Поиск кратчайшего пути. Дейкстра
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
            <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Вырезание прямоугольника
        </div>
    </div>
`
    },
];
```

```

.....
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
</div>
<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
        <h3 class="text-xl font-bold text-rose-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">
                    <p class="text-xl font-mono text-white">
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        ♂ Аналогия 1: Параноик
                    </div>
                    <p class="text-slate-300 text-sm mb-4">
                        еряет ВСЕ возможные дороги V-1 раз подряд.
                    </div>
                    <div class="bg-slate-900 p-6 rounded-lg">
                        <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                            Отрицательный цикл
                        </div>
                        <p class="text-slate-300 text-sm mb-4">
                            опали во временную петлю.</p>
                    </div>
                </div>
                <div id="slot-bellman-viz" class="mt-8"></div>
            </div>
        </div>
</section>
},
{
    id: "floyd",
    title: "16. Графы. Поиск кратчайшего пути. Флойд",
    type: "html",
    category: "Графы. Пути",
    content: `

```

```

{
  id: "johnson-algo",
  title: "17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)",
  type: "html",
  category: "Графы. Пути",
  content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Алгоритм
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-400">17. Графы. Пути
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-amber-400">Алгоритм Джонсона
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-xl font-mono text-white">
          // Алгоритм Джонсона
        </p>
      </div>
      <div class="grid grid-cols-1 gap-6 mb-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">
            ⚠ Суть перевзвешивания
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать <b>Джонсона</b>
            е пути остались теми же.
          </p>
          <ol class="text-sm text-slate-300 space-y-4">
            <li>Добавляем <b>фиктивную</b> вершину
            <li>Запускаем от неё <b>медленный Дейкстра</b> «потенциалы»
            <li>Меняем старые веса: новое расстояние = старое + потенциал
            <li><b>Магия!</b> Все веса теперь минимальные (потенциалы сокращаются).
            <li>Теперь смело запускаем <b>быстрый Дейкстра</b>
          </ol>
        </div>
      </div>
    </div>
  </div>
`

```

```

</section>`
  },
  {
    id: "mst-prima",
    title: "19. Графы. Основное дерево. Прима",
    type: "html",
    category: "Графы. Основные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">19
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">Основное дерево
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Пример графа
        <p class="text-xl font-mono text-white">
          которое торчит из посещенных в непосещенных
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            * Аналогия: Заражение вирусом
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ли вирус). Мы стартуем из одного города, и
            Сеть растет только из одного связного куска
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`
  },
  {
    id: "mst-boruvka",

```

```
.....
.....

import { Chapter } from '../..types';

export const tickets21to24: Chapter[] = [
  {
    id: "string-kmp",
    title: "21. Префикс-функция (п) – Взгляд назад (КМП)",
    type: "html",
    category: "Строки",
    content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-1">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
        сом, оканчивающимся в позиции i.</p>
      </div>

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 i
rder border-emerald-500 shadow-md">
            ♂ Аналогия 1: Поиск без отка
          </div>
          <p class="text-slate-300 text-sm mb">
            Мы идём по строке слева направо.
строки СЕЙЧАС закончился под моим пальцем?»
            Представь, ты ищешь в тексте слова
x</code>. Тупой алгоритм начнет искать зано
</code>, а это начало нашего слова! Не надо
```

```

        </div>
    </details>
    <details class="bg-slate-900/40 rounded-lg"
        <summary class="p-4 font-bold text-slate-300 border-slate-700/50">
            Код C++ (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300"
            <pre class="bg-slate-950 p-4 rounded"
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</section>`
    },
    {
        id: "string-z-func",
        title: "22. Z-функция — Сдвиг и наложение",
        type: "html",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">

```

```

.....
    <div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; padding: 10px; margin: 10px 0;">
        <p>Тот же пример: abcbabcd</p>
        <ul class="list-disc pl-5 space-y-2">
            <li><code>Z[0]</code> – не считаем, так как <b>b</b> не равен <b>b</b>
            <li><code>Z[1]</code> (<b>bcbabcd</b> <code>[1] = 0</code>)</li>
            <li><code>Z[2]</code> (<b>cbcbabcd</b> <code>[2] = 1</code>)</li>
            <li><code>Z[3]</code> (<b>abcbabcd</b> <code>[3] = 2</code>)</li>
        </ul>
        ? Нет, в начале 'a', тут 'd'. Совпало 3 буквы.
        <li><code>Z[4], Z[5], Z[6]</code> (они не совпадают)</li>
    </div>
</details>
<details class="bg-slate-900/40 rounded-lg p-5" style="border: 1px solid #ccc; margin-top: 10px;">
    <summary class="p-4 font-bold text-slate-200" style="border-bottom: 1px solid #ccc; padding-bottom: 5px;">
        Код C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; padding: 10px; margin-top: 5px;">
        <pre class="bg-slate-950 p-4 rounded" style="border: none; margin: 0;">
int l = 0, r = 0;
for (int i = 1; i <= n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] <= n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
        </pre>
    </div>
</div>
</details>
</div>
</section>
},
{
    id: "aho-corasick",
    title: "23. Алгоритм Ахо-Корасик",
    type: "html",
    category: "Строки",

```

```

</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
  <summary class="p-4 font-bold text-slate-400 outline-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <pre class="bg-slate-950 p-4 rounded overflow-x-auto">
int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch);
  }
  return t[v].link;
}

int go(int v, char c) {
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v), t[v].pch);
  }
  return t[v].go[c];
}</pre>
    </div>
  </details>
</div>
</section>
},
{
  id: "complexity-classes",
  title: "24. Классы сложности, сведение задач",
  type: "html",
  category: "Теория сложности",
  content: `
<section id="complexity-classes" class="mb-12 scroll-mt-1">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1">

```


ФАЙЛ 25/77: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМ

```
import { Chapter } from "../../types";
```

```
export const tickets6to13: Chapter[] = [
  {
```

```
    id: "graph-dfs-bfs",
```

```
    title: "6. Графы. Представление, DFS, BFS",
```

```
    type: "html",
```

```
    description: "Представление графов и базовые обходы
```

```
    category: "Графы. База",
```

```
    content: `
```

```
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
```

```
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
```

```
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
```

```
</div>
```

```
<div class="space-y-8">
```

```
  /* База представлений */
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
    <h3 class="text-xl font-bold text-slate-300">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
      <p class="text-lg text-slate-300 italic">
```

```
      <p class="text-xl font-mono text-white">
```

```
      (V + E).</p>
```

```
    </div>
```

```
    <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
      <div class="bg-slate-800 p-6 rounded-lg">
```

```
        <div class="absolute -top-3 left-4">
```

```
        rder border-emerald-500 shadow-md">
```

```
          Аналогия: Матрица = Таблица
```

```
        </div>
```

```
        <p class="text-slate-300 text-sm mb-4">
```

```
        глупо, но проверка "знает ли Вася Петю" мн
```

```
.....
    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4
border border-emerald-500 shadow-md">
      BFS (Поиск в ширину) = Волна
    </div>
    <p class="text-slate-300 text-sm mb"
      <b>Что делает:</b> Концентричес
Работает через Очередь (FIFO). Продвигается
    </p>
    <ul class="text-xs text-emerald-300"
      <li><b>Где используется:</b></li>
      <li>Кратчайший путь в Невзвешенн
      <li>Алгоритм Ахо-Корасик (сборк
      <li>Алгоритм Форда-Фалкерсона /
      <li>Двухнаправленный поиск для у
    </ul>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg"
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код обходов C++ (Основа)
  </summary>
  <div class="grid grid-cols-1 md:grid-co
    <pre class="bg-slate-950 p-3 rounded
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) {
        if (!vis[u]) dfs(u);
    }
}</pre>

    <pre class="bg-slate-950 p-3 rounded
// BFS (Очередь)
```

```

</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
      Аналогия 1: Карта мира
    </div>
    <p class="text-slate-300 text-sm mb-6">
      . Границы не пересекаются в одной точке по-
      ы не сливались цветами, Всегда можно всего
    </div>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "graph-components",
    title: "8. Графы. Компоненты связности",
    type: "html",
    description: "Нахождение кусков графа",
    category: "Графы. База",
    content: `
<section id="graph-components" class="mb-12 scroll-mt-1">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-emerald-400">
        <div class="bg-slate-900 p-4 rounded-lg mb-1">
          <p class="text-lg text-emerald-300 italic">
          <p class="text-xl font-mono text-white">
            ество компонент.</p>
        </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">

```

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
            еская сортировка – это расписание, которое
        >
    </div>
    <div class="bg-slate-900 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 right-4 border-rose-500 shadow-md">
            Ограничения: Циклы
        </div>
        <p class="text-slate-300 text-sm mb-4">
            получить опыт, нужна работа". Алгоритм выявления
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg">
        <summary class="p-4 font-bold text-slate-300">
            Душный мод: Код на C++ (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <p class="text-indigo-400 font-bold">
                <div class="bg-slate-950 p-4 rounded">
                    <pre class="text-xs font-mono">
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int to : adj[v]) {
        if (!visited[to]) {
            dfs(to);
        }
    }
    ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

```

```
.....
<p class="text-slate-300 text-sm mb-4">
  <strong>«Это просто цикл?» – Her!</strong>
</p>
<ul class="list-disc list-inside text-sm">
  <li>Если у тебя есть цикл <code>А → Б →
  <li>Если ты добавишь вершину <strong>Г</strong>,
  етыре вершины – одна большая SCC</strong>.
  <li>Правило: <b>Если два цикла имеют хо
</ul>

<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 i
    rder border-emerald-500 shadow-md">
      Аналогия: Интриги в офисе
    </div>
    <p class="text-slate-300 text-sm mb">
      Сильная связность – это "клуб с
      ни в одной SCC. Добавить курьера Гошу, котор
      йти наружу (к директору), но обратно в клуб
    </p>
  </div>

  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 i
    border-rose-500 shadow-md">
      Связь с Билетом 12
    </div>
    <p class="text-slate-300 text-sm mb">
      <b>SCC (Билет 10)</b> склеивает
    </p>
    <b>Точки сочленения (Билет 12)</b>
    ость</strong> монолита. Вырвешь точку сочле
    </p>
  </div>
</div>

<details class="bg-slate-900/40 rounded-lg">
```

```

и fup[v] > tin[u].</p>
</div>
<div class="grid grid-cols-1 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
      Аналогия 1: Единственный мост
    </div>
    <p class="text-slate-300 text-sm mb-4">
      ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ :
    </p>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает связность",
    category: "Графы. Продвинутое",
    content: `
<section id="graph-articulation" class="mb-12 scroll-mt">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
        Точки сочленения
      </h2>
    </span>
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-rose-400">
        Точка сочленения
      </h3>
      <p class="text-slate-300">
        Точка сочленения — это вершина графа, удаление которой увеличивает число компонент связности.
      </p>
    </div>
  </div>
</div>

```

```

</div>

<details class="bg-slate-900/40 rounded-lg border-1
  <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
      Душный мод: Код и корень DFS (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-4">
      <p class="text-blue-400 font-bold">Особый случай (если нет ребёнка, то выше неё прыгать некуда). Поэтому для него у него <strong>больше 1 независимого ребенка</strong>
    </p>
    <div class="bg-slate-950 p-4 rounded-lg border-1">
      <pre class="text-xs font-mono text-slate-400">
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
</pre>
      </div>
    </div>
  </div>

```

Универсальное пособие • полный исходный код

```
.....
    <p class="text-lg text-indigo-300 italic">
    <p class="text-xl font-mono text-white">
  </div>
  <div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg">
      <div class="absolute -top-3 left-4 i
      rder border-emerald-500 shadow-md">
        Аналогия 1: Рисование не отрыв
      </div>
      <p class="text-slate-300 text-sm mb
е сходится нечетное число линий, значит, из
а везде должно быть четное число {зашел-выш
    </div>
  </div>
</div>
</section>',
  },
};
```

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

```
.....
ФАЙЛ 26/77: src/components/ArticulationPointsViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
```

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```
interface Node {
  id: number;
  x: number;
  y: number;
  label: string;
```



```

.....
interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();
    const currentTin = Array(nodes.length).fill(-1);
    const currentLow = Array(nodes.length).fill(-1);

    const recordStep = (desc: string, u: number | null) => {
      newSteps.push({
        desc,
        visited: new Set(visited),
        ap: new Set(ap),
        currentNode: u,
        tin: [...currentTin],
        low: [...currentLow],
      });
    };

    recordStep("Начало DFS (Тарьян) для поиска точек со

    const dfs = (v: number, p: number = -1) => {
      visited.add(v);
      currentTin[v] = currentLow[v] = timer++;

```

```

        );
    } else if (p === -1) {
        recordStep(
            `Корень дерева DFS ${v} имеет <=1 ребенка, он
            v,
        );
    }
};

for (let i = 0; i < nodes.length; i++) {
    if (!visited.has(i)) {
        dfs(i);
    }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []);

const currentStep = steps[currentStepIndex] || {
    desc: "",
    visited: new Set(),
    ap: new Set(),
    currentNode: null,
    tin: [],
    low: [],
};

useEffect(() => {
    let interval: NodeJS.Timeout;
    if (isPlaying && currentStepIndex < steps.length -
        interval = setInterval(() => {
            setCurrentStepIndex((prev) => prev + 1);
        }, 1500);
    } else if (currentStepIndex >= steps.length - 1) {
        setIsPlaying(false);
    }
    return () => clearInterval(interval);
}

```

```
</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-3 g
```

```
<div className="lg:col-span-2 relative h-96 lg:i
```

```
<svg className="w-full h-full" viewBox="0 0 8
```

```
  { /* Edges */ }
```

```
  { edges.map((edge, i) => {
```

```
    const u = nodes.find((n) => n.id === edge
```

```
    const v = nodes.find((n) => n.id === edge
```

```
    const isBridge =
```

```
      (edge.u === 3 && edge.v === 4) ||
```

```
      (edge.u === 4 && edge.v === 3);
```

```
    return (
```

```
      <line
```

```
        key={"edge-" + i}
```

```
        x1={u.x}
```

```
        y1={u.y}
```

```
        x2={v.x}
```

```
        y2={v.y}
```

```
        stroke={isBridge ? "#f43f5e" : "#475568"}
```

```
        strokeWidth={isBridge ? "4" : "2"}
```

```
        strokeDasharray={isBridge ? "8,4" : ""}
```

```
      />
```

```
    );
```

```
  )}
```

```
  { /* Nodes */ }
```

```
  { nodes.map((node) => {
```

```
    const isCurrent = currentStep.currentNode
```

```
    const isVisited = currentStep.visited.has
```

```
    const isAp = currentStep.ap.has(node.id);
```

```
    const fill = isCurrent
```

```
      ? "#3b82f6"
```

```
      : isAp
```

```
        ? "#e11d48"
```



```

.....
        xt-slate-500"}` }
        >
        {step.desc}
      </div>
    )))
  </div>
</div>
</div>
<div className="mt-4 shrink-0">
  <div className="w-full bg-slate-800 h-2 rounded" style={{
    <div
      className="bg-rose-500 h-full transition-all duration-300"
      style={{
        width: `${(currentStepIndex / Math.max(steps.length, 1)) * 100}%`
      }}
    />
  </div>
</div>
</div>
</div>
</div>
  );
};

```

ФАЙЛ 27/77: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';

```

```

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  iteration: number;
  checkingEdge: { u: string; v: string; w: number } | null;
  distances: { [key: string]: number };
}

```

```

};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
  const list: { [key: string]: { v: string; w: number } } = {};
  nodes.forEach(n => (list[n.id] = []));
  ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
  return list;
};
const adjList = getAdjList(edges);

const codeLines = [
  { line: 1, text: 'function bellman_ford(graph, start)' },
  { line: 2, text: '  dist = {v: ∞}; dist[start] = 0;' },
  { line: 3, text: '  for i from 1 to V - 1: // |V| - 1 iterations' },
  { line: 4, text: '    for (u, v, weight) in E: // E = edges' },
  { line: 5, text: '      if dist[u] + weight < dist[v]' },
  { line: 6, text: '        dist[v] = dist[u] + weight;' },
  { line: 7, text: '  for (u, v, weight) in E: // Проверка на отрицательный цикл' },
  { line: 8, text: '    if dist[u] + weight < dist[v]' },
  { line: 9, text: '      return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
  { line: 10, text: '  return dist' },
];

useEffect(() => {
  const simulationSteps: Step[] = [];
  let dist: { [key: string]: number } = { S: 0, A: 99 };
  let prev: { [key: string]: string | null } = { S: null };

  simulationSteps.push({
    iteration: 0, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: ' Оура заведена. Начинаем с Базы (S). Расстояние до А: 99',
    activeCodeLine: 2,
  });

  const vCount = nodes.length;

```

```

.....
    cycleFound = true;
    simulationSteps.push({
      iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
      distances: { ...dist }, previous: { ...prev },
      log: `  ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}
      activeCodeLine: 9,
    });
    break;
  }
}

if (!cycleFound) {
  simulationSteps.push({
    iteration: vCount, checkingEdge: null,
    distances: { ...dist }, previous: { ...prev },
    log: '  Проверка завершена. Ни одно ребро не имеет двух соседей.
    activeCodeLine: 10,
  });
}

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

```

```
</div>
```

```
{currentStep.hasNegativeCycle && (
  <div className="absolute top-3 right-3 bg-r
    animate-pulse shadow-lg shadow-rose-600/50
    ▲ Отрицательный цикл!
  </div>
)}
```

```
<svg className="w-full h-auto max-h-[500px]"
  <defs>
    <marker id="arrow-bf-normal" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
    <marker id="arrow-bf-active" markerWidth=
    <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
    <marker id="arrow-bf-cycle" markerWidth=
    path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
  </defs>
```

```
{edges.map((edge, idx) => {
  const uNode = nodes.find((n) => n.id ===
  const vNode = nodes.find((n) => n.id ===
  const isChecking = currentStep.checkingEd
  v;
  const isNeg = edge.w < 0;
  const isCyclePart = hasCycle && ((edge.u
  && edge.v === 'D'));
```

```
let strokeColor = '#475569';
let marker = 'url(#arrow-bf-normal)';
if (isChecking) { strokeColor = '#f59e0b'
else if (currentStep.hasNegativeCycle &&
```

```
return (
  <g key={idx}>
    <line x1={uNode.x} y1={uNode.y} x2={v
asNegativeCycle && isCyclePart) ? 4 : 2} ma
g ? '4,4' : 'none' />
```



```

.....
    <div key={u} className="flex flex-wrap gap-2
    -colors">
      <span className="font-bold text-white">
        {adjList[u].map((edge, idx) => {
          const isChk = currentStep.checkingEdge === edge
          return (
            <span key={idx} className={`px-2 py-1
            edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-rose-900/80 text-rose-300'}>
              {edge.v} ({edge.w})
            </span>
          )
        })}
      </div>
    )}
  </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  {/* Таблица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div>
      <h4 className="text-lg font-bold text-rose-300">Таблица расстояний
      <div className="grid grid-cols-4 gap-2 text-rose-300">
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge === n
          return (
            <div key={n.id} className={`p-2 rounded
            : 'border-slate-700/60 bg-slate-800/40'}`}>
              <div className="text-xs text-slate-400">{n.v}</div>
              <div className={`font-mono font-bold text-rose-300`}>{d}</div>
            </div>
          )
        })}
      </div>
    </div>
  </div>
</div>

```

Универсальное пособие • полный исходный код

```
.....
    { id: 6, label: '6', x: 525, y: 250 },
  ];

const EDGES = [
  { from: 0, to: 1 }, { from: 0, to: 2 },
  { from: 1, to: 3 }, { from: 1, to: 4 },
  { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [1], 4: [1], 5: [1],
};

interface Step {
  activeLine: number;
  queue: number[];
  visited: number[];
  currentNode: number | null;
  logs: string;
}

const generateBfsSteps = (): Step[] => {
  const steps: Step[] = [];
  const queue: number[] = [];
  const visited = new Set<number>();

  // Init
  steps.push({
    activeLine: 1,
    queue: [],
    visited: [],
    currentNode: null,
    logs: "Инициализация: начинаем с корня (0).",
  });

  queue.push(0);
  visited.add(0);
  steps.push({
```

```

.....
        queue.push(n);

        steps.push({
            activeLine: 9,
            queue: [...queue],
            visited: Array.from(visited),
            currentNode: curr,
            logs: `Помечаем ${n} как посещенную и добавляем в очередь.`,
        });
    }
}
} else {
    steps.push({
        activeLine: 6,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Соседей нет или все уже посещены.`,
    });
}
}

steps.push({
    activeLine: 12,
    queue: [],
    visited: Array.from(visited),
    currentNode: null,
    logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
    const [stepIdx, setStepIdx] = useState(0);
    const [isPlaying, setIsPlaying] = useState(false);

```

```

</button>
<button
  onClick={() => setIsPlaying(p => !p)}
  className="p-2 text-blue-400 hover:text-blue-600"
  title={isPlaying ? 'Пауза' : 'Воспроизведение'}
>
  {isPlaying ? <Pause className="w-5 h-5" /> : <Play className="w-5 h-5" />}
</button>
<button
  onClick={handleNext}
  disabled={stepIdx >= STEPS.length - 1}
  className="p-2 text-emerald-400 hover:text-emerald-600 disabled:opacity-50 disabled:cursor-not-allowed"
  title="Далее"
>
  <StepForward className="w-5 h-5" />
</button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
  {/* Визуал графа */}
  <div className="bg-slate-800 border border-slate-700 p-4 min-h-[350px]">
    <svg className="w-full h-full min-h-[350px]">
      {/* Рисуем граф */}
      {EDGES.map((e, idx) => {
        const n1 = NODES.find(n => n.id === e.from)
        const n2 = NODES.find(n => n.id === e.to)
        const isVisited = step.visited.includes(n1.id)
        return (
          <line
            key={idx}
            x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.y}
            className={`transition-all duration-500 ${isVisited ? 'stroke-emerald-400' : 'stroke-slate-600'} stroke-2`}>
          </line>
        )
      })}
    </svg>
  </div>

```

```

        />
        <text
          x={n.x} y={n.y}
          textAnchor="middle"
          dy=".3em"
          className="fill-white font-bold text-slate-400"
        >
          {n.label}
        </text>
      </g>
    );
  }}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">|||ar {step}</span>
    </div>
    <div className="p-4">
      {l
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visited.add(start);' },
        { line: 3, code: '}' },
        { line: 4, code: 'while not queue.isEmpty()' },
        { line: 5, code: '  node = queue.pop()' },
        { line: 6, code: '  for neighbor in graph[node]:' },
        { line: 7, code: '    if neighbor not in visited:' },
        { line: 8, code: '      visited.add(neighbor)' },
        { line: 9, code: '      queue.push(neighbor)' },
      ].map((cl) => {
        const isActive = step.activeLine === cl.line
        return (

```

ФАЙЛ 29/77: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string; caption: string; borderColor: string; captionColor: string }> = {
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: 'Добрый — только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Фура по болоту',
    caption: 'Тяжёлая, но ей пофиг на ямы!',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Все ко Всем Флойд',
    caption: 'Все связаны со всеми!',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};
```

```
export default function ChapterImage({ vizType }: { vizType: string }) {
  const info = imageMap[vizType];
  if (!info) return null;
```

```

disabled={!prev}
onClick={() => prev && onGo(prev.id)}
title={prev ? prev.title : "Это первая тема"}
className="flex items-center gap-1 px-2.5 py-1
  :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
>
  <ChevronLeft className="w-4 h-4" />
  <span className="hidden sm:inline">Назад</span>
</button>
<span className="text-slate-500 font-mono tabular-nums" style={{flex: 1}}>
  {index + 1} / {total}
</span>
<button
  disabled={!next}
  onClick={() => next && onGo(next.id)}
  title={next ? next.title : "Это последняя тема"}
  className="flex items-center gap-1 px-2.5 py-1
    :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
  >
    <span className="hidden sm:inline">Вперёд</span>
    <ChevronRight className="w-4 h-4" />
  </button>
</div>
);
}

return (
  <nav className="grid grid-cols-1 sm:grid-cols-2 gap-2" style={{width: 100%;}}>
    {prev ? (
      <button
        onClick={() => onGo(prev.id)}
        className="group text-left p-4 rounded-xl bg-white
          transition-all"
        >
          <div className="flex items-center gap-1.5 text-slate-600" style={{width: 100%;}}>
            <ChevronLeft className="w-3.5 h-3.5" /> Пре
          </div>
        </button>
      ) : null}
  </nav>
);

```

```

.....
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
}

const isTouch = () => typeof window !== "undefined" &&

export const ChapterView: React.FC<Props> = ({ chapter,
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);

  useTermHints(contentRef, [chapter.id]);

  const cancelHide = useCallback(() => {
    if (hideTimer.current) {
      window.clearTimeout(hideTimer.current);
      hideTimer.current = null;
    }
  }, []);

  const scheduleHide = useCallback(() => {
    cancelHide();
    hideTimer.current = window.setTimeout(() => setAnchor(
  }, [cancelHide]);

  useEffect(() => {
    setAnchor(null);
  }, [chapter.id]);

  const open = useCallback(
    (el: HTMLElement) => {
      const termId = el.dataset.termId;
      if (!termId) return;

```



```

.....
      className="prose prose-invert max-w-none"
      onMouseOver={handlePointerOver}
      onMouseOut={handlePointerOut}
      onClick={handleClick}
      onFocus={handleFocus}
      dangerouslySetInnerHTML={{ __html: chapter.content }}
    />

    <p className="mt-6 flex items-start gap-2 text-
      ed-lg px-3 py-2">
      <MousePointerClick className="w-4 h-4 shrink-0" />
      <span>
        Подчёркнутые термины – интерактивные: наведи
        объяснение на пальцах, мини-анимацию и кнопк
      </span>
    </p>

    {viz && (
      <section id="chapter-viz" className="mt-8 scr
        <div className="flex items-center justify-b
          <h3 className="flex items-center gap-2 te
            <Sparkles className="w-4 h-4 text-indigo
              Демонстрация: {viz.title}
            </h3>
            <button
              onClick={() => onOpenSimulator(chapter.
              className="flex items-center gap-1.5 te
            >
              <Maximize2 className="w-3.5 h-3.5" /> P
            </button>
          </div>
          {viz.hint && <p className="text-xs text-sla
            <viz.Component />
          </section>
        )}

    <ChapterNav prev={prev} next={next} index={inde

```

Универсальное пособие • полный исходный код

```

.....
{ line: "          if (visited[to]) {", highlight: "normal" },
{ line: "              low[v] = min(low[v], tin[to]);", highlight: "normal" },
{ line: "          } else {", highlight: "normal" },
{ line: "              dfs(to, v);", highlight: "normal" },
{ line: "              low[v] = min(low[v], low[to]);", highlight: "normal" },
{ line: "          }", highlight: "normal" },
{ line: "          if (low[to] > tin[v]) {", highlight: "normal" },
{ line: "              // ЭТО МОСТ!", highlight: "normal" },
{ line: "          }", highlight: "normal" },
{ line: "      }", highlight: "normal" },
{ line: "  }", highlight: "normal" },
{ line: "}", highlight: "normal" },
};

```

```

interface DfsStep {
  currentNode: number | null;
  visitedIndices: number[];
  tin: Record<number, number>;
  low: Record<number, number>;
  stack: number[];
  bridges: string[];
  log: string;
  activeEdge: [number, number] | null;
  backEdge: [number, number] | null;
  activeCodeLine: number[];
}

```

```

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

```

```

const GRAPH_EDGES = [

```

```

.....
},
{
  currentNode: 2, visitedIndices: [0,1,6,2],
  tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, sta
  activeEdge: [1,2], backEdge: null,
  log: "Из В идём в С. tin[C]=4, low[C]=4.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 2, visitedIndices: [0,1,6,2],
  tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, sta
  activeEdge: null, backEdge: [2,0],
  log: "Видим соседа А (уже посещён)! Обратное ребро
  activeCodeLine: [7,8,9]
},
{
  currentNode: 3, visitedIndices: [0,1,6,2,3],
  tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3
  activeEdge: [2,3], backEdge: null,
  log: "Идём С→D. tin[D]=5, low[D]=5.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 4, visitedIndices: [0,1,6,2,3,4],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2
  activeEdge: [3,4], backEdge: null,
  log: "D→E. tin[E]=6, low[E]=6.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
  tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6
  activeEdge: [4,5], backEdge: null,
  log: "E→F. tin[F]=7, low[F]=7.",
  activeCodeLine: [4,5,7,10,11]
},
{
  currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],

```

```

.....
    activeCodeLine: [11,13,16,17,18]
  },
  {
    currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:1},
    activeEdge: null, backEdge: null,
    log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм закончен.",
    activeCodeLine: []
  },
];

export function DfsBridgesSimulator() {
  const [dfsIdx, setDfsIdx] = useState(0);
  const [dfsAuto, setDfsAuto] = useState(false);
  const dfsTimer = useRef<NodeJS.Timeout | null>(null);

  useEffect(() => {
    if (dfsAuto) {
      dfsTimer.current = setInterval(() => {
        setDfsIdx(prev => {
          if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
          return prev + 1;
        });
      }, 2500);
    }
    return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
  }, [dfsAuto]);

  const step = DFS_STEPS[dfsIdx];

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h4 className="text-base font-bold text-white">Шаг {step}
        <div className="flex items-center gap-1 bg-slate-800 p-1">
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
            disabled={dfsIdx === 0}
            className="p-1.5 hover:bg-slate-700 rounded">Автоматически
      </div>
    </div>
  );
}

```

```

.....
    let sc = "#475569", sw = 2, dash = "none"
    if (bridge) { sc = "#eab308"; sw = 4; }
    else if (activeTree) { sc = "#10b981"; sw
    else if (activeBack) { sc = "#f43f5e"; sw
    else if (step.visitedIndices.includes(e.u
    return <line key={e.key} x1={u.x} y1={u.y
        stroke={sc} strokeWidth={sw} strokeDash
        className="transition-all duration-500"
    }}}
    {step.currentNode !== null && step.currentN
        const n = GRAPH_NODES.find(x => x.id ===
        if (!n) return null;
        return <circle cx={n.x} cy={n.y} r={18} f
    >;
    }}())}
    {GRAPH_NODES.map(n => {
        const cur = step.currentNode === n.id;
        const vis = step.visitedIndices.includes(
        const st = step.stack.includes(n.id);
        return <g key={n.id}>
            <circle cx={n.x} cy={n.y} r={12}
                fill={cur ? "#10b981" : st ? "#064e3b
                stroke={cur ? "#fff" : st ? "#34d399"
                strokeWidth={2.5}
                className="transition-all duration-300
            <text x={n.x} y={n.y+3} textAnchor="mid
        label}</text>
            {vis && (
                <
                    <rect x={n.x-18} y={n.y-28} width={
                "transition-all duration-300" />
                    <text x={n.x} y={n.y-20} textAnchor=
                } l:{step.low[n.id]||"?"}</text>
                </>
            )}
        </g>;
    }}}
</svg>

```

```

.....
    {step.stack.length === 0
      ? <div className="text-[10px] text-slate-400"
        : step.stack.map((id, i) => {
          <div key={id} className={`px-2 py-0
            i === step.stack.length-1
              ? "bg-emerald-600/20 text-emerald-500"
              : "bg-slate-900 text-slate-400"
            }`>Вершина {GRAPH_NODES.find(n=>n.id===id).name}
            <span className="text-slate-500 ml-2">{step.stack[i].name}
          </div>
        ))
    }
  </div>
</div>

{/* Log */}
<div className="bg-indigo-950/10 border border-indigo-500 p-4"
  <p className="text-[11px] text-slate-300 leading-tight">Шаг {step.id}
  <div className="mt-2 pt-2 border-t border-slate-700"
    <span className="text-slate-400">Мосты:</span>
    <span className="font-mono font-bold text-slate-300">
      {step.bridges.length > 0 ? step.bridges.map(b => b.name).join(' ') : ''}
    </span>
  </div>
</div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 33/77: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import { useState, useEffect } from 'react';
```

```

const adjList: { [key: string]: { v: string; w: number }[] } = {
  nodes.forEach(n => (adjList[n.id] = []));
  edges.forEach(e => {
    adjList[e.u].push({ v: e.v, w: e.w });
  });
};

```

```

const codeLines = [
  { line: 1, text: 'function dijkstra(graph, start):' },
  { line: 2, text: '    dist = {v: ∞}; dist[start] = 0' },
  { line: 3, text: '    pq = PriorityQueue(); pq.push(start, 0)' },
  { line: 4, text: '    while not pq.empty():' },
  { line: 5, text: '        d, u = pq.pop() // Извлекаем минимальный элемент' },
  { line: 6, text: '        if d > dist[u]: continue' },
  { line: 7, text: '        for v, weight in graph.neighbors(u):' },
  { line: 8, text: '            if dist[u] + weight < dist[v]:' },
  { line: 9, text: '                dist[v] = dist[u] + weight' },
  { line: 10, text: '                pq.push(v, dist[v])' },
  { line: 11, text: '    return dist // Все кратчайшие пути найдены' },
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useState(0);
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
  const simulationSteps: Step[] = [];
  const dist: Record<string, number> = { A: 0, B: 999 };
  const visited: Record<string, boolean> = { A: false, B: false };
  const prev: Record<string, string | null> = { A: null, B: null };

  simulationSteps.push({
    currentNode: null, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited },
    log: ' Дейкстра готов к доставке. Начинаем из Пикет-Постов',
    activeCodeLine: 2,
  });
});

```

```

for (let iter = 0; iter < nodes.length; iter++) {

```

```

.....
        log: `    Улучшение (Релаксация)! Новое кратчайшее
        activeCodeLine: 9,
    });
  }
}
}

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: `    Дейкстра всё доставил! Все возможные кратчайшие
  activeCodeLine: 11,
});

setSteps(simulationSteps);
}, [1]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => {
      setCurrentStepIndex((prev) => prev + 1);
    }, 1500);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border
    { /* Шапка симулятора */ }
    <div className="flex flex-wrap items-center justify
      <div className="flex items-center gap-3">

```



```

<div className="w-full lg:w-2/3 bg-indigo-950/20
  justify-center min-h-[400px]">
  <div className="absolute top-3 left-3 bg-indigo-950/20
    font-bold shadow-sm">
    Шаг {currentStepIndex + 1} из {steps.length}
  </div>

  <svg className="w-full h-auto max-h-[500px]" >
    <defs>
      <marker id="arrow-dh-normal" markerWidth="10"
        <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
      </marker>
      <marker id="arrow-dh-active" markerWidth="10"
        <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
      </marker>
      <marker id="arrow-dh-opt" markerWidth="6"
        <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
      </marker>
    </defs>
    {/* Рёбра */}
    {edges.map((edge, idx) => {
      const uNode = nodes.find((n) => n.id === edge.u);
      const vNode = nodes.find((n) => n.id === edge.v);
      const isChecking =
        currentStep.checkingEdge === edge.id;
      const isOptimal = currentStep.previousEdge === edge.id;

      let marker = 'url(#arrow-dh-normal)';
      if (isChecking) marker = 'url(#arrow-dh-active)';
      else if (isOptimal) marker = 'url(#arrow-dh-opt)';

      return (
        <g key={idx}>
          <line
            x1={uNode.x} y1={uNode.y}
            x2={vNode.x} y2={vNode.y}
            stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b981' : '#475569'}
            stroke-width={isChecking ? 2 : isOptimal ? 1 : 1}
          />
          <marker
            href={marker}
            dx={vNode.x - uNode.x}
            dy={vNode.y - uNode.y}
            style={{position: 'absolute', left: 0, top: 0}}
          />
        </g>
      );
    })}
  </svg>

```

```

.....
      <text x={node.x} y={node.y + 5} fill=
        {node.id}
      </text>
      <text x={node.x} y={node.y - 28} fill=
"middle" className="bg-indigo-950 px-1 py-0
        {dist === 999 ? '∞' : `d=${dist}`}
      </text>
      <text x={node.x} y={node.y + 35} fill=
        {node.name.split(' ')[0]}
      </text>
    </g>
  );
  }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded-2xl" >
  <p className="font-semibold text-amber-400" >
  <p className="min-h-[40px] flex items-center" >
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/50" >
  <h4 className="text-lg font-bold text-emerald-400" >
  <div className="space-y-2.5 flex-1 overflow-y-hidden" >
    {Object.keys(adjList).map((u) => {
      const isCurrentNode = currentStep.currentNode === u;
      return (
        <div key={u} className={`p-3 rounded-lg ${
          isCurrentNode ? 'bg-emerald-900/60 text-white' : 'text-slate-300'
        }`} >
          <div className="flex items-center gap-2" >
            <span className={`px-2 py-0.5 rounded` +
              (isCurrentNode ? 'bg-emerald-900/60 text-white' : 'text-slate-300')} >
              {u}
            </span>
            <span className="text-slate-400" >→</span>
          </div>
        </div>
      );
    })}
  </div>

```

```

.....
    const dist = currentStep.distances[n.id]
    const prev = currentStep.previous[n.id]
    const vis = currentStep.visited[n.id]
    const isCurr = currentStep.currentName === n.name

    return (
      <tr key={n.id} className={`border-1
        isCurr ? 'bg-indigo-950/60 font-medium' : ''}>
        <td className="py-2.5 px-3 flex items-center">
          <span className={`w-2 h-2 rounded-full ${vis ? 'bg-indigo-950/60' : 'bg-gray-500'} border-1 border-gray-300`} />
          {n.name}
        </td>
        <td className="py-2.5 px-3 font-medium">
          {dist === 999 ? '∞' : dist}
        </td>
        <td className="py-2.5 px-3 text-gray-500">
          {prev || '-'}
        </td>
      </tr>
    );
  }
}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
  <div>
    <h4 className="text-lg font-bold text-slate-200">Алгоритм
    <div className="bg-slate-950/80 rounded-lg p-4">
      {codeLines.map((item) => {
        const isActive = currentStep.activeCodeLine === item.line
        return (
          <div key={item.line} className={`py-1 px-2
            isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'}`>

```

Универсальное пособие • полный исходный код

```
.....
  /* 4 */ "      memo[n] = fibMemo(n - 1, memo) + fibMemo
  /* 5 */ "      return memo[n];",
  /* 6 */ "}"
};

export function DPVisualizer() {
  const [targetN, setTargetN] = useState<number>(5);
  const [steps, setSteps] = useState<DPStep[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState<boolean>(f
  const [speed] = useState<number>(800);
  const [activeTab, setActiveTab] = useState<'table' |

  useEffect(() => {
    const generatedSteps: DPStep[] = [];
    let stepId = 0;
    const memoMap: { [key: number]: number } = {};

    function simulateFib(n: number): number {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'call',
        desc: `Вызов fibMemo(${n}). Проверяем наличие в
        codeLine: 2,
        memoState: { ...memoMap }
      });

      if (n in memoMap) {
        generatedSteps.push({
          id: stepId++,
          n,
          action: 'memo_hit',
          desc: `✂ Кэш-хит! fibMemo(${n}) уже посчитано
          codeLine: 2,
          memoState: { ...memoMap }
        });
        return memoMap[n];
      }
    }
  });
}
```

.....

```

    const finalRes = simulateFib(targetN);

    generatedSteps.push({
      id: stepId++,
      n: targetN,
      action: 'done',
      desc: `  Расчет завершен! fibMemo(${targetN}) = $
      codeLine: 6,
      memoState: { ...memoMap }
    });

    setSteps(generatedSteps);
    setCurrentStepIndex(0);
    setIsPlaying(false);
  }, [targetN]);

  useEffect(() => {
    let timer: any = null;
    if (isPlaying && currentStepIndex < steps.length -
      timer = setTimeout(() => {
        setCurrentStepIndex(prev => prev + 1);
      }, speed);
    } else if (currentStepIndex >= steps.length - 1) {
      setIsPlaying(false);
    }
    return () => clearTimeout(timer);
  }, [isPlaying, currentStepIndex, steps, speed]);

  const currentStep = steps[currentStepIndex] || null;
  const memoState = currentStep?.memoState || {};

  return (
    <div className="bg-slate-900 rounded-2xl border bor
      /* Header & Controls */
    <div className="flex flex-col lg:flex-row lg:item
      <div>
        <div className="flex items-center gap-3 mb-2">

```

```

        title="⏮️ ⬅️"
      >
        <ChevronLeft className="w-5 h-5" />
      </button>
      <button
        onClick={() => setIsPlaying(!isPlaying)}
        className={`flex items-center gap-1.5 px-2 py-2
          ${isPlaying
            ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
            : 'bg-emerald-600 text-white hover:bg-emerald-700'}
        `}
      >
        {isPlaying ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4" />}
        {isPlaying ? 'Пауза' : 'Пуск'}
      </button>
      <button
        onClick={() => { setIsPlaying(false); setSteps([0]); }}
        disabled={currentStepIndex === steps.length - 1}
        className="p-2 text-slate-400 hover:text-slate-300"
        title="⏭️ ➡️"
      >
        <ChevronRight className="w-5 h-5" />
      </button>
    </div>
  </div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded-2xl" >
  <h4 className="text-xs font-bold uppercase tracking-wider" >DP Table</h4>
  <div className="flex flex-wrap items-center gap-2" >
    {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
      const isCached = memoState[num] !== undefined;
      const val = num <= 2 ? 1 : memoState[num];
      const isCurrent = currentStep?.n === num;

      return (
        <div
          className={`flex items-center gap-1 px-2 py-1 rounded
            ${isCurrent ? 'bg-emerald-600 text-white' : 'bg-slate-800 text-slate-300'}
          `
        >
          {num}
          {isCached ? <Check className="w-3 h-3" /> : <X className="w-3 h-3" />}
        </div>
      );
    })}
  </div>
</div>

```

```

.....
      ? 'border-emerald-500 text-emerald-400 bg
      : 'border-transparent text-slate-400 hove
    }` }
  >
    <Eye className="w-4 h-4" />
    Стен-бай-стен состояние
  </button>
  <button
    onClick={() => setActiveTab('code')}
    className={`flex items-center gap-2 px-6 py-3
      activeTab === 'code'
        ? 'border-emerald-500 text-emerald-400 bg
        : 'border-transparent text-slate-400 hove
    }` }
  >
    <Code className="w-4 h-4" />
    Интерактивный код
  </button>
</div>

{/* Tab Content */}
<div className="min-h-[320px] flex flex-col justify
  {activeTab === 'table' && (
    <div className="bg-slate-950 p-6 rounded-2xl
      <h5 className="font-bold text-white text-sm
        <span className="text-emerald-400"> </span>
      </h5>
      <p className="text-sm text-slate-400">
        В классической рекурсии для  $N=\{targetN\}$  п
        вычисленное значение в таблице, мы момента
      </p>
      <div className="p-4 bg-slate-900 rounded-xl
        t-slate-300">
        <span>Всего шагов симуляции: {steps.length}
        <span className="text-emerald-400 font-bo
      </div>
    </div>
  )}

```

```
.....
    <span>Шаг {currentStepIndex + 1} из {steps.length}
  </div>
  <div className="h-2 bg-slate-950 rounded-full overflow-hidden">
    <div
      className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
      style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
    ></div>
  </div>
</div>
</div>
);
}
```

ФАЙЛ 35/77: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import { useState } from "react";
import { Undo } from "lucide-react";
```

```
const C1_NODES = [
  { id: 0, label: "A", x: 190, y: 60 },
  { id: 1, label: "B", x: 280, y: 140 },
  { id: 2, label: "C", x: 100, y: 140 },
  { id: 3, label: "D", x: 280, y: 250 },
  { id: 4, label: "E", x: 100, y: 250 }
];
```

```
const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];
```

```
export function EulerSimulator() {
```



```

.....
    } else {
      setClMsg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! C
    }
    setClDone(true);
  } else {
    setClMsg(`Пройдено рёбер: ${nextEdges.length} / $
  }
};

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
      <button onClick={resetCl} className="flex items
        old border border-slate-700 transition-all">
        <Undo className="h-3.5 w-3.5" /> Сброс
      </button>
    </div>

    <div className="bg-slate-950 rounded-xl p-4 borde
      -hidden">
      <div className="absolute inset-0 bg-[radial-gra
      <svg className="w-full h-[260px] z-10 relative"
        {Cl_EDGES.map((e, i) => {
          const u = Cl_NODES.find(n => n.id === e.u)!
          const v = Cl_NODES.find(n => n.id === e.v)!
          const key = `${Math.min(e.u,e.v)}-${Math.ma
          const used = clVisitedEdges.includes(key);
          return <line key={i} x1={u.x} y1={u.y} x2={
            stroke={used ? "#6366f1" : "#475569"}
            strokeWidth={used ? 4 : 2}
            className="transition-all duration-300" />
        })}
        {Cl_NODES.map(n => {
          const isLast = clPath[clPath.length-1] ===
          const visited = clPath.includes(n.id);
          return <g key={n.id} className="cursor-poin
            {isLast && <circle cx={n.x} cy={n.y} r={1

```

```

.....
    id: number;
    question: string;
    options: string[];
    correctAnswer: number;
    explanation: string;
}

const quizQuestions: Question[] = [
  {
    id: 1,
    question: 'Что мы делаем в алгоритме Краскала, если  

      ребро, красным) цвет?',
    options: [
      'Мы радуемся и добавляем его в минимальное остовное дерево',
      'Мы перекрашиваем их в синий цвет и делим граф по  

      компонентам связности',
      'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
      'Мы вызываем алгоритм Дейкстры для поиска нового ребра'
    ],
    correctAnswer: 2,
    explanation: 'Верно! Если вершины уже в одном клане  

      это нарушит структуру дерева.'
  },
  {
    id: 2,
    question: 'Почему алгоритм Дейкстры впадает в ступор?',
    options: [
      'Потому что он написан только для положительных весов',
      'Он жадный и считает, что каждый следующий шаг дает  

      минимальный путь к закрытым вершинам.',
      'Отрицательные ребра создают гравитационный коллапс',
      'Дейкстра просто не любил отрицательные балансы на ребрах'
    ],
    correctAnswer: 1,
    explanation: 'Именно! Дейкстра уверен: если ты пришел  

      в вершину, то для чего нужен алгоритм Беллмана-Форда.'
  },
  {
    id: 3,
  }
]

```

Универсальное пособие • полный исходный код

```
.....
export const ExpressQuiz: React.FC = () => {
  const [currentQIndex, setCurrentQIndex] = useState<number>(0);
  const [selectedOption, setSelectedOption] = useState<string>('');
  const [isAnswered, setIsAnswered] = useState<boolean>(false);
  const [score, setScore] = useState<number>(0);
  const [showResults, setShowResults] = useState<boolean>(false);

  const currentQ = quizQuestions[currentQIndex];

  const handleSelectOption = (index: number) => {
    if (isAnswered) return;
    setSelectedOption(index);
    setIsAnswered(true);
    if (index === currentQ.correctAnswer) {
      setScore(prev => prev + 1);
    }
  };

  const handleNext = () => {
    if (currentQIndex + 1 < quizQuestions.length) {
      setCurrentQIndex(prev => prev + 1);
      setSelectedOption('');
      setIsAnswered(false);
    } else {
      setShowResults(true);
    }
  };

  const handleRestart = () => {
    setCurrentQIndex(0);
    setSelectedOption('');
    setIsAnswered(false);
    setScore(0);
    setShowResults(false);
  };

  if (showResults) {
    return (

```

```
.....
    <span className="bg-purple-600 text-white px-4">
      Экспресс-тест
    </span>
    <h3 className="text-xl font-bold text-white">
  </div>
  <span className="text-sm font-mono text-slate-400">
    Bonpoc {currentQIndex + 1} / {quizQuestions.length}
  </span>
</div>

<div className="mb-8">
  <h4 className="text-xl md:text-2xl font-bold text-slate-800">
    {currentQ.question}
  </h4>

  <div className="space-y-4">
    {currentQ.options.map((option, idx) => {
      const isSelected = selectedOption === idx;
      const isCorrect = idx === currentQ.correctAnswer;

      let optionStyle = "bg-slate-800/80 hover:bg-slate-700/80";
      if (isAnswered) {
        if (isCorrect) {
          optionStyle = "bg-emerald-950/80 border-emerald-500";
        } else if (isSelected) {
          optionStyle = "bg-rose-950/80 border-rose-500";
        } else {
          optionStyle = "bg-slate-800/40 border-slate-700";
        }
      }
    })}

    return (
      <div
        key={idx}
        onClick={() => handleSelectOption(idx)}
        className={"flex items-start gap-4 p-4"}
      >
        <div className="mt-0.5 shrink-0">
```

```
.....
        (!isAnswered
          ? "bg-slate-800 text-slate-500 border border-slate-800"
          : "bg-indigo-600 hover:bg-indigo-500 active:bg-indigo-700")
      )
    >
      {currentQIndex + 1 < quizQuestions.length ? 'next' : 'finish'}
    </button>
  </div>
</div>
);
};
```

ФАЙЛ 37/77: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....

import { useState, useEffect } from 'react';

interface Step {
  k: number; i: number; j: number;
  matrix: number[][][]; updated: boolean;
  log: string; activeCodeLine: number;
}

export default function FloydViz() {
  const [steps, setSteps] = useState<Step[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5'];
  const nodesSvg = [
    { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
    { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
    { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
    { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
    { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
  ];
}
```

```

simulationSteps.push({
  k: -1, i: -1, j: -1, matrix: dist.map(r => [...r]),
  log: '  Инициализация матрицы прямых путей,  $\infty$  (999)',
  activeCodeLine: 2,
});

const N = 7;
for (let k = 0; k < N; k++) {
  simulationSteps.push({
    k, i: -1, j: -1, matrix: dist.map(r => [...r]),
    log: `+ Внешний цикл. Разрешаем пути через пром. узел ${k}`,
    activeCodeLine: 3,
  });

  for (let i = 0; i < N; i++) {
    for (let j = 0; j < N; j++) {
      if (i === j || i === k || j === k) continue;

      const oldVal = dist[i][j];
      const viaVal = dist[i][k] + dist[k][j];
      if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
        dist[i][j] = viaVal;
        simulationSteps.push({
          k, i, j, matrix: dist.map(r => [...r]),
          log: `  Улучшение пути [${i}]→[${j}] через пром. узел [${k}],  

             $\infty$ `,
          activeCodeLine: 7,
        });
      }
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
  log: '  Все пары отработаны. Алгоритм Флойда завершён.',
  activeCodeLine: 8,
});

```

```

.....
    <button onClick={() => { setIsPlaying(false);
      ; }} className="px-3 py-2 bg-emerald-600 te
    </div>
  </div>

  <div className="flex flex-col lg:flex-row gap-6 m
    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-indigo-950/2
      stify-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-indi
        ont-bold shadow-sm">
        {currentStep.k >= 0 && currentStep.k < 7 ?
      </div>

      <svg className="w-full h-auto max-h-[500px]"
        <defs>
          <marker id="arrow-fw-normal" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
          <marker id="arrow-fw-active" markerWidth=
          <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
          <marker id="arrow-fw-via" markerWidth="6"
          th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></f
        </defs>
        {Object.keys(adjList).flatMap((uStr) => {
          const u = parseInt(uStr);
          return adjList[u].map((edge) => {
            const uNode = nodesSvg.find((n) => n.id
            const vNode = nodesSvg.find((n) => n.id
            const isTarget = currentStep.i === u &&
            const isVia = (currentStep.i === u && c
            let strokeColor = '#475569'; let marker
            if (isTarget) { strokeColor = '#10b981'
            else if (isVia) { strokeColor = '#818cf8
            return (
              <g key={` ${u} - ${edge.v} `}>
                <line x1={uNode.x} y1={uNode.y} x2=
                markerEnd={marker} strokeDasharray={isTarg
                <circle cx={({uNode.x + vNode.x) / 2

```



```
.....
        </div>
      )})
    </div>
  </div>
</div>
</div>
</div>
};
}
```

.....
ФАЙЛ 38/77: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

.....
import { useState, useEffect, useMemo } from 'react';
import { Play, Pause, SkipForward, Undo, RefreshCw } from 'lucide-react';

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```
const nodes: Node[] = [
  { id: 'A', label: 'A', x: 200, y: 40 },
  { id: 'B', label: 'B', x: 100, y: 120 },
  { id: 'C', label: 'C', x: 300, y: 120 },
  { id: 'D', label: 'D', x: 50, y: 200 },
  { id: 'E', label: 'E', x: 150, y: 200 },
  { id: 'F', label: 'F', x: 250, y: 200 },
  { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```

.....
    steps.push({ type: 'backtrack', node: v, desc:
        ueOrStack: [...stack], visitedNodes: Array.from(
    }

    dfs(start);
    return steps;
}

function generateBFSSteps(start: string) {
    const steps: Action[] = [];
    const vis = new Set<string>();
    const q: string[] = [];

    q.push(start);
    vis.add(start);
    steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start}`,
        ray.from(vis)} });

    while (q.length > 0) {
        const v = q[0];
        steps.push({ type: 'process', node: v, desc: `Действуем с узлом ${v}`,
            q}, visitedNodes: [...Array.from(vis)] });
        q.shift();

        for (const u of adj[v]) {
            if (!vis.has(u)) {
                vis.add(u);
                q.push(u);
                steps.push({ type: 'visit', node: u, desc: `Визит в узел ${u}`,
                    tack: [...q], visitedNodes: [...Array.from(vis)] });
            }
        }
    }

    steps.push({ type: 'backtrack', node: '', desc: `Очищаем стек и возвращаемся к началу`,
        om(vis)} });

    return steps;
}

```

```

.....
      <button onClick={() => setMode("dfs")}
        className={`px-4 py-2 text-xs font-
text-slate-400 hover:text-slate-300`} >
        DFS (В глубину)
      </button>
      <button onClick={() => setMode("bfs")}
        className={`px-4 py-2 text-xs font-
"text-slate-400 hover:text-slate-300`} >
        BFS (В ширину / Волна)
      </button>
    </div>

    <div className="flex flex-col md:flex-row g
      {/* SVG Graph View */}
      <div className="bg-slate-900 border bor
ive min-h-[300px]">
        <svg className="w-full h-full max-
          {/* Edges */}
          {edges.map((e, i) => {
            const u = nodes.find(n => n
            const v = nodes.find(n => n
            // Check if edge is part of
            const edgeTraversed = isVis

            return (
              <line key={i} x1={u.x}
                className={`stroke-
500' : 'stroke-emerald-500') : 'stroke-slate
              );
            })}
          {/* Nodes */}
          {nodes.map(n => {
            const visited = isVisited(n
            const current = isCurrent(n

            let fillColor = "fill-slate
            let strokeColor = "stroke-s

```

```

.....
    <div className={`absolute -top-3 left-3
      ${mode === 'dfs' ? 'border border-emerald-500'}`}>
      {mode === 'dfs' ? 'Стек вызовов' : ''}
    </div>

    <div className="flex flex-col gap-2" style={
      {step.queueOrStack && step.queueOrStack.length > 0}
    }>
      <div key={idx} className={`w-full
        ${mode === 'dfs' ? 'bg-slate-900 text-emerald-200'}
        ${idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
          ? 'border border-emerald-500'}`}>
        {step.queueOrStack[idx]}
        {mode === 'dfs' ? 'Вызван: {item}' : ''}
        {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
          ? 'Вызван: {item}' : ''}
      </div>
    </div>
  </div>
</div>

<div className="flex flex-col sm:flex-row gap-2" style={
  {step.queueOrStack && step.queueOrStack.length > 0}
}>
  <div className="flex bg-slate-900 border border-emerald-500
    p-2 text-slate-400 hover:text-white hover:shadow-lg" style={
      {step.queueOrStack && step.queueOrStack.length > 0}
    }>
    <Undo strokeWidth={3} size={16} />
    <button onClick={() => setAutoPlay(!autoPlay)}
      justify-center min-w-[80px] ${mode === 'dfs' ? 'text-emerald-200'}
    >
      {autoPlay ? '<>' : 'Pause'}
  </div>
  <div className="flex flex-col gap-2" style={
    {step.queueOrStack && step.queueOrStack.length > 0}
  }>
    <div key={idx} className={`w-full
      ${mode === 'dfs' ? 'bg-slate-900 text-emerald-200'}
      ${idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
        ? 'border border-emerald-500'}`}>
      {step.queueOrStack[idx]}
      {mode === 'dfs' ? 'Вызван: {item}' : ''}
      {idx === (mode === 'dfs' ? step.queueOrStack.length - 1 : 0)
        ? 'Вызван: {item}' : ''}
    </div>
  </div>
</div>

```

Универсальное пособие • полный исходный код

```
.....
  emoji: string;
  symptom: string;
  priority: number; // 1 to 99 (Severity)
  animState: 'in' | 'idle' | 'winner' | 'out';
}

// Max-heap helpers for Patients
function siftUp(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  let idx = a.length - 1;
  while (idx > 0) {
    const parent = Math.floor((idx - 1) / 2);
    if (a[parent].priority < a[idx].priority) {
      [a[parent], a[idx]] = [a[idx], a[parent]];
      idx = parent;
    } else break;
  }
  return a;
}

function siftDown(arr: Patient[]): Patient[] {
  const a = arr.map(x => ({ ...x }));
  const n = a.length;
  let idx = 0;
  while (true) {
    let largest = idx;
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < n && a[l].priority > a[largest].priority) largest = l;
    if (r < n && a[r].priority > a[largest].priority) largest = r;
    if (largest !== idx) {
      [a[largest], a[idx]] = [a[idx], a[largest]];
      idx = largest;
    } else break;
  }
  return a;
}
```

```
.....  
})();
```

```
let uid = 300;
```

```
export const HeapViz: React.FC = () => {  
  const [heap, setHeap] = useState<Patient[]>([]);  
  const [customName, setCustomName] = useState('');  
  const [customPriority, setCustomPriority] = useState(1);  
  const [log, setLog] = useState<string[]>([]);  
  const [shakeEmpty, setShake] = useState(false);  
  const [busy, setBusy] = useState(false);  
  const [healingPatient, setHealing] = useState<Patient>({});  
  
  const addLog = (msg: string) => setLog(prev => [...prev, msg], {  
    merge: true,  
  });  
  
  // --- INSERT: Новый пациент поступает в приемный покой  
  const insertPatient = useCallback(({name: string, priority: number}) => {  
    if (busy || heap.length >= 15) {  
      addLog('⚠ Все палаты переполнены! Дождитесь выписки');  
      setShake(true);  
      setTimeout(() => setShake(false), 400);  
      return;  
    }  
    setBusy(true);  
  
    const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];  
    const finalName = name || preset.name.split(' ')[0];  
    const finalEmoji = preset.emoji;  
    const finalSymptom = preset.symptom;  
  
    const newPatient: Patient = {  
      id: `p${uid++}`,  
      name: finalName,  
      emoji: finalEmoji,  
      symptom: finalSymptom,  
      priority,  
      animState: 'in',  
    };  
    setHeap(prev => [...prev, newPatient]);  
    setHealing(newPatient);  
  }, [busy, heap, setHeap, setHealing, setLog, setShake, setBusy]);  
}
```

```

.....
const topPatient = heap[0];
setHealing(topPatient);

addLog(` СРОЧНО! Забираем самого тяжелого: ${topPa

// Step 1: Root lights up as winner
setHeap(prev => prev.map((x, i) => i === 0 ? { ...x

setTimeout(() => {
  // Step 2: Root exits the tree, tree reorganizes
  setHeap(prev => prev.map((x, i) => i === 0 ? { ..

  setTimeout(() => {
    // Run binary heap extraction
    setHeap(prev => {
      if (prev.length === 0) return [];
      const a = prev.map(x => ({ ...x }));
      a[0] = { ...a[a.length - 1] };
      a.pop();
      return siftDown(a).map(x => ({ ...x, animStat
    });

    // Patient gets healed in the bed
    setTimeout(() => {
      setHealing(prev => prev ? { ...prev, animStat
      addLog(` Успех! Пациент ${topPatient.name} п
      setTimeout(() => {
        setHealing(null);
        setBusy(false);
      }, 600);
    }, 800);

    }, 350);
  }, 650);
}, {busy, heap});

const reset = () => {
  setHeap(INITIAL PATIENTS.map(x => ({ ...x, animStat

```

```

.....
<div className="text-3xl"> </div>
<div>
  <h3 className="font-black text-emerald-400 text-center">
    Priority Queue</h3>
  <p className="text-xs text-slate-300 mt-1 leading-relaxed">
    В реанимации не важно, кто пришел раньше, а кто в
    остояние</b> (максимальный приоритет).
    Куча (Heap) автоматически перестраивает пац
    оказывался на самом верху дерева (в корне).
  </p>
</div>
</div>

{/* УПРАВЛЕНИЕ */}
<div className="flex flex-col lg:flex-row items-stretch gap-4">

  {/* Добавить случайного */}
  <button
    onClick={handleRandomInsert}
    disabled={busy || heap.length >= 15}
    className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-200 to-slate-400
      opacity-40 disabled:cursor-not-allowed text-slate-700 font-medium
      rounded-lg shadow-sm transition-all cursor-pointer focus:outline-none"
  >
    <span> Привезти по Скорой (INSERT)</span>
  </button>

  {/* Добавить кастомного */}
  <form onSubmit={handleCustomInsert} className="flex-1">
    <input
      type="text"
      required
      value={customName}
      onChange={e => setCustomName(e.target.value)}
      placeholder="Имя пациента"
      className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder-slate-400"
    />
  </form>
</div>

```



```

.....
    </button>
  </div>

  { /* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */ }
  <div className="grid grid-cols-1 lg:grid-cols-12" >

    { /* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */ }
    <div className={`lg:col-span-8 bg-slate-900 rounded-
      -[380px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`}>
      <div className="absolute top-4 left-4 text-[12px] font-medium">
        Сортировка по тяжести (Двоичная куча / Max-Heap)
      </div>

      <div className="relative w-full h-full min-h-[380px]">
        <svg className="absolute inset-0 w-full h-full" >
          {edges}
        </svg>

        {heap.map(({patient, idx}) => {
          const pos = nodePos(idx);
          const isRoot = idx === 0;
          return (
            <div
              key={patient.id}
              className={`
                absolute flex flex-col items-center
                -translate-x-1/2 -translate-y-1/2 transition-all
                ${patient.animState === 'in' ? 'animate-in' : ''}
                ${patient.animState === 'winner' ? 'animate-winner' : ''}
                ${patient.animState === 'out' ? 'animate-out' : ''}
                ${isRoot ? 'bg-rose-950/80 border-rose-500' : ''}
              `}
              style={{
                left: `${pos.x}%`,
                top: `${pos.y}%`,
                width: isRoot ? 60 : 52,
                height: isRoot ? 60 : 52,
              }}
            </div>
          )
        })}
      </div>
    </div>
  </div>

```

```

.....
      <span>Пациент: {healingPatient.name}
    </h4>
    <p className="text-emerald-400 text-x"
      Состояние: Стабилизация ({healingPa
    </p>
    <div className="flex justify-center g
      <span className="w-2 h-2 rounded-fu
      <Heart className="w-4 h-4 text-rose
    </div>
  </div>
</div>
) : {
  <div className="text-center text-slate-60
    <span className="text-5xl mb-2 block">
    <p className="text-xs font-bold font-mo
    <p className="text-[10px] mt-1">Ждём са
  </div>
  </div>
  <div className="w-full h-3 bg-gradient-to-r f
</div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
  <div className="text-[10px] font-mono text-slate
    {log.map((entry, idx) => {
      <div
        key={idx}
        className={`text-xs font-mono flex items-ce
      >
        {idx === 0 ? <span className="text-emerald-
        <span>{entry}</span>
      </div>
    })}
  </div>

```

Универсальное пособие • полный исходный код

```
.....  
export const Svg: React.FC<{ children: React.ReactNode;  
  <div className="w-full">  
    <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-  
      {children}  
    </svg>  
    {caption && <div className="text-[10px] text-slate-  
  </div>  
}>  
);
```

```
export const Node: React.FC<  
  x: number;  
  y: number;  
  r?: number;  
  fill: string;  
  label?: string | number;  
  stroke?: string;  
> = ({ x, y, r = 13, fill, label, stroke }) => (  
  <g style={{ transition: "all .35s ease" }}>  
    <circle cx={x} cy={y} r={r} fill={fill} stroke={str  
    {label !== undefined && (  
      <text x={x} y={y} textAnchor="middle" dominantBas  
        {label}  
      </text>  
    )}  
  </g>  
);
```

```
export const Edge: React.FC<  
  a: [number, number];  
  b: [number, number];  
  color?: string;  
  width?: number;  
  dashed?: boolean;  
> = ({ a, b, color = C.edge, width = 2, dashed }) => (  
  <line  
    x1={a[0]}  
    y1={a[1]}  
    x2={b[0]}
```

```

.....
    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
  </Svg>
);
};

```

/** Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    [ ["A", "B"], ["B", "C"] ],
    [ ["D", "E"] ],
    [ ["F", "G"], ["G", "H"], ["F", "H"] ],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return {
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => {
          <Edge key={` ${ci} - ${i}`} a={pos[u]} b={pos[v]}

```

```

.....
        fill={C.warn}
      >
        {w}
      </text>
    })
  </g>
);
}}
{Object.entries(pos).map(([k, [x, y]]) => (
  <Node key={k} x={x} y={y} label={k} fill={C.idle}
))}
</Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты — огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => (
        <g key={i}>
          <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
          <text x={startX + i * (boxW + 4) + boxW / 2} y={30}>
            {v}
          </text>
        </g>
      ))}
    </Svg>
  );
};

```

```

.....
};
const GRAPH_EDGES: [string, string][][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step,
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      }}})
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
      <Node key={k} x={x} y={y} label={k}
        fill={wave.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
        stroke={wave.includes(k) ? "#a5b4fc" : undefined}
      </Node>
    })}
    </Svg>
  );
};

```

```

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join("
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) < 2;
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
      }}})
    {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {

```

```

</Svg>
);
};

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь  

      <Edge a={60, 70} b={200, 70} color={improved ? "red" : "black"} />  

      <text x={130} y={54} textAnchor="middle" fontSize=14 />  

      <Node x={60} y={70} r={20} fill={C.active} label="3" />  

      <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />  

      <text x={60} y={106} textAnchor="middle" fontSize=14 />  

      <text x={200} y={106} textAnchor="middle" fontSize=14 />  

      <text x={130} y={22} textAnchor="middle" fontSize=14 />  

    </Svg>
  );
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : "Граф связен"} />
    {edges.map(({[u, v], i}) => {
      const isBridge = u === "C" && v === "D";
      if (isBridge && cut) return null;
      return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge && cut ? "red" : "black"} />
    })}
    {Object.entries(pos).map(({[k, [x, y]]) => (

```

```

.....
const taken = new Set(chosen.slice(0, step));
return (
  <Svg caption={`Жадно берём самые лёгкие рёбра без ц
    {edges.map((e, i) => {
      const on = taken.has(`${e.u}-${e.v}`);
      return (
        <g key={i}>
          <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
          <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
            textAnchor="middle" fontSize={9} fontWeigh
          </g>
        );
      });
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (<Node
  </Svg>
);
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45
  ];
  const edges: [string, string][][] = [[["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => (
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      )})}
      {Object.entries(pos).map(([k, [x, y]]) => (
        <Node key={k} x={x} y={y} label={k} fill={step =
      )})}
    </Svg>
  );
};
};

```



```

.....
<text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
{a.map((v, i) => (
  <g key={i}>
    <rect x={20 + i * 39} y={24} width={34} height={12}
      fill={show && i >= l && i <= r ? C.active : C.dim}>
    <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
  </g>
)}}
<text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
{pref.map((v, i) => (
  <g key={i}>
    <rect x={20 + i * 33} y={78} width={29} height={12}
      fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
      stroke={C.idleStroke} style={{ transition: "stroke 0.5s" }}>
    <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
  </g>
)}}
</Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для`}>
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={12}
            fill={C.dim}>
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      ))}
      {Array.from({ length: count }).map((_, i) => (
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.dim}>

```

Универсальное пособие • полный исходный код

```

.....
export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} height={44}
            fill={step > 0 && (idx < len || (idx > i - len)) ? C.fill : C.dim}
            style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle">{idx}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle">{len}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle">{i}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle">{step}</text>
        </g>
      ))}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
    <text x={130} y={106} textAnchor="middle" fontSize={12}>
      π[i] – длина самого длинного «префикс = суффикс»
    </text>
  </Svg>
  );
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово $
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (

```

```

.....
const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  trie: TrieDemo,
  "trie-bfs": TrieBfsDemo,
  "suffix-link": SuffixLinkDemo,
  toposort: ToposortDemo,
  relax: RelaxDemo,
  "prefix-sum": PrefixSumDemo,
  "sparse-table": SparseTableDemo,
  "segment-tree": SegmentTreeDemo,
  bridge: BridgeDemo,
  articulation: ArticulationDemo,
  mst: MstDemo,
  amortized: AmortizedDemo,
  np: NpDemo,
  scc: SccDemo,
  "prefix-function": PrefixFunctionDemo,
  dp: DpDemo,
  floyd: FloydDemo,
  components: ComponentsDemo,
  graph: GraphBasicsDemo,
  "coord-compress": CoordCompressDemo,
};

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
  const Cmp = REGISTRY[kind];
  if (!Cmp) return null;
  return (
    <div className="bg-slate-950/70 rounded-lg border b
      <Cmp />
    </div>
  );
};

```

```

.....
    <div className="absolute inset-0 bg-slate-950/80" />
    <div className="relative w-full sm:max-w-5xl max-h-
      ded-2xl shadow-2xl flex flex-col">
      <div className="flex items-center gap-3 px-4 py-4" />
      <div className="min-w-0 flex-1">
        <h3 className="font-bold text-white text-sm" />
        {viz.hint && <p className="text-[11px] text-white" />}
      </div>
      <button
        onClick={onClose}
        className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
        aria-label="Заккрыть симулятор"
      >
        <X className="w-5 h-5" />
      </button>
    </div>
    <div className="overflow-y-auto overscroll-contain" />
    <Component />
  </div>
</div>,
document.body
);
};

```

ФАЙЛ 46/77: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

.....
import React, { useEffect, useLayoutEffect, useRef, useImperativeHandle } from "react";
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";
import { getViz } from "../vizRegistry";

```

```

useEffect(() => {
  if (!anchor) return;
  const onKey = (e: KeyboardEvent) => {
    if (e.key === "Escape") onClose();
  };
  window.addEventListener("keydown", onKey);
  window.addEventListener("scroll", onClose, true);
  return () => {
    window.removeEventListener("keydown", onKey);
    window.removeEventListener("scroll", onClose, true);
  };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
  <div
    ref={cardRef}
    onMouseEnter={onCardEnter}
    onMouseLeave={onCardLeave}
    className="fixed z-[100] term-popover"
    style={{
      top: pos ? pos.top : -9999,
      left: pos ? pos.left : -9999,
      width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
      visibility: pos ? "visible" : "hidden",
    }}
    role="dialog"
  >
    <div className="bg-slate-900 border border-indigo-500 p-4"
      <div className="flex items-start gap-2 px-3 pt-3"
        <h4 className="text-sm font-bold text-indigo-400"
          <button

```

```
    });  
  };  
};  
  
.....
```

ФАЙЛ 47/77: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
.....  
import { useState } from 'react';  
import { RotateCcw, SkipForward, Undo } from 'lucide-react';  
  
export function JohnsonViz() {  
  const [step, setStep] = useState(0);  
  const MAX_STEP = 7;  
  
  const nodes = [  
    { id: 'S', x: 200, y: 150, label: 'S' },  
    { id: 'A', x: 200, y: 50, label: 'A' },  
    { id: 'B', x: 320, y: 220, label: 'B' },  
    { id: 'C', x: 80, y: 220, label: 'C' },  
  ];  
  
  const edges = [  
    { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },  
    { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },  
    { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },  
    { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 0, labelDx: 0, labelDy: 0 },  
    { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 0, labelDx: 0, labelDy: 0 },  
    { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 0, labelDx: 0, labelDy: 0 },  
  ];  
  
  const isNodeVisible = (id: string) => {  
    if (id !== 'S' && (step !== 0 || step !== 7)) return false;  
    return true;  
  };  
  
  const getPotential = (id: string) => {  
    if (id !== 'S' && (step !== 0 || step !== 7)) return 0;  
    return 0;  
  };  
}
```



```

.....
    if (step === 2) return "url(#arrow-pink)";
    return "url(#arrow-slate-dim)";
  }
  if (e.id === 'AB' && step === 4) return "url(#a
  if (e.id === 'AB' && step > 4) return "url(#arr
  if (e.id === 'BC' && step === 5) return "url(#a
  if (e.id === 'BC' && step > 5) return "url(#arr
  if (e.id === 'CA' && step === 6) return "url(#a
  if (e.id === 'CA' && step > 6) return "url(#arr

  if (step >= 4) return "url(#arrow-slate-dim)";
  return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
  if (e.u === 'S') return e.weight;

  if (step < 4) return e.weight;

  if (e.id === 'AB') {
    if (step === 4) return `${e.weight} + (${0}
    return e.newWeight;
  }
  if (e.id === 'BC') {
    if (step < 5) return e.weight;
    if (step === 5) return `${e.weight} + (${-2}
    return e.newWeight;
  }
  if (e.id === 'CA') {
    if (step < 6) return e.weight;
    if (step === 6) return `${e.weight} + (${-1}
    return e.newWeight;
  }
  return e.weight;
};

const isEdgeTextHighlighted = (e: any) => {
  if (e.id === 'AB' && step === 4) return true;

```

```

.....
        <b>Шаг 3: Магическая формула.</b><b>Шаг 3.1: Пересчет ребра A → B.</b>
        Всем ребрам графа назначаем новый вес.
        <div className="text-center font-monospace">
        Это повышает или понижает вес ребра
        </div>
    );
    case 4: return (
        <div>
            <b>Шаг 3.1: Пересчет ребра A → B.</b>
            Старый вес: <span className="text-emerald-300">0</span>
            Вычисляем:  $-2 + 0 - (-2) =$  <span className="text-emerald-300">-2</span>
        </div>
    );
    case 5: return (
        <div>
            <b>Шаг 3.2: Пересчет ребра B → C.</b>
            Старый вес: 1. <br/>
            Вычисляем:  $1 + (-2) - (-1) =$  <span className="text-emerald-300">-2</span>
        </div>
    );
    case 6: return (
        <div>
            <b>Шаг 3.3: Пересчет ребра C → A.</b>
            Старый вес: 2. <br/>
            Вычисляем:  $2 + (-1) - 0 =$  <span className="text-emerald-300">1</span>
        </div>
    );
    case 7: return (
        <div className="text-emerald-300">
            <b>Готово!</b> Фиктивную вершину S мы добавили.
            Все новые веса ребер в графе стали отрицательными.
            При этом старые кратчайшие пути остались некорректными!
        </div>
    );
    default: return "";
}
};

```

```

.....
    const v = nodes.find(n => n.id === vId);

    const isS = e.u === 'S';
    if (isS && (step === 0 || step === 1)) {
      return;
    }

    const midX = (u.x + v.x) / 2;
    const midY = (u.y + v.y) / 2;

    const edgeColorClass = getEdgeColorClass(e);
    const textHigh = isEdgeTextHigh(e);
    const textEmer = isEdgeTextEmer(e);

    return (
      <g key={i}>
        <line
          x1={u.x} y1={u.y}
          className={`stroke-${edgeColorClass}`}
          markerEnd={getMarkerEnd(e)}
        />

        <text
          x={midX} y={midY}
          textAnchor="middle"
          dominantBaseline="middle"
          className={`text-${edgeColorClass}`}
          style={{
            textHigh: textHigh ? 'normal' : 'none',
            textEmer: textEmer ? 'normal' : 'none',
            isS: isS ? 'fill:#f00' : 'fill:#00f'
          }}
        >{getEdgeContent(e)}
        </text>
      </g>
    )
  }
}
}

{/* Nodes */}
{nodes.map(n => {
  if (!isNodeVisible(n.id)) return;
  const {x, y} = n;
  const {color, size} = n;
  return (
    <g key={n.id}>
      <circle
        cx={x} cy={y}
        r={size}
        fill={color}
        stroke={color}
        stroke-width={2}
      />
    </g>
  )
})}

```

```

.....
        </div>

        {/* Controls */}
        <div className="flex bg-slate-900 border
            <button onClick={() => setStep(1)}
                className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
                <RotateCcw strokeWidth={3}
            </button>
            <button onClick={() => setStep(1)}
                className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
                <Undo strokeWidth={3} size=
            </button>
            <button onClick={() => setStep(1)}
                className={`flex-1 font-bol
                ${step === MAX_STEP
                    ? 'bg-emerald-600/50'
                    : 'bg-indigo-600 ho
                {step === MAX_STEP ? "Готов
                {step !== MAX_STEP && <Skip
            </button>
        </div>

        {/* Progress Dots */}
        <div className="flex justify-center
            {Array.from({length: MAX_STEP +
                <div key={i} className={`w-1
125' : i < step ? 'bg-indigo-900' : 'bg-sla
                )}}
            </div>
        </div>
    </div>
</div>
);
}

```

```

.....
    { u: 3, v: 4 },
    { u: 4, v: 3 },
  ];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini";
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    // Generate step-by-step Kosaraju steps
    const generatedSteps: AlgoStep[] = [];
    const visited = new Set<number>();
    const order: number[] = [];
    const sccMap: Record<number, number> = {};

    const recordStep = (
      phase: AlgoStep["phase"],
      desc: string,
      currentNode: number | null,
      transposed: boolean,
      currentSccId: number,
    ) => {
      generatedSteps.push({
        phase,
        desc,
        currentNode,
        transposed,
        currentSccId,
        visited,
        order,
        sccMap,
      });
    };
  });
};

```

```

    }
  }

  order.push(u);
  recordStep(
    "dfs1",
    `DFS 1: вершина ${u} полностью обработана. Запи-
    u,
    false,
    -1,
  );
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    dfs1(i);
  }
}

const topoOrder = [...order].reverse();
recordStep(
  "transpose",
  `Первый проход завершен! Стек выхода в топологиче-
  ебра.` ,
  null,
  true,
  -1,
);

// Transpose adj
const adjT: number[][] = Array.from({ length: 5 },
initialEdges.forEach((e) => adjT[e.v].push(e.u)));

// Reset visited for phase 2
visited.clear();
let sccId = 0;

```

.....

```
    });
  }
}
```

```
recordStep(
  "finished",
  `Алгоритм успешно завершен! Найдено ${sccId} компонент,
  null,
  true,
  sccId,
);
```

```
setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []);
```

```
const step = steps[currentStepIdx] || {
  phase: "init",
  desc: "Загрузка...",
  visited: new Set(),
  currentNode: null,
  order: [],
  transposed: false,
  sccMap: {},
  currentSccId: -1,
  activeEdges: initialEdges,
};
```

```
useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1) {
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1500);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
});
```

```

.....
        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
    { /* Playfield SVG */}
    <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]" >
            { /* Markers for directed arrows */}
            <defs>
                <marker
                    id="arrow"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#6
                </marker>
                <marker
                    id="arrow-active"
                    viewBox="0 0 10 10"
                    refX="22"
                    refY="5"
                    markerWidth="6"
                    markerHeight="6"
                    orient="auto-start-reverse"
                >
                    <path d="M 0 1 L 10 5 L 0 9 z" fill="#1
                </marker>
                <marker
                    id="arrow-transposed"

```



```

.....
        markerId = "arrow-active";
    }
}

// Adjust slightly for bidirectional link
const isBidi =
    (edge.u === 3 && edge.v === 4) ||
    (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 :

return (
    <line
        key={`edge-${idx}`}
        x1={x1}
        y1={y1 + dy}
        x2={x2}
        y2={y2 + dy}
        stroke={stroke}
        strokeWidth="2.5"
        markerEnd={`url(#${markerId})`}
        className="transition-all duration-300"
    />
);
}}}

{/* Nodes */}
{initialNodes.map((node) => {
    const isCurrent = step.currentNode === node.id;
    const isVisited =
        step.visited.has(node.id) ||
        (step.phase === "dfs1" && step.currentNode === node.id);
    const sccId = step.sccMap[node.id];

    let classes = "fill-slate-800 stroke-slate-800";
    if (sccId) {
        classes = getSccColor(sccId);
    } else if (isCurrent) {
        classes = "fill-amber-600 stroke-amber-600";
    }
    return (
        <div
            key={node.id}
            className={classes}
            style={{
                width: node.w,
                height: node.h,
                position: "relative",
            }}
        >
            {node.label}
        </div>
    );
})}

```

```

        </g>
      );
    }}}
  </svg>

```

```

{step.transposed && (
  <div className="absolute top-4 left-4 bg-in
    1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>
)}
</div>

```

```

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
  00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-4
    Порядок обхода и стек
  </h4>

```

```

<div className="bg-slate-900 border border-sl
  <p className="text-xs text-white leading-re
    {step.desc}
  </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-a
  {/* Top-Sort output / Order stack represent
  <div>
    <span className="text-[10px] text-slate-5
      СТЕК ВЫХОДА (DFS 1 ORDER):
    </span>
    <div className="flex flex-wrap gap-1 bg-s
      {step.order.length === 0 ? (
        <span className="text-slate-600 text-
      ) : (
        [...step.order].reverse().map((val, i
        <div

```

```

.....
        <button
          disabled={currentStepIdx === 0}
          onClick={() => setCurrentStepIdx((prev)
          className="bg-slate-900 border border-s
        >
        Назад
      </button>
      <button
        disabled={currentStepIdx >= steps.length
        onClick={() => setCurrentStepIdx((prev)
        className="bg-slate-900 border border-s
      >
      Вперед
    </button>
  </div>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 49/77: src/components/KruskalSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

.....

import React, { useState } from 'react';
import { Play, RotateCcw, SkipForward, HelpCircle, Chec
  ide-react';

interface Vertex {
  id: number;
  label: string;
  x: number;
  y: number;
  color: string; // 'none', 'red', 'blue', 'purple', 'm

```

Универсальное пособие • полный исходный код

```
.....  
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending'  
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending'  
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending'  
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending'  
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending'  
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending'  
];
```

```
export const KruskalSimulator: React.FC = () => {  
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'  
  const [vertices, setVertices] = useState<Vertex[]>(in  
  const [edges, setEdges] = useState<Edge[]>(initialEdg  
  const [stepIndex, setStepIndex] = useState<number>(0)  
  const [heapQueue, setHeapQueue] = useState<string[]>(  
  const [activeCheatTab, setActiveCheatTab] = useState<  
  
  const [logs, setLogs] = useState<StepLog[]>([  
    {  
      title: 'Введение в раскраску (Краскал)',  
      description: '«Краскал – это раскраска?». Предста  
        овы начинать!',  
      type: 'info',  
    },  
  ]),  
});
```

```
// --- KRUSKAL STEPS ---
```

```
const kruskalSteps = [  
  // Step 1: Inspect A-B (2)  
  () => {  
    setEdges(prev => prev.map(e => e.id === '0-1' ? {  
    setLogs(prev => [{  
      title: 'Шаг 1: Берем самое дешевое ребро A-B (в  
      description: 'Оно соединяет две бесцветные верш  
      type: 'info'  
    }, ...prev]));  
  },  
  // Step 2: Include A-B  
  () => {
```

```

.....
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
      title: 'Успех: Ребро B-C в осто́ве',
      description: 'Вершина C перекрашена в красный ц
      type: 'success'
    }, ...prev]));
  },
// Step 7: Inspect A-D (5)
() => {
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
  setLogs(prev => [{
    title: 'Шаг 4: Берем ребро A-D (вес 5)',
    description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
      ю – мы перекрашиваем всё в один цвет!"',
    type: 'warning'
  }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
  setVertices(prev => prev.map(v => v.color === 'bl
  setEdges(prev => prev.map(e => e.status === 'incl
  setLogs(prev => [{
    title: 'Слияние кланов: Ребро A-D добавлено',
    description: 'Мы объединили красный и синий клан
    type: 'success'
  }, ...prev]));
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '1-4' ? {
  setLogs(prev => [{
    title: 'Шаг 5: Берем ребро B-E (вес 6)',
    description: 'Н0: это ребро соединяет вершины B
    type: 'warning'
  }, ...prev]));
},
// Step 10: Reject B-E

```

```

.....
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.',
        type: 'success'
    }, ...prev]);
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Берем ребро C-F (вес 9)',
        description: 'Оно соединяет вершины C и F, кото
        type: 'warning'
    }, ...prev]);
},
// Step 16: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро C-F.',
        type: 'error'
    }, ...prev]);
},
// Step 17: Inspect G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {
    setLogs(prev => [{
        title: 'Шаг 9: Берем ребро G-H (вес 10)',
        description: 'Соединяет фиолетовую G и бесцветн
        type: 'info'
    }, ...prev]);
},
// Step 18: Include G-H
() => {
    setVertices(prev => prev.map(v => v.id === 7 ? {

```

```

.....
    setLogs(prev => [{
      title: '    Успех: MST построено Краскалом!',
      description: 'Все 9 вершин окрашены в единый фи
        + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
      type: 'success'
    }, ...prev]));
  },
];

// --- PRIM STEPS ---
const primSteps = [
  // Step 1: Start at E
  () => {
    setVertices(prev => prev.map(v => v.id === 4 ? {
    setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E
    setEdges(prev => prev.map(e => ['3-4', '1-4', '4-3
    setLogs(prev => [{
      title: 'Шаг 1: Плесень зарождается в Токио (Вер
      description: 'Мы выбрали вершину E как стартовую
        ритетную очередь (Heap): D-E(3), B-E(6), E-F(
      type: 'info'
    }, ...prev]));
  },
  // Step 2: Pop D-E (3)
  () => {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
    setLogs(prev => [{
      title: 'Шаг 2: Куча выдает самое дешевое ребро
      description: 'Плесень тянется по ребру D-E к вер
      type: 'info'
    }, ...prev]));
  },
  // Step 3: Include D-E, add D's edges to heap
  () => {
    setVertices(prev => prev.map(v => v.id === 3 ? {
    setEdges(prev => prev.map(e => e.id === '3-4' ? {
      ..e, status: 'in_heap' } : e));
    setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D

```

```

.....
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)']);
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-C(4) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 8: Pop B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
        description: 'Плесень тянется к вершине C.',
        type: 'info'
    }, ...prev]);
},
// Step 9: Include B-C, add C's edges
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
        eap' } : e));
    setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (9)']);
    setLogs(prev => [{
        title: 'Успех: Вершина C захвачена',
        description: 'Ребро C-F(9) добавлено в кучу.',
        type: 'success'
    }, ...prev]);
},
// Step 10: Pop B-E (6) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '1-4' ? {
    setLogs(prev => [{
        title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
        description: 'ВНИМАНИЕ! Плесень проверяет вершину E.',
        type: 'warning'
    }, ...prev]);
},

```



```

        description: 'Плесень расширяется вниз к G.',
        type: 'info'
    }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
        eap' } : e));
    setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
    setLogs(prev => [{
        title: 'Успех: Вершина G захвачена',
        description: 'В кучу добавлено G-H(10).',
        type: 'success'
    }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
        description: 'Вершины C и F уже заражены плесенью',
        type: 'warning'
    }, ...prev]);
},
// Step 17: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
    setLogs(prev => [{
        title: 'Отказ: Игнорируем ребро C-F',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]);
},
// Step 18: Pop G-H (10)
() => {
    setEdges(prev => prev.map(e => e.id === '6-7' ? {

```

```

.....
    () => {
      setEdges(prev => prev.map(e => e.id === '7-8' ? {
      setLogs(prev => [{
        title: 'Шаг 12: Куча выдает H-I (вес 12)',
        description: 'Плесень тянется к последней чистой',
        type: 'info'
      }, ...prev]);
    },
  },
// Step 23: Include H-I
() => {
  setVertices(prev => prev.map(v => v.id === 8 ? {
  setEdges(prev => prev.map(e => e.id === '7-8' ? {
  setHeapQueue(['F-I (13 - цикл)']));
  setLogs(prev => [{
    title: '    Успех: Плесень захватила весь граф
    description: 'Все 9 вершин поглощены. Итоговый
      иной – мы разрастались из одной точки с помо
    type: 'success'
  }, ...prev]);
},
],
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
  // Step 1: Five-Year Plan 1 (Inspecting outgoing)
  () => {
    // Highlight the cheapest outgoing edges for EACH
    // Node 0(A) -> A-B(2)
    // Node 1(B) -> A-B(2)
    // Node 2(C) -> B-C(4)
    // Node 3(D) -> D-E(3)
    // Node 4(E) -> D-E(3)
    // Node 5(F) -> E-F(7)
    // Node 6(G) -> D-G(8)
    // Node 7(H) -> G-H(10)
    // Node 8(I) -> H-I(12)
    setEdges(prev => prev.map(e =>
      ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8

```

```

.....
    // Cheapest is A-D (5).
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Вторая пятилетка: Колхозы выбирают мост
        description: 'Теперь Красный и Синий колхозы од
        -D с весом 5. Остальные мосты B-E(6) и C-F(9)
        type: 'warning'
    }, ...prev]));
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
    setVertices(prev => prev.map(v => ({ ...v, color:
    setEdges(prev => prev.map(e =>
        e.status === 'included' || e.id === '0-3' ? { .
    }));
    setLogs(prev => [{
        title: 'Объединение завершено: Один гигантский
        description: 'Оба колхоза объединились по мосту
        type: 'success'
    }, ...prev]));
},
// Step 5: Check remaining edges & mark as rejected
() => {
    setEdges(prev => prev.map(e => e.status === 'pend
    setLogs(prev => [{
        title: ' Успех: MST построен Борувкой за 2 шаг
        description: 'Все лишние ребра (B-E, E-H, C-F,
        вес: 51. Благодаря параллельности, мы потрати
        type: 'success'
    }, ...prev]));
}
};

const currentSteps =
    activeAlgo === 'kruskal' ? kruskalSteps :
    activeAlgo === 'prim' ? primSteps :
    boruvkaSteps;

```

```

    });
  }
};

```

```

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex
    case 'blue': return 'bg-blue-500 border-blue-300
    case 'purple': return 'bg-purple-600 border-purple
    case 'mold': return 'bg-emerald-500 border-emerald
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIn
        return 'bg-slate-700 border-emerald-500 text-
      }
      return 'bg-slate-700 border-slate-500 text-slate
    }
  };

```

```

const getEdgeStroke = (status: string, color: string)
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {
    if (color === 'red') return '#ef4444';
    if (color === 'blue') return '#3b82f6';
    if (color === 'purple') return '#a855f7';
    if (color === 'mold') return '#10b981';
  }
  return '#475569';
};

```

```

return (
  <div className="space-y-12">

    {/* 3-in-1 MST Simulator Box */}
    <div className="bg-slate-900/90 border border-sla
      <div className="flex flex-col lg:flex-row justi
        <div>

```

```

.....
      <Layers className="w-3.5 h-3.5" />
      <span>Прима (Билет 19)</span>
    </button>
    <button
      onClick={() => handleReset('boruvka')}
      className={
        "flex items-center gap-1.5 px-3 py-2
+
        (activeAlgo === 'boruvka'
          ? 'bg-rose-600 text-white shadow-md
          : 'text-slate-400 hover:text-slate-1
        }
    >
      <Users className="w-3.5 h-3.5" />
      <span>Борувка (Билет 20)</span>
    </button>
  </div>

  <button
    onClick={() => handleReset(activeAlgo)}
    className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
  >
    <RotateCcw className="w-3.5 h-3.5" />
    <span>Сбросить</span>
  </button>

  <button
    onClick={handleNextStep}
    disabled={stepIndex >= currentSteps.length}
    className={
      "flex items-center justify-center gap-2
initial cursor-pointer " +
      (stepIndex >= currentSteps.length
        ? "bg-slate-800 text-slate-500 border
        : activeAlgo === 'kruskal'
        ? "bg-indigo-600 hover:bg-indigo-500
        : activeAlgo === 'prim'

```

```

.....
      <div className="flex items-center gap-4">
        <div className="w-3 h-3 rounded-full bg-slate-300">
        </div>
      </div>
    ) : (
      <div className="flex items-center gap-4">
        <div className="w-3 h-3 rounded-full bg-slate-300">
        </div>
        <div className="flex items-center gap-4">
          <div className="w-3 h-3 rounded-full bg-slate-300">
          </div>
          <div className="flex items-center gap-4">
            <div className="w-3 h-3 rounded-full bg-slate-300">
            </div>
          </div>
        </div>
      </div>
    )}
  </div>

  <div className="absolute top-4 right-4 bg-slate-300 p-4" style={{
    width: 100px, height: 100px, border: 1px solid black,
    display: flex, align-items: center, justify-content: center,
    font-size: 10px, font-weight: bold, color: white,
    text-align: center, line-height: 1.2,
    overflow: hidden,
  }}>
    {stepIndex} / {currentSteps.length}
  </div>

  {/* SVG Edges */}
  <svg className="absolute inset-0 w-full h-full" style={{
    pointerEvents: "none",
  }}>
    {edges.map(edge => {
      const u = vertices[edge.u];
      const v = vertices[edge.v];
      const strokeColor = getEdgeStroke(edge);
      const isInspecting = edge.status === 'inspect';
      const isRejected = edge.status === 'rejected';
      const isInHeap = edge.status === 'in_heap';

      return (
        <g key={edge.id}>
          <line
            x={u.x} y={u.y} x2={v.x} y2={v.y}
            stroke={strokeColor}
            stroke-width={1}
            style={{
              opacity: isInspecting ? 1 : 0.5,
              stroke-dasharray: isRejected ? [5, 5] : [],
            }}
          />
        </g>
      );
    })}
  </svg>

```

```

>
    <span>{vertex.label}</span>
    {vertex.id === 4 && activeAlgo === 'p
      <span className="text-[8px] block -f
    }}
  </div>
)}}
</div>

{stepIndex >= currentSteps.length && (
  <div className="absolute inset-x-6 bottom
  shadow-xl z-20">
    <p className={"font-bold text-base " +
    'ld-300' : 'text-rose-300'})}>
      MST построено с помощью {activeAlgo
    </p>
    <p className="text-slate-300 text-xs mt
      Общий вес минимального остова: 51. Вс
    </p>
  </div>
)}
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl bord
  <div className="p-4 bg-slate-900 border-b b
    <h4 className="font-bold text-slate-200 t
      Ход выполнения
    </h4>
  </div>

{activeAlgo === 'prim' && (
  <div className="p-3 bg-slate-900/60 borde
    <p className="text-[11px] font-bold tex
      ⚡ Приоритетная очередь (Min-Heap):
    </p>
    <div className="flex flex-wrap gap-1.5":
      {heapQueue.length === 0 ? (

```

```

        </div>
      </div>
    </div>
  </div>

  { /* Cheat Sheet: Complexity & Code for all 3 algo
  <div className="bg-slate-900/80 border border-sla
    <div className="flex flex-col sm:flex-row items
      <div className="flex items-center gap-2.5">
        <BookOpen className="w-5 h-5 text-indigo-400
        <h4 className="text-xl font-bold text-white
      </div>

    <div className="flex bg-slate-950 p-1 rounded
      <button
        onClick={() => setActiveCheatTab('kruskal
        className={"px-3 py-1.5 rounded text-xs f
        === 'kruskal' ? "bg-indigo-600 text-white" : "
      >
        Краскал (Билет 18)
      </button>
      <button
        onClick={() => setActiveCheatTab('prim')}
        className={"px-3 py-1.5 rounded text-xs f
        === 'prim' ? "bg-emerald-600 text-white" : "
      >
        Прима (Билет 19)
      </button>
      <button
        onClick={() => setActiveCheatTab('boruvka
        className={"px-3 py-1.5 rounded text-xs f
        === 'boruvka' ? "bg-rose-600 text-white" : "
      >
        Борувка (Билет 20)
      </button>
    </div>
  </div>

```



```
.....
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False
```

2. Жадный выбор ребер

edges.sort() # $O(E \log E)$

mst = []

for w, u, v in edges:

if union(u, v):

mst.append((u, v, w))`}

</pre>

</div>

</div>

</div>

}}

{activeCheatTab === 'prim' && {

<div className="grid grid-cols-1 lg:grid-cols-

<div className="lg:col-span-1 space-y-4">

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Clock className="w-3.5 h-3.5 text-em

</p>

<p className="text-2xl font-black text-r

<p className="text-slate-400 text-xs mt

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

<Award className="w-3.5 h-3.5 text-em

</p>

<p className="text-2xl font-black text-r

<p className="text-slate-400 text-xs mt

</div>

<div className="bg-slate-950 p-4 rounded-

<p className="text-slate-400 text-xs for

ложении:</p>

```

.....
        <p className="text-slate-400 text-xs font-medium">
            <Clock className="w-3.5 h-3.5 text-rose-500" />
        </p>
        <p className="text-2xl font-black text-rose-500">
            <p className="text-slate-400 text-xs font-medium">
                <div>
                    </div>
                <div className="bg-slate-950 p-4 rounded-2xl">
                    <p className="text-slate-400 text-xs font-medium">
                        <Award className="w-3.5 h-3.5 text-rose-500" />
                    </p>
                    <p className="text-2xl font-black text-rose-500">
                        <p className="text-slate-400 text-xs font-medium">
                            </div>
                        <div className="bg-slate-950 p-4 rounded-2xl">
                            <p className="text-slate-400 text-xs font-medium">
                                ении:</p>
                                <p className="text-xs text-slate-300 leading-5">
                                    , организовав массовое распараллеленное слияние
                                </p>
                            </div>
                        </div>
                    <div className="lg:col-span-2">
                        <div className="bg-slate-950 p-4 rounded-2xl">
                            <p className="text-slate-400 text-xs font-medium">
                                <Code className="w-3.5 h-3.5 text-rose-500" />
                            </p>
                            <pre className="text-xs text-emerald-400 font-mono">
                                80 flex-1">
                                {`def boruvka(n, edges):
                                    parent = list(range(n))
                                    mst = []
                                    while len(mst) < n - 1:
                                        # Для каждого колхоза ищем самый дешёвый мост на
                                        cheapest = [-1] * n
                                        for u, v, w in edges:
                                            ru, rv = find(u), find(v)
                                            if ru != rv:
                                                if cheapest[ru] == -1 or cheapest[ru][2]

```

```

.....
    badge: 'text-blue-400/70 bg-blue-950/40',
  },
  {
    img: bellmanImg,
    alt: 'Фура по болоту Форд-Беллман',
    title: '    Фура по Болоту (Ф-Б)',
    desc: 'Медленный, тяжёлый — зато тащит',
    highlight: 'отрицательные',
    highlightColor: 'text-rose-400',
    rest: 'веса и детектит отрицательные циклы.',
    complexity: 'O(V · E)',
    border: 'border-rose-500/30 hover:border-rose-400/6',
    titleColor: 'text-rose-400',
    badge: 'text-rose-400/70 bg-rose-950/40',
  },
  {
    img: floydImg,
    alt: 'Все-ко-Все Флойд',
    title: '    Все ко Все (Флойд)',
    desc: 'Матрица + три цикла {',
    highlight: 'k, i, j',
    highlightColor: 'text-emerald-400',
    rest: '}'. Ищет пути от всех ко всем.',
    complexity: 'O(V³)',
    border: 'border-emerald-500/30 hover:border-emerald',
    titleColor: 'text-emerald-400',
    badge: 'text-emerald-400/70 bg-emerald-950/40',
  },
];

```

```

export default function MnemonicCards() {
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
      {cards.map((c, i) => (
        <div key={i} className={`bg-slate-800 rounded-2xl p-4">
          <div className="h-48 overflow-hidden">
            <img src={c.img} alt={c.alt}
              className="w-full h-full object-cover group-hover:scale-110 transition duration-300">

```

```

.....
const [open, setOpen] = useState(false);

useEffect(() => {
  document.body.style.overflow = open ? "hidden" : "";
  return () => {
    document.body.style.overflow = "";
  };
}, [open]);

useEffect(() => {
  const onKey = (e: KeyboardEvent) => e.key === "Escape";
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, []);

return (
  <div className="lg:hidden sticky top-[57px] z-40">
    <button
      onClick={() => setOpen(true)}
      className="w-full flex items-center gap-2.5 p-2"
      aria-label="Открыть содержание"
    >
      <span className="p-1.5 rounded-lg bg-indigo-600 text-white">
        <List className="w-4 h-4" />
      </span>
      <span className="min-w-0 flex-1">
        <span className="block text-[10px] uppercase">
          Содержание • {index + 1} из {total}
        </span>
        <span className="block text-[13px] font-bold">
          {title}
        </span>
      </span>
    </button>
  </div>

  {open && (
    <div className="lg:hidden fixed inset-0 z-[90]">
      <div className="absolute inset-0 bg-slate-950">

```

Универсальное пособие • полный исходный код

```
.....  
}  
  
export const Navbar: React.FC<NavbarProps> = ({ activeTab }) => {  
  return (  
    <header className="bg-slate-900 border-b border-slate-800">  
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">  
        <div className="flex items-center gap-3 w-full">  
          <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">  
            <Sparkles className="w-6 h-6" />  
          </div>  
          <div className="min-w-0 flex-1">  
            <h1 className="text-xl font-extrabold text-white">  
              Универсальное пособие  
            </h1>  
            <p className="text-xs text-indigo-400 font-medium">  
              Подготовка к экзаменам: Алгоритмы, Структуры данных  
            </p>  
          </div>  
        </div>  
  
        <div className="flex items-center w-full lg:w-auto">  
          <button  
            id="tab-guide-btn"  
            onClick={() => setActiveTab('guide')}  
            className={`flex-1 lg:flex-none flex items-center justify-center text-white duration-200 whitespace-nowrap ${  
              activeTab === 'guide'  
                ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50' : 'text-slate-400 hover:text-white'>  
            </button>  
            <BookOpen className="w-4 h-4" /> Учебное пособие  
          </button>  
          <button  
            id="tab-simulator-btn"
```

```
};
```

ФАЙЛ 53/77: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
export function PlanarityDemo() {
  return (
    <div className="space-y-4">
      <h4 className="text-base font-bold text-white">K5
      <p className="text-xs text-slate-400">Просто смотрим
      <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
        { /* K5 */ }
        <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
          <div className="absolute top-2 left-2 bg-slate-900 text-white text-xs font-mono p-2">
            00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
          <svg className="w-full h-full absolute inset-0" >
            {(() => {
              const pts = [
                {id:0,x:160,y:40},
                {id:1,x:270,y:100},
                {id:2,x:230,y:230},
                {id:3,x:90,y:230},
                {id:4,x:50,y:100}
              ];
              const edges = [
                [0,1],[0,2],[0,3],[0,4],
                [1,2],[1,3],[1,4],
                [2,3],[2,4],
                [3,4]
              ];
              function findPt(id:number) { return pts.find(p => p.id === id); }
              function intersect(a:number,b:number,c:number,d:number) {
                if (a===c||a===d||b===c||b===d) return null;
                const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
                const [x,y] = intersectLineSegments(p1,p2,p3,p4);
                return x !== null ? x : y;
              }
            })()}
          </svg>
        </div>
      </div>
    );
}
```

```

.....
    { /* K3,3 */}
    <div className="bg-slate-950 rounded-xl p-4 border-2 border-slate-800">
      <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
      <svg className="w-full h-full absolute inset-0" >
        {{{() => {
          const pts = [
            {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
            {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
          ];
          const edges = [
            [0,3],[0,4],[0,5],
            [1,3],[1,4],[1,5],
            [2,3],[2,4],[2,5]
          ];
          function findPt(id:number) { return pts.find(p => p.id === id); }
          function intersect(a:number,b:number,c:number,d:number) {
            if (a===c||a===d||b===c||b===d) return true;
            const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
            if (!p1||!p2||!p3||!p4) return false;
            function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
              return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
            }
            return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
              && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
          }
          const crosses:number[]=[];
          for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
              if (!crosses.includes(i)) crosses.push(i);
              if (!crosses.includes(j)) crosses.push(j);
            }
          }
          const lines = []; const circles = [];
          for(let i=0;i<edges.length;i++) {
            const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
            const xing=crosses.includes(i);
            lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} />);
            if (xing) {
              const x=edges[i][0]+edges[i][1]+1;
              const y=edges[i][0]+edges[i][1]+1;
              circles.push(<circle key={`c${i}`} cx={x} cy={y} r={10} />);
            }
          }
          return <div>
            {lines.map(l=><div>{l}</div>)}
            {circles.map(c=><div>{c}</div>)}
          </div>
        }}}
      </svg>
    </div>
  </div>

```

```

.....
  name: string;
  emoji: string;
  color: string;
  bubbleText: string;
  animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
  { name: 'Студент Вася', emoji: '🍷', color: 'from-black', bubbleText: 'матана!' },
  { name: 'Кодер Семён', emoji: '🍲', color: 'from-violet', bubbleText: 'порцию борща!' },
  { name: 'Кот Борис', emoji: '🐈', color: 'from-amber', bubbleText: 'сметаны, плиз.' },
  { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose', bubbleText: 'сметаны, плиз.' },
  { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald', bubbleText: 'салатик для нашего НЛО!' },
  { name: 'Бизнесмен', emoji: '💼', color: 'from-slate', bubbleText: 'сметаны, плиз.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
  const [queue, setQueue] = useState<Customer[]>([
    { id: 'q1', name: 'Студент Вася', emoji: '🍷', color: 'from-black', bubbleText: 'голода после матана!', animState: 'idle' },
    { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'from-violet', bubbleText: 'покажу ИИ за порцию борща!', animState: 'idle' },
    { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber', bubbleText: 'сметаны, плиз.', animState: 'idle' },
  ]);

  const [servedHistory, setServedHistory] = useState<string[]>([]);
  const [isServing, setIsServing] = useState(false);
  const [shakeEmpty, setShake] = useState(false);
  const [log, setLog] = useState<string[]>([]);

```



```

    }

    setIsServing(true);
    const headGuest = queue[0];

    addLog(`  DEQUEUE: Повар наливает суп первому – ${headGuest.name}`);

    // Step 1: Head guest gets eating animation
    setQueue(prev => prev.map((c, idx) => idx === 0 ? {
      ...c,
      animState: 'eating'
    }) : c);

    setTimeout(() => {
      // Step 2: Guest leaves with happy face, queue shifts
      setQueue(prev => prev.map((c, idx) => idx === 0 ? {
        ...c,
        animState: 'happy'
      }) : c);

      setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

      setTimeout(() => {
        setQueue(prev => prev.slice(1));
        setIsServing(false);
        addLog(`  ${headGuest.name} поел и ушел сытым.`, 400);
      }, 900);
    }, [queue, isServing]);

    const reset = () => {
      counter = 3;
      setQueue([
        { id: 'r1', name: 'Студент Вася', emoji: '🍜', color: 'red', col: 1, text: 'Я голода после матана!', animState: 'in' },
        { id: 'r2', name: 'Кодер Семён', emoji: '👨‍💻', color: 'blue', col: 2, text: 'Напишу ИИ за порцию борща!', animState: 'in' },
        { id: 'r3', name: 'Кот Борис', emoji: '🐱', color: 'green', col: 3, text: 'Дайте котика без сметаны, плиз.', animState: 'in' },
      ]);
      setServedHistory([]);
      setIsServing(false);
      setLog(['  Столовая сброшена. Снова голодная толпа!']);
      setTimeout(() => {

```

```

.....
    <span> Налить суп первому (DEQUEUE)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800
      r-pointer border border-slate-700"
  >
    Сброс
  </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12" >

  {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
  <div className={`md:col-span-9 bg-slate-900 rounded-3xl
    -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`} >
    <div className="absolute top-4 left-4 text-[1.2em] font-mono" >
      Очередь голодных гостей (Буфер FIFO / BFS)
    </div>

    <div className="flex-1 flex flex-col justify-between" >
      <div className="flex items-end justify-between" >

        {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}
        <div className="flex flex-col items-center" >
          <div className="relative" >
            <div className="absolute -top-10 left-1/2 -translate-x-1/2" >
              <div className="w-1.5 h-6 bg-slate-400 rounded" >
                <div className="w-2 h-4 bg-slate-400 rounded" >
              </div>
            <span className="text-5xl block animate-pulse" >
            </div>
          <div className="text-center" >
            <span className="text-3xl block" > <
            <span className="text-[1.1em] font-mono" >
              <span className="text-[10px] font-mono tracking-wider" >

```

```

.....
        `}>
        <div className="font-bold text-3xl" style={{
        {customer.bubbleText}
        <div className="absolute -bottom-10 left-10
it rotate-45"></div>
        </div>

        {/* Персонаж */}
        <div className={`
        w-14 h-14 rounded-2xl border-2
relative
        ${customer.color}
        ${isHead ? 'ring-4 ring-amber-500' : ''}
        `}>
        <span className="text-3xl">{customer.name}</span>

        {/* Индексы HEAD/TAIL */}
        {(isHead || isTail) && (
        <span className={`
        absolute -bottom-2 text-3xl
        ${isHead ? 'bg-amber-500' : ''}
        `}>
        {isHead && isTail ? 'h+t' : ''}
        </span>
        )}
        </div>
        </div>
        );
    })
  })
</div>

</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-amber-500 to-amber-500" style={{
</div>

```

```
.....
    </div>

    {/* ЛОГ ДЕЙСТВИЙ */}
    <div className="bg-slate-900 border border-slate-700" >
      <div className="text-[10px] font-mono text-slate-100" >
        {log.map(({entry, idx}) => {
          <div
            key={idx}
            className={`text-xs font-mono flex items-center justify-between`} >
            {idx === 0 ? <span className="text-indigo-500">[0] : <span>[...]}
            <span>{entry}</span>
          </div>
        })}
      </div>
    </div>
  );
};
```

ФАЙЛ 55/77: src/components/SalmonAutomatonWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
  id: number;
  char: string;
  parent: number;
  children: { [key: string]: number };
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
  depth: number;
}
```

```

};
let count = 0;

wordList.forEach((word) => {
  let curr = 0;
  for (let i = 0; i < word.length; i++) {
    const c = word[i].toUpperCase();
    if (newNodes[curr].children[c] === undefined) {
      count++;
      newNodes[curr].children[c] = count;
      newNodes[count] = {
        id: count,
        char: c,
        parent: curr,
        children: {},
        fail: 0,
        term_link: 0,
        is_terminal: false,
        words: [],
        depth: newNodes[curr].depth + 1
      };
    }
    curr = newNodes[curr].children[c];
  }
  if (!newNodes[curr].words.includes(word)) {
    newNodes[curr].words.push(word);
    newNodes[curr].is_terminal = true;
  }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
  const childKeys = Object.values(newNodes[u].children);
  newNodes[u].y = 40 + depth * paddingY;

```

```

        return;
    }
    const updated = [...words, cleanWord];
    setWords(updated);
    setNewWord('');
    buildInitialTrie(updated);
};

const removeWord = (wordToRemove: string) => {
    if (words.length <= 1) {
        setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить');
        return;
    }
    const updated = words.filter((w) => w !== wordToRemove);
    setWords(updated);
    buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
    const rootChildren = Object.values(nodes[0].children);
    const queue = [...rootChildren];
    const updatedNodes = { ...nodes };
    rootChildren.forEach((childId) => {
        updatedNodes[childId].fail = 0;
    });

    setNodes(updatedNodes);
    setBfsQueue(queue);
    setSimulationStep(1);
    setCurrentNode(null);
    setSpeechText(
        'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
    );
};

// Один шаг BFS

```



```

.....
        <circle cx="148" cy="85" r="6" fill="black">
        <circle cx="151" cy="82" r="2" fill="black">
    </>
    })

    {/* Рот (анимация разговора) */}
    {isTalking ? (
        <path d="M 165 105 Q 155 120 165 125 Q 165 125 Q 155 110 Q 165 110 Q 155 115 168 118" s
    ) : (
        <path d="M 165 110 Q 155 115 168 118" s
    )}

    {/* Улыбка / жабры */}
    <path d="M 130 80 Q 125 100 130 120" stroke="black" />
    <path d="M 120 85 Q 115 100 120 115" stroke="black" />
</svg>

    {/* Декоративные пузыри */}
    <div className="absolute -top-1 right-4 text-right">
    <div className="absolute top-8 right-1 text-right">
</div>
    <span className="mt-3 bg-rose-600/30 border border-rose-600/30 padding-2">
        Эхо-Лосось Сеня
    </span>
</div>

    {/* Пузырь с текстом (Баббл) */}
    <div className="md:col-span-2 bg-slate-800 border border-slate-800 rounded-2xl p-4
    tween min-h-[140px]">
    {/* Треугольник баббла */}
    <div className="absolute -left-3 top-1/2 -transform rotate-90deg
    slate-800 border-b-[12px] border-b-transparent">

    <div className="flex items-center space-x-2 text-white">
        <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}` />
        <span>Прямое вещание из аквариума:</span>
    </div>

```



```

</button>
  })
</div>
</div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
  <h5 className="text-white font-bold mb-3 flex items-center" >
    <span> </span> Дерево и Суффиксные (fail) ссылки
  </h5>

  {{{() => {
    let maxX = 0;
    let maxY = 0;
    Object.values(nodes).forEach(n => {
      if (n.x && n.x > maxX) maxX = n.x;
      if (n.y && n.y > maxY) maxY = n.y;
    });
    const svgWidth = Math.max(maxX + 80, 400);
    const svgHeight = Math.max(maxY + 60, 200);

    return (
      <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
        <defs>
          <marker id="arrowhead" markerWidth="6" markerHeight="6" >
            <polygone points="0 0, 6 3, 0 6" fill="white" />
          </marker>
          <marker id="fail-arrow" markerWidth="6" markerHeight="6" >
            <polygone points="0 0, 6 3, 0 6" fill="white" />
          </marker>
        </defs>

        {/* Draw Edges */}
        {Object.values(nodes).map(node => {
          if (node.id === 0) return null;
          const parent = nodes[node.parent];

```

```

.....
    // Simple bezier curve for fail link
    const isCurrent = currentNode === node..

    return (
      <path
        key={`fail-${node.id}`}
        d={`M ${node.x} ${node.y - 10} Q ${
- 16}`}
        fill="none"
        stroke={isCurrent ? '#f43f5e' : 'rgb(
        strokeWidth={isCurrent ? 2 : 1.5}
        strokeDasharray="4 4"
        markerEnd="url(#fail-arrow)"
      />
    );
  }}}

  {/* Draw Nodes */}
  Object.values(nodes).map(node => {
    if (!node.x || !node.y) return null;
    const isRoot = node.id === 0;
    const isCurrent = currentNode === node..
    const isInQueue = bfsQueue.includes(node..

    let fill = '#1e293b';
    let stroke = '#475569';

    if (isCurrent) {
      fill = '#2563eb';
      stroke = '#60a5fa';
    } else if (isInQueue) {
      fill = '#b45309';
      stroke = '#fbbf24';
    } else if (node.is_terminal) {
      fill = '#059669';
      stroke = '#34d399';
    }
  })

```

```

.....
    { /* Управление словарем */ }
    <div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
      <div className="bg-slate-900/50 p-5 rounded-xl">
        <div>
          <h5 className="text-white font-bold mb-3">
            <span> </span> Словарь (Паттерны)
          </h5>
          <div className="flex flex-wrap gap-2 mb-4">
            {words.map((w) => (
              <span
                key={w}
                className="bg-slate-800 border border-slate-700
                  px-2 py-1 rounded text-white font-mono"
              >
                {w}
                <button
                  onClick={() => removeWord(w)}
                  className="text-slate-500 hover:text-white
                    font-mono text-sm"
                  title="Удалить"
                >
                  x
                </button>
              </span>
            ))}
          </div>
        </div>
      </div>

      <form onSubmit={handleAddWord} className="flex gap-2">
        <input
          type="text"
          value={newWord}
          onChange={(e) => setNewWord(e.target.value)}
          placeholder="НОВОЕ СЛОВО..."
          maxLength={8}
          className="bg-slate-800 border border-slate-700
            outline-none focus:border-blue-500"
        />
        <button

```

```

.....
    return (
      <tr
        key={node.id}
        className={`transition-colors ${
          isCurrent
            ? 'bg-blue-900/40 font-bold text-white'
            : isInQueue
            ? 'bg-slate-800/80'
            : 'hover:bg-slate-800/40'
          }`}
      >
        <td className="py-2.5 px-3 flex items-center" >
          <span
            className={`w-2 h-2 rounded-full ${
              node.id === 0
                ? 'bg-purple-500'
                : isCurrent
                ? 'bg-blue-400 animate-pulse'
                : isInQueue
                ? 'bg-amber-400'
                : 'bg-slate-600'
            }`}
          ></span>
          <span>#{node.id}</span>
          {node.id === 0 && <span className="font-size-0.8" >
            </td>
            <td className="py-2.5 px-3">
              <span className="bg-slate-800 border border-slate-700 rounded px-2" >
                {node.char}
              </span>
            </td>
            <td className="py-2.5 px-3 text-slate-400">
              {node.id === 0 ? '-' : `#${node.id}`}
            </td>
            <td className="py-2.5 px-3">
              {node.id === 0 ? (
                <span className="text-slate-500 font-size-0.8" >
                  ) : (

```

```
.....
    </div>
  </div>
);
};
```

ФАЙЛ 56/77: src/components/SegmentTreeVisualizer.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```
.....
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

// ===== TYPES =====
interface Step {
  id: number;
  desc: string;
  codeLine: number;
  treeState: Record<number, number>; // node index to value
  highlightNodes: number[]; // current highlighted nodes
  mergeChildren?: [number, number]; // pair being merged
  arrayHighlight?: [number, number]; // [L, R] range in array
  result?: number; // partial result
}

// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, number> {
  const n = arr.length;
  const tree: Record<number, number> = {};
  function go(v: number, l: number, r: number) {
    if (l === r) { tree[v] = arr[l]; return; }
    const m = (l + r) >> 1;
    go(2 * v, l, m);
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
  }
  go(1, 0, n - 1);
}
```

Универсальное пособие * полный исходный код

```
.....
    steps.push({ id: id++, desc: `Вернулись в build(${v},
      lightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v + 1, m + 1, r);
    tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
    steps.push({ id: id++, desc: `Объединяем: tree[${v}],
      state: { ...tree }, highlightNodes: [v], mergeChildren:
    });
  go(1, 0, n - 1);
  steps.push({ id: id++, desc: `Дерево построено! Корень:
    Highlight: [0, n - 1] });
  return steps;
}
```

// 2. Top-Down Query steps

```
function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
  const n = arr.length;
  const tree = buildTreeFull(arr);
  const steps: Step[] = [];
  let id = 0;

  function go(v: number, l: number, r: number, ql: number, qr: number) {
    if (ql > r || qr < l) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}],
        codeLine: 2, treeState: tree, highlightNodes: [v],
      });
      return 0;
    }
    if (ql <= l && r <= qr) {
      steps.push({ id: id++, desc: `query(${v}, [${l}..${r}],
        result = ${tree[v]}, codeLine: 4, treeState: tree,
      });
      return tree[v];
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `query(${v}, [${l}..${r}],
      codeLine: 6, treeState: tree, highlightNodes: [v],
    });
    const left = go(2 * v, l, m, ql, qr);
    const right = go(2 * v + 1, m + 1, r, ql, qr);
    const res = left + right;
    steps.push({ id: id++, desc: `Возврат в узел ${v}: ${res}` });
  }
}
```

Универсальное пособие • полный исходный код

```
.....
    steps.push({ id: id++, desc: `r=${r + 1} нечётный
      , treeState: tree, highlightNodes: [r], arrayHi
    highlights.push(r);
  }
  l >>= 1;
  r >>= 1;
  if (l < r) {
    steps.push({ id: id++, desc: `Поднимаемся: l = ${
      , arrayHighlight: [qL, qR], result: res });
  }
}
steps.push({ id: id++, desc: ` Запрос sum([${qL}..${q
  ighlightNodes: [1], arrayHighlight: [qL, qR], result
return steps;
}
```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val:
```

```
function buildVisualTree(treeState: Record<number, number>) {
  if (l === r) return { v, l, r, val: treeState[v] ?? null };
  const m = (l + r) >> 1;
  return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(
    v + 1, m + 1, r) };
}
```

// ===== CODE SNIPPETS =====

```
const BUILD_CODE = [
  "build(v, l, r) {",
  "  if (l === r) { tree[v] = A[l]; return; }",
  "  // -----",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  build(2*v, l, mid);",
  "  build(2*v+1, mid+1, r);",
  "  tree[v] = tree[2*v] + tree[2*v+1];",
  "}"
];
```

```

.....
const step = steps[idx] || steps[0];
if (!step) return null;

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-
border-indigo-500 bg-indigo-600', childRing: 'ring-
-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-e
30 border-emerald-500 bg-emerald-600', childRing: '
-slate-900' }, amber: { btn: 'bg-amber-600 hover:bg
der-amber-500 bg-amber-600', childRing: 'ring-rose-
[color];

return { step, idx, setIdx, playing, setPlaying, speed
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr
| 'emerald' | 'amber'; arr: number[] }) {
const p = Player({ steps, color });
if (!p) return null;
const { step, idx, setIdx, playing, setPlaying, speed
const n = arr.length;

const vTree = useMemo(() => buildVisualTree(step.tree

const renderNode = useCallback((nd: VNode): React.Rea
const isHigh = step.highlightNodes.includes(nd.v);
const isChild = step.mergeChildren?.includes(nd.v);
const has = nd.val !== null;

let cls = 'bg-slate-800 border-slate-700 text-slate
if (isHigh) cls = `${clr.ring} text-white ring-2 sc
else if (isChild) cls = `${clr.childRing} text-ambe
else if (has) cls = `${clr.filled} text-slate-200`;

return (
  <div key={nd.v} className="flex flex-col items-ce
    <div className={`flex flex-col items-center jus
      ${cls}`>

```



```

    });
  }}}
</div>

{/* Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-slate-900/60 border-1 border-indigo-500">
  <div className="scale-[0.85] origin-top min-w-max">
    <pre>
      {code.map((line, i) => {
        const ln = i + 1;
        const active = step.codeLine === ln;
        return (
          <div key={ln} className={`px-2 py-0.5 rounded-2 border-indigo-500 text-indigo-200 ${active ? 'bg-amber-600/20 border-l-2 border-amber-600' : ''}`}>
            <span className="inline-block w-4 text-slate-900">{ln}</span>
          </div>
        );
      })}
    </pre>

    {/* Description + result */}
    <div className="px-3 py-2.5 bg-slate-900/60 border-1 border-indigo-500">
      <span className={`mt-0.5 inline-block w-2 h-2 rounded-2 border-indigo-500 text-indigo-200 ${active ? 'bg-amber-600/20 border-l-2 border-amber-600' : ''}`}></span>
      <span className="leading-relaxed">{step.desc}</span>
      {step.result !== undefined && (
        <span className={`ml-auto shrink-0 font-mono text-emerald-300`}>
          {step.result}
        </span>
      )}
    </div>
  </div>

```

```

.....
const [qR, setQR] = useState(4);
const [view, setView] = useState<'build' | 'queries'

// Clamp query range when array changes
useEffect(() => {
  setQL(prev => Math.min(prev, arr.length - 1));
  setQR(prev => Math.min(prev, arr.length - 1));
}, [arr]);

const handleApply = (e: React.FormEvent) => {
  e.preventDefault();
  const parsed = customInput.split(',').map(s => pars
  if (parsed.length >= 2 && parsed.length <= 16) {
    setArr(parsed);
  }
};

const randomize = () => {
  const size = arr.length;
  const a = Array.from({ length: size }, () => Math.f
  setArr(a);
  setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSte
const queryBUSteps = useMemo(() => genQueryBottomUpSt

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
  <div className="bg-slate-900 rounded-2xl border bor
    /* ---- TOP BAR: array editor + query range ----
    <div className="mb-6 space-y-4">
      <div className="flex flex-col lg:flex-row gap-4
        /* Array input */
        <form onSubmit={handleApply} className="flex-
          <div className="flex items-center justify-b

```

```

.....
        className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
      />
    </div>
    <div className="flex items-center gap-1">
      <span className="text-xs text-slate-400">
        <input
          type="number"
          min={0}
          max={arr.length - 1}
          value={qR}
          onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
          className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
        />
      </div>
    </div>
    <div className="text-[10px] text-slate-500">
      Запрос:  $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.slice(qL, qR+1).reduce((sum, val) => sum + val, 0))\}$ 
    </div>
  </div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1">
  <button onClick={() => setView('build')} className="text-sm font-medium text-slate-400 hover:text-slate-200" >
    <Hammer className="w-3.5 h-3.5" /> Построить
  </button>
  <button onClick={() => setView('queries')} className="text-sm font-medium text-slate-400 hover:text-slate-200" >
    Запрос:  $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.slice(qL, qR+1).reduce((sum, val) => sum + val, 0))\}$ 
  </button>
  <button onClick={() => setView('theory')} className="text-sm font-medium text-slate-400 hover:text-slate-200" >
    Теория
  </button>

```

```

    </div>
  })

  {view === 'theory' && {
    <div className="space-y-5">
      {/* Why it splits this way */}
      <div className="bg-slate-950 p-5 rounded-xl b
        <h4 className="text-base font-bold text-whi
          во разбилось именно так для N={arr.length}?
          <p className="text-sm text-slate-300 leadin
            Каждый узел берёт <code className="bg-sla
            к получает <code className="bg-slate-900 px
            -slate-900 px-1 rounded font-mono text-emerg
            евая часть забирает на 1 больше (округление
            </p>
            <div className="bg-slate-900 p-4 rounded-lg
              <div className="text-slate-400">Корень: L
              <div className="text-slate-300">mid = {(0
              <div className="text-indigo-300">→ Левый:
              <div className="text-emerald-300">→ Правый
              h - 1) >> 1)} эл.)</div>
              <div className="text-slate-500 mt-2">Всег
            } внутренних.</div>
            </div>
          </div>

        {/* Comparison cards */}
        <div className="grid grid-cols-1 xl:grid-cols
          <div className="bg-slate-950 p-5 rounded-xl
            <div className="absolute -top-3 left-4 bg
              e border border-emerald-500 shadow-md">
                Запрос Сверху-вниз (Рекурсия)
              </div>
              <p className="text-xs text-slate-300 mb-3
                Начинаем с корня. Если отрезок узла полн
                наче спускаемся в обоих детей.
              </p>
              <div className="space-y-1.5 text-xs">

```

```
.....  
    }}  
  </div>  
};  
}
```

ФАЙЛ 57/77: src/components/Sidebar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....  
import React, { useMemo, useState } from "react";  
import { ChevronRight, Compass, Search, Sparkles, X } from "lucide-react";  
import { chapters } from "../data/content";  
import { VIZ_REGISTRY } from "../vizRegistry";
```

```
interface SidebarProps {  
  selectedChapterId: string;  
  setSelectedChapterId: (id: string) => void;  
  /** Мобильный режим: список живёт в выдвижной панели */  
  onNavigate?: () => void;  
}
```

```
export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId, setSelectedChapterId, onNavigate }) => {  
  const [query, setQuery] = useState("");
```

```
  const groups = useMemo(() => {  
    const q = query.trim().toLowerCase();  
    const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;  
    const map = new Map<string, typeof chapters>();  
    for (const c of filtered) {  
      const key = c.category?.trim() || "Прочие темы";  
      if (!map.has(key)) map.set(key, []);  
      (map.get(key) as typeof chapters).push(c);  
    }  
    return Array.from(map.entries());  
  }, [query]);
```

```

.....
      <button
        key={chapter.id}
        onClick={() => {
          setSelectedChapterId(chapter.id);
          onNavigate?.();
          window.scrollTo({ top: 0, behavior: 'smooth' });
        }}
        className={`w-full text-left px-3 py-2
          ${selectedChapterId === chapter.id
            ? "bg-indigo-600/15 border-indigo-600/20"
            : "bg-slate-800/40 border-transparent"}
          `}
      >
        {hasViz && (
          <span
            className="w-1.5 h-1.5 rounded-full"
            title="Есть интерактивная визуализация"
          />
        )}
        <span className="font-semibold text-white">
          <ChevronRight
            className={`w-4 h-4 shrink-0 mt-0.5
              ${selectedChapterId === chapter.id
                ? "text-indigo-400"
                : "text-white"}
            `}
          />
        </button>
      );
    }
  }
</div>
</div>
)))
</div>

```

```

<div className="shrink-0 m-3 mt-0 bg-gradient-to-r from-indigo-600/20 to-indigo-600/0"
  > <div className="font-bold text-indigo-400 flex items-center justify-between"
    > <Sparkles className="w-3.5 h-3.5" /> Как пользоваться
  </div>

```

```

.....
const [activeTab, setActiveTab] = useState<'stack' |

// Stack state
const [stack, setStack] = useState<Plate[]>([
  { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
  { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
  { id: '3', name: 'Блюдец от торта', icon: ' ' },
]);
const [popMessage, setPopMessage] = useState<string |

// Queue state
const [queue, setQueue] = useState<Person[]>([
  { id: '1', name: 'Студент Вася', icon: ' ' },
  { id: '2', name: 'Голодный кодер', icon: ' ' },
  { id: '3', name: 'Кот Борис', icon: ' ' },
]);
const [dequeueMessage, setDequeueMessage] = useState<

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
  { id: '1', text: 'Кандидат: "Привет, я искусственный"', icon: ' ' },
  { id: '2', text: 'Кандидат: "Привет, я испек вкусный"', icon: ' ' },
  { id: '3', text: 'Кандидат: "Привет, я испанский ин"', icon: ' ' },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.1);
const [heapMessage, setHeapMessage] = useState<string |

// Stack handlers
const addPlate = () => {
  const presets = [
    { name: 'Сковорода от омлета', icon: ' ' },
    { name: 'Кастрюля от супа', icon: ' ' },
    { name: 'Чашка от кофе', icon: '☺' },
    { name: 'Миска с остатками салата', icon: ' ' },
    { name: 'Тарелка от тако', icon: ' ' },
  ];
  const randomPreset = presets[Math.floor(Math.random()

```

```

.....
    id: Date.now().toString(),
    name: randomPreset.name,
    icon: randomPreset.icon
  };
  setQueue(prev => [...prev, newPerson]);
  setDequeueMessage(` Встал в конец очереди: ${newPe
});

const servePerson = () => {
  if (queue.length === 0) {
    setDequeueMessage("⚠ Очередь пуста! Суп ждет нов
    return;
  }
  const firstPerson = queue[0];
  setQueue(prev => prev.slice(1));
  setDequeueMessage(` Выдан суп первому в очереди (F
});

const resetQueue = () => {
  setQueue([
    { id: '1', name: 'Студент Вася', icon: ' ' },
    { id: '2', name: 'Голодный кодер', icon: ' ' },
    { id: '3', name: 'Кот Борис', icon: ' ' },
  ]);
  setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
  e.preventDefault();
  if (!newCustomText.trim()) return;
  const item: Hypothesis = {
    id: Date.now().toString(),
    text: `Кандидат: "${newCustomText}"`,
    probability: Number(newCustomProb)
  };
  setHeap(prev => [...prev, item]);
  setNewCustomText('');

```



```

</h2>
<p className="text-slate-400 text-sm mt-2">
  Проверь работу принципов LIFO, FIFO и Приоритет
</p>
</div>

{/* Tabs */}
<div className="grid grid-cols-1 md:grid-cols-3 gap-3">
  <button
    onClick={() => setActiveTab('stack')}
    className={`flex items-center justify-between gap-2
      ${activeTab === 'stack'
        ? 'bg-blue-600/20 border-blue-500 text-blue-600'
        : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
    `}
  >
    <div className="flex items-center gap-3">
      <Layers className={`w-5 h-5 ${activeTab === 'stack' ? 'text-blue-600' : 'text-slate-400'}
        <div className="text-left">
          <div className="font-bold text-sm text-white">
            <div className="text-xs opacity-80">LIFO
          </div>
        </div>
      </div>
      <span className="text-xs bg-blue-500/20 text-blue-600">
        DFS
      </span>
    </div>
  </button>

  <button
    onClick={() => setActiveTab('queue')}
    className={`flex items-center justify-between gap-2
      ${activeTab === 'queue'
        ? 'bg-indigo-600/20 border-indigo-500 text-indigo-600'
        : 'bg-slate-800/60 border-slate-700/60 text-slate-400'}
    `}
  >
    <div className="flex items-center gap-3">
      <AlignLeft className={`w-5 h-5 ${activeTab === 'queue' ? 'text-indigo-600' : 'text-slate-400'}

```

```

.....
    <span>    Симуляция стопки тарелок</span>
    <span className="text-xs font-mono bg-blue-950/60 border border-blue-500/40 px-4 py-2.5 rounded-xl" />
  </h3>
  <p className="text-slate-400 text-xs mt-1">
    Новые тарелки ставятся наверх. Мыть можно.
  </p>
</div>
<div className="flex flex-wrap items-center gap-2">
  <button
    onClick={addPlate}
    className="flex-1 md:flex-none flex items-center justify-center gap-2 rounded-xl text-sm font-bold shadow-lg transition-colors cursor-pointer"
  >
    <Plus className="w-4 h-4" /> Поставить тарелку
  </button>
  <button
    onClick={popPlate}
    className="flex-1 md:flex-none flex items-center justify-center gap-2 border-blue-500/40 px-4 py-2.5 rounded-xl text-sm font-bold shadow-lg transition-colors cursor-pointer"
  >
    Помыть верхнюю
  </button>
  <button
    onClick={resetStack}
    title="Сбросить"
    className="p-2.5 bg-slate-800 hover:bg-slate-700 text-white text-sm font-bold shadow-lg transition-colors cursor-pointer"
  >
    <RotateCcw className="w-5 h-5" />
  </button>
</div>
</div>

{popMessage && {
  <div className="bg-blue-950/60 border border-blue-500/40 px-4 py-2.5 rounded-xl text-sm font-mono animate-fade-in">
    <span className="inline-block w-2 h-2 rounded-full bg-blue-500/40" /> {popMessage}
  </div>
}
```

```

.....
        </div>
        <span className={`text-xs font-mono
          isTop ? 'bg-blue-500 text-white' : ''
        }`>
          {isTop ? 'Готово к помывке' : ''}
        </span>
      </div>
    );
  }}}
</div>
})
<div className="mt-6 text-xs text-slate-500" >
  ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) •
</div>
</div>
</div>
})

{/* QUEUE TAB */}
{activeTab === 'queue' && (
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-xl
      tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-white">
          <span> Симуляция очереди за супом</span>
          <span className="text-xs font-mono bg-blue-500 text-white" style={
            /span>
          </h3>
          <p className="text-slate-400 text-xs mt-1">
            Кто первым пришел, тот первым получил суп
          </p>
        </div>
        <div className="flex flex-wrap items-center">
          <button
            onClick={addPerson}
            className="flex-1 md:flex-none flex items-center justify-center
              -2.5 rounded-xl text-sm font-bold shadow-lg"
          >

```

```

) : (
  <div className="w-full flex items-center" >
    <div className="flex flex-col items-center justify-center gap-3" style={{ width: 300 }}>
      <span className="text-3xl"> </span>
      <span className="text-xs font-bold font-mono">
        </div>

      <div className="text-indigo-500 flex items-center gap-2">
        <ArrowRight className="w-6 h-6 animate-pulse" />
      </div>

      {queue.map((person, index) => {
        const isFirst = index === 0;
        return (
          <div
            key={person.id}
            className={`flex flex-col items-center justify-center gap-2 ${
              isFirst
                ? 'bg-gradient-to-b from-indigo-500 to-white'
                : 'bg-slate-900/90 border-slate-800'
            }`}
            style={{ width: 300, height: 100, position: 'relative' }}
          >
            {isFirst && (
              <span className="absolute -top-10 left-0 right-0 text-center text-sm">
                Первый (Head)
              </span>
            )}
            <span className="text-4xl mb-2">{person.name}</span>
            <span className={`font-bold text-${isFirst ? 'indigo-500' : 'white'} font-mono`} style={{ width: 100px }}>
              {person.name}
            </span>
            <span className={`text-${isFirst ? 'indigo-500' : 'white'} font-mono`} style={{ width: 100px }}>
              {isFirst ? 'Получает cyn' : 'В'}
            </span>
          </div>
        )
      })}
    </div>
  </div>
)

```

```

.....
      py-2.5 rounded-xl text-sm font-bold shadow-
    >
      Взять кандидата с макс. вероятностью
    </button>
    <button
      onClick={resetHeap}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-
s-12 gap-4 items-end">
  <div className="md:col-span-6">
    <label className="block text-xs font-bold">
    <input
      type="text"
      value={newCustomText}
      onChange={e => setNewCustomText(e.target
placeholder='например: "Привет, я лучший"'
      className="w-full bg-slate-900 border b
rder-emerald-500 transition-colors"
    />
  </div>
  <div className="md:col-span-3">
    <label className="block text-xs font-bold">
      Вероятность: <span className="text-emerg
    </label>
    <input
      type="range"
      min="0.01"
      max="0.99"
      step="0.01"

```

```

.....
    <span>КАНДИДАТ / ГИПОТЕЗА</span>
    <span>ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)</span>
  </div>

  {sortedHeap.map(({item, index}) => {
    const isTop = index === 0;
    const percent = (item.probability * 100).toFixed(1);
    return {
      <div
        key={item.id}
        className={`flex items-center justify-between
          ${isTop ? 'bg-gradient-to-r from-emerald-500 to-slate-900/10 scale-[1.01]' : 'bg-slate-900/80 border-slate-800'}
        `}
      >
        <div className="flex items-center justify-between" >
          <span className={`flex items-center justify-between
            ${isTop ? 'bg-emerald-500 text-white' : 'bg-slate-900/80 text-slate-400'}
          `}>
            #{index + 1}
          </span>
          <div>
            <div className={`font-bold text-white`} >
              {item.text}
            </div>
            {isTop && (
              <span className="text-[10px] text-white font-medium" >
                Вершина Кучи (Root) • Будущее
              </span>
            )}
          </div>
        </div>
      </div>

      <div className="flex items-center justify-between" >
        {/* Custom progress bar */}

```

```
.....
interface Props {
  /** Открыть тренажёр в модальном окне. */
  onOpen: (vizId: string) => void;
  /** Перейти к теме, к которой привязан тренажёр. */
  onGoChapter: (chapterId: string) => void;
}

/**
 * Каталог всех интерактивных тренажёров.
 *
 * Раньше вкладка «Симулятор» показывала один случайный
 * теперь это витрина: видно всё, что вообще можно поты
 */
export const SimulatorHub: React.FC<Props> = ({ onOpen,
  const [query, setQuery] = useState("");

  const items = useMemo(() => {
    const titleById = new Map(chapters.map((c) => {c.id
    const all = Object.entries(VIZ_REGISTRY).map(([id,
      id,
      entry,
      chapterTitle: titleById.get(id),
    }));
    const q = query.trim().toLowerCase();
    if (!q) return all;
    return all.filter(
      (i) =>
        i.entry.title.toLowerCase().includes(q) ||
        (i.entry.hint ?? "").toLowerCase().includes(q)
        (i.chapterTitle ?? "").toLowerCase().includes(q)
    );
  }, [query]);

  return (
    <div>
      <div className="mb-6">
        <h2 className="text-xl sm:text-2xl font-extrabo
        <p className="text-sm text-slate-400 leading-re
```

```

.....
      <button
        onClick={() => onOpen(id)}
        className="flex items-center gap-1.5 text-white
transition-colors"
      >
        <Play className="w-3.5 h-3.5" /> Запустить
      </button>
      {chapterTitle && {
        <button
          onClick={() => onGoChapter(id)}
          title={chapterTitle}
          className="flex items-center gap-1.5 text-white
0 hover:text-white transition-colors min-w-[100px]"
        >
          <BookOpen className="w-3.5 h-3.5 shrink-0" />
          <span className="truncate max-w-[9rem]" />
        </button>
      }}
    </div>
  </div>
)}}
</div>
</div>
);
};

```

ФАЙЛ 60/77: src/components/SplayTreeViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....

import React, { useState } from "react";
import {
  RotateCcw,
  HelpCircle,
  BookOpen,
} from "lucide-react";

```


Универсальное пособие • полный исходный код

```
.....
// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
  if (!node) return null;
  return {
    key: node.key,
    left: cloneTree(node.left),
    right: cloneTree(node.right),
    x: node.x,
    y: node.y,
  };
};

export const SplayTreeViz: React.FC = () => {
  // Let's create an elegant initial unbalanced or semi-balanced tree
  // Values: 10, 20, 30, 40, 50, 60
  const createInitialTree = (): SplayNode => {
    let r: SplayNode | null = null;
    const initialKeys = [40, 20, 50, 10, 30, 60];
    initialKeys.forEach((key) => {
      r = insertBST(r, key);
    });
    return r!;
  };

  const [tree, setTree] = useState<SplayNode>(createInitialTree());
  const [inputValue, setInputValue] = useState<string>('');
  const [log, setLog] = useState<string[]>([
    "Дерево инициализировано. Попробуйте кликнуть на узел.",
  ]);
  const [highlightedNode, setHighlightedNode] = useState<SplayNode | null>(null);

  // Splay operation that tracks steps!
  const splayStepByStep = (
    root: SplayNode | null,
    key: number,
  ): { newRoot: SplayNode | null; steps: string[] } => {
    const steps: string[] = [];
    if (!root) return { newRoot: null, steps: ["Дерево пустое."] };
    if (root.key === key) return { newRoot: root, steps: [] };
    if (root.left && root.left.key < key) {
      // Left-Right case
      const [newLeft, newRoot] = splayStepByStep(root.left, key);
      root.left = newLeft;
      return { newRoot, steps: [...steps, "LR"] };
    }
    if (root.right && root.right.key < key) {
      // Right-Left case
      const [newRight, newRoot] = splayStepByStep(root.right, key);
      root.right = newRight;
      return { newRoot, steps: [...steps, "RL"] };
    }
    if (root.left && root.left.key > key) {
      // Left-Left case
      const [newLeft, newRoot] = splayStepByStep(root.left, key);
      root.left = newLeft;
      return { newRoot, steps: [...steps, "LL"] };
    }
    if (root.right && root.right.key > key) {
      // Right-Right case
      const [newRight, newRoot] = splayStepByStep(root.right, key);
      root.right = newRight;
      return { newRoot, steps: [...steps, "RR"] };
    }
    // Root is the only node left
    return { newRoot: root, steps: [] };
  };
};
```

```

.....
    node: SplayNode | null,
    k: number,
  ): SplayNode | null => {
    if (!node || node.key !== k) return node;

    if (k < node.key) {
      if (!node.left) return node;

      if (k < node.left.key) {
        // Zig-Zig (Left-Left)
        node.left.left = splayAction(node.left.left,
        node = rightRotate(node);
        steps.push(
          `Вращение Zig-Zig (Правый поворот родителя
        );
      } else if (k > node.left.key) {
        // Zig-Zag (Left-Right)
        node.left.right = splayAction(node.left.right
        if (node.left.right) {
          node.left = leftRotate(node.left);
          steps.push(`Компонент Zig-Zag: левый поворо
        }
      }

      if (!node.left) return node;
      steps.push(
        `Финальный поворот Zig (Правое вращение корня
      );
      return rightRotate(node);
    } else {
      if (!node.right) return node;

      if (k < node.right.key) {
        // Zag-Zig (Right-Left)
        node.right.left = splayAction(node.right.left
        if (node.right.left) {
          node.right = rightRotate(node.right);
          steps.push(`Компонент Zig-Zag: правый поворо

```

Универсальное пособие • полный исходный код

```
.....
    let newTree = cloneTree(tree);
    newTree = insertBST(newTree, val);
    const { newRoot, steps } = splayStepByStep(newTree,
    if (newRoot) {
        setTree(newRoot);
    }
    setLog(['Вставили вершину ${val}'].`, ...steps]);
    setInputValue("");
    setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
    e.preventDefault();
    const val = parseInt(inputValue);
    if (isNaN(val)) return;
    handleSplay(val);
    setInputValue("");
};

const resetTree = () => {
    setTree(createInitialTree());
    setHighlightedNode(null);
    setLog(["Дерево возвращено в исходное состояние."]);
};

// Assign coordinate positions using a simple recursive
const computeCoordinates = {
    node: SplayNode | null,
    x: number,
    y: number,
    levelWidth: number,
}: void => {
    if (!node) return;
    node.x = x;
    node.y = y;
    if (node.left) {
        computeCoordinates(node.left, x - levelWidth, y +
    }
```

```

        lines = lines.concat(renderLines(node.right));
    }
    return lines;
};

const renderNodes = (node: SplayNode | null): React.R
    if (!node) return [];
    let circles: React.ReactNode[] = [];
    const isHighlighted = highlightedNode === node.key;

    circles.push(
        <g
            key={`n-${node.key}`}
            className="cursor-pointer group"
            onClick={() => handleSplay(node.key)}
            data-title={`Кликните, чтобы выполнить Splay(${node.key})`}
        >
            <circle
                cx={node.x}
                cy={node.y}
                r="18"
                fill={isHighlighted ? "#4f46e5" : "#1e293b"}
                stroke={isHighlighted ? "#818cf8" : "#64748b"}
                strokeWidth={isHighlighted ? "3" : "2"}
                className="transition-all duration-300 group-hover:stroke-4"
            />
            <text
                x={node.x}
                y={node.y}
                dy=".3em"
                textAnchor="middle"
                fill="white"
                fontSize="12px"
                fontWeight="bold"
                className="select-none pointer-events-none"
            >
                {node.key}
            </text>
        </g>
    );

```

```

.....
        value={inputValue}
        onChange={(e) => setInputValue(e.target
        className="w-24 bg-slate-950 text-white
00 focus:outline-none"
      />
      <button
        type="submit"
        onClick={handleFind}
        className="bg-indigo-600 hover:bg-indigo
      >
        Поиск / Splay
      </button>
      <button
        type="button"
        onClick={(e) => {
          e.preventDefault();
          const val = parseInt(inputValue);
          if (!isNaN(val)) {
            let newTree = cloneTree(tree);
            newTree = insertBST(newTree, val);
            const { newRoot, steps } = splaySte
            if (newRoot) setTree(newRoot);
            setLog([`Вставили ${val}`], ...st
            setInputValue("");
            setHighlightedNode(val);
          }
        }}
        className="bg-indigo-950 hover:bg-slate
0"
      >
        Вставить
      </button>
    </form>

    <button
      onClick={resetTree}
      className="flex items-center gap-1.5 px-3
    >

```

```
.....
        </p>
      </div>
    </div>
  </div>

  /* Answers Section inside the visualizer widget
  <div className="bg-slate-950/80 p-5 border-t border-
    <h4 className="text-indigo-400 font-bold text-s
      <HelpCircle className="w-4 h-4 text-indigo-40
        Разбор экзаменационных вопросов (Без нитья):
      </h4>

    <div className="grid grid-cols-1 md:grid-cols-3
      <div className="bg-slate-900/60 p-4 rounded-l
        <div>
          <p className="font-bold text-slate-300 mb
            1. Отсутствующий элемент
          </p>
          <p className="text-slate-400 leading-rela
            Допустим, мы ищем{" "}
            <code className="text-indigo-400 font-s
              его нет. Алгоритм спустится к листу, на
              неудачей (например,{" "}
              <code className="text-indigo-400">[20]<
              <code className="text-indigo-400">[30]<
              поднимет в корень{" "}
              <strong className="text-white">
                последний успешно просмотренный узел
              </strong>
              , на котором он споткнулся! Это оставля
              сверху.
            </p>
          </div>
        <div className="mt-3 text-[10px] text-indig
          Поиск настраивает локальный кэш под реали
        </div>
      </div>
```

```
.....
      <div className="mt-3 text-[10px] text-emerald-500">
        Каждое "растяжение" компенсируется "сжатием"
      </div>
    </div>
  </div>
</div>
</div>
);
};
```

ФАЙЛ 61/77: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

interface Plate {
  id: string;
  name: string;
  emoji: string;
  color: string;
  isDirty: boolean;
  animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE_PRESETS = [
  { name: 'Тарелка из-под спагетти', emoji: '🍝', color: 'from-pink-500 to-orange-400' },
  { name: 'Тарелка из-под пиццы', emoji: '🍕', color: 'from-pink-500 to-orange-400' },
  { name: 'Блюдец от торта', emoji: '🍰', color: 'from-pink-500 to-orange-400' },
  { name: 'Тарелка от тако', emoji: '🌮', color: 'from-pink-500 to-orange-400' },
  { name: 'Тарелка из-под суши', emoji: '🍣', color: 'from-pink-500 to-orange-400' },
  { name: 'Сковорода от глазуньи', emoji: '🍳', color: 'from-pink-500 to-orange-400' },
];

let counter = 0;
```

```

.....

    setDirtyStack(prev => [...prev, newPlate]);
    addLog(`  PUSH: Положили ${newPlate.emoji} сверху с

    setTimeout(() => {
      setDirtyStack(prev => prev.map(p => p.id !== newP
    }, 500);
  }, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {
    setShake(true);
    addLog('△ POP: Нечего мыть! Все тарелки чистые.')
    setTimeout(() => setShake(false), 400);
    return;
  }

  setIsWashing(true);
  const topPlate = dirtyStack[dirtyStack.length - 1];

  addLog(`  POP: Начинаем мыть верхнюю тарелку с ${top

// Step 1: Start washing animation (sponge and soap
setDirtyStack(prev => prev.map(p => p.id !== topPla
setShowSoap(true);

setTimeout(() => {
  // Step 2: Wash complete. Plate becomes clean and
  const cleanPlate: Plate = {
    ...topPlate,
    isDirty: false,
    animState: 'clean',
  };

  setCleanRack(prev => [cleanPlate, ...prev].slice(
  setDirtyStack(prev => prev.slice(0, prev.length -

```



```

{ /* УПРАВЛЕНИЕ */ }
<div className="flex items-center gap-3 flex-wrap"
  <button
    onClick={pushPlate}
    disabled={isWashing}
    className="flex-1 min-w-[150px] bg-gradient-to-r
      from-slate-400 to-slate-500 opacity-40 disabled:cursor-not-allowed text-white
      font-size-95 transition-all cursor-pointer flex items-center justify-center"
  >
    <span> Положить грязную (PUSH)</span>
  </button>

  <button
    onClick={popPlate}
    disabled={isWashing || dirtyStack.length === 0}
    className="flex-1 min-w-[150px] bg-gradient-to-r
      from-slate-400 to-slate-500 opacity-40 disabled:cursor-not-allowed text-white
      font-size-95 transition-all cursor-pointer flex items-center justify-center"
  >
    <span> Помыть верхнюю (POP)</span>
  </button>

  <button
    onClick={reset}
    className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white
      font-size-95 transition-all cursor-pointer border border-slate-700"
  >
    Сброс
  </button>
</div>

{ /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
<div className="grid grid-cols-1 md:grid-cols-12"
  >
    { /* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */ }
    <div className="md:col-span-7 bg-slate-900 rounded-3xl
      p-10 flex flex-direction-reverse flex-wrap"
      >

```

```

.....
    <span className="text-xs font-mono text-emerald gap-1">
      <ArrowUp className="w-3.5 h-3.5" />
      <span>ВЕРШИНА (TOP)</span>
    </span>
    <span className="text-blue-400 animate-pulse">
  </div>

  {/* Тарелки (отрисовываем в обратном порядке) */}
  {dirtyStack.slice().reverse().map((plate, idx) => {
    const isTop = idx === topIndex;
    return (
      <div
        key={plate.id}
        className={`
          w-full max-w-[280px] h-10 rounded
          shadow-md select-none transition-all duration-300
          ${plate.color}
          ${plate.animState === 'in' ? 'animate-in' : ''}
          ${plate.animState === 'washing' ? 'animate-washing' : ''}
          ${isTop ? 'ring-4 ring-blue-500/50' : ''}
        `}
      >
        <span className="text-lg">{plate.name}</span>
        <span className="text-slate-800 text-sm">
          {isTop && (
            <span className="text-[9px] bg-blue-500/50 rounded px-1">
              LIFO
            </span>
          )}
        </span>
      </div>
    );
  })}
</div>
);
}}}
</div>
)}

{/* Столешница раковины */}

```

```
.....
      {/* Столешница сушилки */}
      <div className="w-full h-4 bg-gradient-to-r f
    </div>
  </div>

  {/* ЛОГ ДЕЙСТВИЙ */}
  <div className="bg-slate-900 border border-slate-:
    <div className="text-[10px] font-mono text-slate
      {log.map((entry, idx) => {
        <div
          key={idx}
          className={`text-xs font-mono flex items-ce
        >
          {idx === 0 ? <span className="text-blue-500
          <span>{entry}</span>
        </div>
      }}}
    </div>
  </div>
);
};
```

.....
ФАЙЛ 62/77: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
.....
import { useState, useEffect, useRef, useMemo } from "r
import { Play, Pause, SkipForward, Undo, RefreshCw } fr

function generateKmpSteps(pattern: string, text: string
  if (!pattern) pattern = " ";
  const s = pattern + "#" + text;
  const n = s.length;
  const pi = new Array(n).fill(0);
  const steps: any[] = [];
```

```

.....
const steps: any[] = [];

let l = 0, r = 0;
steps.push({ i: 0, l, r, z: [...z], desc: `Начало.` });

for (let i = 1; i < n; i++) {
    steps.push({ i, l, r, z: [...z], desc: `Шаг i=$` });

    if (i <= r) {
        steps.push({ i, l, r, z: [...z], desc: `Внимание!` });
        z[i] = Math.min(r - i + 1, z[i - 1]);
        steps.push({ i, l, r, z: [...z], desc: `Предел. Копипаста сработала.` });
    }

    steps.push({ i, l, r, zTarget: z[i], zSource: i });
    while (i + z[i] < n && s[z[i]] === s[i + z[i]]) {
        steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i] });
        падают! Окно растёт.` });
        z[i]++;
    }

    if (i + z[i] < n) {
        steps.push({ i, l, r, zTarget: z[i], zSource: i + z[i] });
        [i]}' разные. Стоп.` });
    } else {
        steps.push({ i, l, r, z: [...z], desc: `Упёрся.` });
    }

    steps.push({ i, l, r, z: [...z], desc: `Фиксируем.` });

    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
        steps.push({ i, l, r, z: [...z], desc: `Обновление` });
    }
}

```

```

        });
      }, 1200);
    }
    return () => { if (timer.current) clearInterval
  }, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setStep
const prevStep = () => { setAutoPlay(false); setStep
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-b
      <div className="flex bg-slate-900 borde
        <button onClick={() => setMode("kmp
          className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
          КМП (π-ФУНКЦИЯ)
        </button>
        <button onClick={() => setMode("z")
          className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
          Z-ФУНКЦИЯ
        </button>
      </div>

    <div className="flex items-center gap-2
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={pattern} onChange=
xt-white text-xs font-bold font-mono px-2 p
      </div>
      <div className="bg-slate-800 p-1 fl
        <span className="text-[10px] te
        <input value={text} onChange={e

```

```

.....
    return (
      <div key={index} className=
        {/* L/R markers for Z *
        {mode === "z" && step.l
          <div className="abs
        }}
        {mode === "z" && step.r
          <div className="abs
        }}

        {/* Index */}
        <span className={`text-

        {/* Char Box */}
        <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
        {char}
        </div>

        {/* Value array box */}
        <div className="text-[1
slate-700/50 font-mono font-bold">
          {mode === "kmp" ? s
        </div>

        {/* KMP fingers */}
        {mode === "kmp" && step
          <div className="abs

            <span classNam
          </div>
        }}
        {mode === "kmp" && step
          <div className="abs
            <span classNam
          </div>
        }}
        {mode === "kmp" && step

```

```

<div className="flex flex-col sm:flex-row gap-4" style={{width: '100%'}}
  {/* Controls */}
  <div className="flex bg-slate-900 border-1 border-slate-800 rounded-lg disabled:opacity-30 disabled:hover:opacity-50" style={{width: '100%'}}
    <button onClick={prevStep} disabled={!steps.length} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-50" style={{width: '20%'}}
      <Undo strokeWidth={3} size={16} style={{display: 'block', margin: '0 auto'}}
    </button>
    <button onClick={playPause} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-50" style={{width: '20%'}}
      {autoplay ? <Play size={16} style={{display: 'block', margin: '0 auto'}}> : <Pause size={16} style={{display: 'block', margin: '0 auto'}}>
    </button>
    <button onClick={nextStep} disabled={steps.length === 1} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-50" style={{width: '20%'}}
      <SkipForward strokeWidth={3} size={16} style={{display: 'block', margin: '0 auto'}}
    </button>
    <button onClick={reset} className="text-white rounded-lg disabled:opacity-30 disabled:hover:opacity-50" style={{width: '20%'}}
      <RefreshCw strokeWidth={3} size={16} style={{display: 'block', margin: '0 auto'}}
    </button>
  </div>

  {/* Description Log */}
  <div className="flex-1 bg-slate-900/50 border-1 border-slate-800 rounded-lg overflow-hidden" style={{width: '100%'}}
    <div className="absolute top-0 right-0 text-white font-mono text-sm p-2" style={{width: '100px', height: '20px'}}
      <span>{steps.length}</span>
    </div>
    <div className="text-[10px] text-slate-400" style={{width: '100%'}}
      {steps.map((step, index) => (
        <div className="text-sm text-slate-400" style={{width: '100%'}}
          {step.desc}
        </div>
      ))}
    </div>
  </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl" style={{width: '100%'}}

```

ФАЙЛ 63/77: src/components/TopologicalSortViz.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React, { useState, useEffect } from "react";
import {
  Play,
  Pause,
  RotateCcw,
  Clock,
} from "lucide-react";

interface TNode {
  id: number;
  x: number;
  y: number;
  label: string;
  name: string;
}

interface TEdge {
  u: number;
  v: number;
}

const dagNodes: TNode[] = [
  { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
  { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
  { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
  { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
  { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
  { u: 0, v: 1 },
  { u: 0, v: 2 },
  { u: 1, v: 3 },
  { u: 2, v: 3 },
  { u: 3, v: 4 },
];
```



```

.....
    recordStep(
      "Начало топологической сортировки. Используем кла
      null,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, (
    dagEdges.forEach((e) => adj[e.u].push(e.v)));

    const dfs = (u: number) => {
      visited.add(u);
      dfsStack.push(u);
      recordStep(
        `DFS: зашли в вершину ${u} (${dagNodes[u].name
        u,
      );

      for (const to of adj[u]) {
        if (!visited.has(to)) {
          dfs(to);
        } else if (!fullyProcessed.has(to)) {
          // Explaining potential cycle detected (not i
          recordStep(
            `⚠ Внимание: Рёбра в серую вершину ${to} о
            u,
          );
        }
      }
    }

    fullyProcessed.add(u);
    dfsStack.pop();
    topoList.push(u); // prepend behavior: we write i
    recordStep(
      `DFS: закончили обработку '${dagNodes[u].name}'
      ,
      u,
    );
  };
};

```

```

.....
    return () => clearInterval(timer);
  }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)
    .join(" → ");
};

return (
  <div className="bg-slate-900 rounded-xl border border-
    <div className="bg-slate-800 p-4 border-b border-
      <h3 className="font-bold text-white flex items-
        <Clock className="w-5 h-5 text-indigo-400" />
        Топологическая сортировка (Зависимости обучен
      </h3>

      <div className="flex items-center gap-2 bg-slate-
        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            500 text-white"
        >
          {isPlaying ? {
            <Pause className="w-4 h-4 ml-1" />
          } : {
            <Play className="w-4 h-4 ml-1" />
          }}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

```

```

    { /* Edges */
    {dagEdges.map((edge, idx) => {
      const u = dagNodes.find((n) => n.id === edge.u)
      const v = dagNodes.find((n) => n.id === edge.v)

      const isActive =
        (step.currentNode === edge.u || step.currentNode === edge.v) ||
        step.visited.has(edge.v);

      return (
        <line
          key={`edge-${idx}`}
          x1={u.x}
          y1={u.y}
          x2={v.x}
          y2={v.y}
          stroke={isActive ? "#818cf8" : "#334155"}
          strokeWidth={isActive ? "3" : "2"}
          markerEnd={`url(#${isActive ? "dag-arrow" : "dag-line"})`}
          className="transition-all duration-300"
        />
      );
    })}
  )}
}

```

```

    { /* Nodes */
    {dagNodes.map((node) => {
      const isCurrent = step.currentNode === node.id
      const isVisited = step.visited.has(node.id)
      const isProcessed = step.fullyProcessed.has(node.id)

      let fill = "#1e293b";
      let stroke = "#334155";
      let textColor = "text-slate-400";
      void textColor;

      if (isCurrent) {
        fill = "#312e81";
      }
    })}
  )}
}

```

```

.....
        y={node.y + 12}
        textAnchor="middle"
        fill="#94a3b8"
        fontSize="9px"
        className="pointer-events-none"
      >
        {node.name.split(" ").slice(1).join(" ")}
      </text>
    </g>
  );
}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-400">
    Статус сортировки
  </h4>

  <div className="bg-slate-900 border border-slate-800 p-4">
    <p className="text-xs text-indigo-300 min-h-[100px]">
      {step.desc}
    </p>
  </div>

  <div className="flex-1 space-y-4 overflow-y-auto">
    {/* Topological List output */}
    <div>
      <span className="text-[10px] text-slate-500">
        РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):
      </span>
      <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
        {step.topoList.length === 0 ? (
          <span className="text-slate-600 text-sm">
            ожидаем обработку узлов...

```

```

.....
        Назад
      </button>
      <button
        disabled={currentStepIdx >= steps.length}
        onClick={() => setCurrentStepIdx((prev)
        className="bg-slate-900 border border-s
t-white"
      >
        Вперед
      </button>
    </div>
  </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 64/77: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

.....

import React from "react";

import { ArticulationPointsViz } from "../ArticulationPo
import BellmanFordViz from "../BellmanFordViz";
import BfsViz from "../BfsViz";
import { DPVisualizer } from "../DPVisualizer";
import { DfsBridgesSimulator } from "../DfsBridgesSimula
import DijkstraViz from "../DijkstraViz";
import { EulerSimulator } from "../EulerSimulator";
import FloydViz from "../FloydViz";
import { GraphTraversalViz } from "../GraphTraversalViz"
import { HeapViz } from "../HeapViz";
import { JohnsonViz } from "../JohnsonViz";
import { KosarajuViz } from "../KosarajuViz";

```

```

.....
    Component: () => (
      <div className="space-y-10">
        <QueueViz />
        <BfsViz />
      </div>
    ),
  },
  "heap-beam-search": {
    title: "Куча (priority queue)",
    hint: "Добавляйте гипотезы – куча сама держит лучшую",
    Component: HeapViz,
  },
  "segment-trees": {
    title: "Дерево отрезков",
    hint: "Запросы и обновления на отрезке за  $O(\log n)$ .",
    Component: SegmentTreeVisualizer,
  },
  "sparse-table": {
    title: "Разреженная таблица (1D и 2D)",
    hint: "Наведите курсор на любую ячейку – подсветится",
    Component: SparseTableViz,
  },
  "prefix-sums-2d": {
    title: "Префиксные суммы (1D и 2D)",
    hint: "Наведите на число – увидите, из чего оно сложилось",
    Component: PrefixSumViz,
  },
  "dynamic-programming": {
    title: "Динамическое программирование",
    hint: "Таблица подзадач заполняется на глазах.",
    Component: DPVisualizer,
  },
  "splay-tree": { title: "Splay-дерево", Component: SplayTreeViz },
  "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: GraphViz },
  "top-sort": { title: "Топологическая сортировка", Component: TopSortViz },
  "scc-kosaraju": { title: "Компоненты сильной связности", Component: SCCViz },
  "graph-articulation": { title: "Точки сочленения", Component: GraphArticulationViz },
  "bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesCodeViz },

```

```
.....
  hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
  Component: () => (
    <div className="space-y-10">
      <WaterfallAnimationWidget />
      <SalmonAutomatonWidget />
    </div>
  ),
},
intro: { title: "Мнемокарточки", Component: MnemonicCards },
"everyday-basics": {
  title: "Бытовой тренажёр: стек, очередь, куча",
  hint: "Тарелки, очередь в столовой и мешок гипотез",
  Component: Simulator,
},
};
```

```
export const getViz = (id?: string): VizEntry | undefined;
```

```
.....
ФАЙЛ 65/77: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО
```

```
.....
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
```

```
// Структура заранее подготовленного красивого бора для визуализации
// Словарь: ["HE", "SHE", "HIS", "HERS"]
```

```
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
  x: number; // Координаты для SVG водопада
  y: number;
  depth: number;
  fail: number;
  term link: number;
```

```
// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
```

```
const [textToScan, setTextToScan] = useState<string>('');
const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('');
  'Режим поиска: Введите текст и жмите «Следующий шаг»';
);
const [jumpPath, setJumpPath] = useState<{ from: number, to: number }>({ from: 0, to: 0 });
const [foundMatches, setFoundMatches] = useState<{ index: number, match: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);
```

```
// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
```

```
// Шаги обучения от 0 до 8 (по числу вершин в боре, и
```

```
const [trainStep, setTrainStep] = useState<number>(0);
```

```
const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const clean = e.target.value.toUpperCase().replace(/ /g, '');
  setTextToScan(clean);
  resetSearchSimulation();
};
```

```
const resetSearchSimulation = () => {
  setCurrentIndex(0);
  setCurrentNode(0);
  setIsSearchDone(false);
  setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
  setJumpPath(null);
  setFoundMatches([]);
  setActiveSearchCodeLine(1);
};
```

```
const stepSearchSimulation = () => {
  if (currentIndex >= textToScan.length) {
    setIsSearchDone(true);
    setSearchLog('Мы проплыли весь текст! Лосось доволен');
    setActiveSearchCodeLine(4);
  }
};
```



```

.....
    const currentMatches: { index: number; word: string }[] = [];
    const matchedWordsForLog: string[] = [];

    let temp = curr;
    while (temp !== 0) {
        if (WATERFALL_NODES[temp].is_terminal) {
            WATERFALL_NODES[temp].words.forEach((word) => {
                currentMatches.push({ index: currentIndex, word });
                matchedWordsForLog.push(word);
            });
        }
        temp = WATERFALL_NODES[temp].term_link;
    }

    if (matchedWordsForLog.length > 0) {
        logMsg += ` БИНГО! Найдено слово: ${matchedWordsForLog.join(' ')}\n`;
        setActiveSearchCodeLine(4);
    }

    setCurrentNode(curr);
    setCurrentIndex(currentIndex + 1);
    setSearchLog(logMsg);
    if (currentMatches.length > 0) {
        setFoundMatches((prev) => [...prev, ...currentMatches]);
    }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В ЧИСТОМ ВИДЕ)
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обрабатывается. Остальные вершины сразу инициализируются с fail = ROOT',
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'ø',
    },

```

```

.....
    desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель :
        Н" и "S" не подходят. Ветки нет, fail[HI] = ROOT
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'I',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 8,
    title: 'Шаг 5 (Depth 2): Вершина #8 (SH) – КАК СТА
    desc: '    ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на D
        ). Рыба плывет в ROOT и ищет ветку "Н". У корня
        суффикс "Н"!',
    codeLine: 4,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'H',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 3,
    title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
    desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
        е подходят. fail[HER] = ROOT.',
    codeLine: 3,
    scoutPos: 0,
    inspectedParentFail: 0,
    checkTargetChar: 'R',
    checkingBranchesFrom: 0,
},
{
    targetNodeId: 6,
    title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
    desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
        но ветка "S" → #7 (S) совпадает! fail[HIS] = #7
    codeLine: 4,
    scoutPos: 0,

```

```

.....
    <div>
      <h4 className="text-2xl font-bold text-white">
        Анимация водопада: Полное обучение автомата
      </h4>
      <p className="text-sm text-slate-400">
        Визуализация пошагового построения всех функций
      </p>
    </div>
  </div>

  { /* Переключатель режимов */ }
  <div className="bg-slate-900 p-1.5 rounded-xl border">
    <button
      onClick={() => setActiveMode('training')}
      className={`flex items-center space-x-1.5 p-2
        activeMode === 'training'
          ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
          : 'text-slate-400 hover:text-white'
        }`
    >
      <GitBranch className="w-4 h-4" />
      <span>Режим 1: Полное обучение (BFS от корня)</span>
    </button>
    <button
      onClick={() => setActiveMode('search')}
      className={`flex items-center space-x-1.5 p-2
        activeMode === 'search'
          ? 'bg-blue-600 text-white shadow-lg shadow-blue-500'
          : 'text-slate-400 hover:text-white'
        }`
    >
      <Play className="w-4 h-4" />
      <span>Режим 2: Поиск в тексте</span>
    </button>
  </div>
</div>

{ /* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */ }

```

```

>
  Назад
</button>
<button
  onClick={() => setTrainStep((prev) => M
  disabled={trainStep === TRAINING_STEPS_
  className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
  >
    {trainStep === TRAINING_STEPS_INFO.leng
  </button>
</div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
  >
  <div>
    <h5 className="text-white font-bold mb-3
      <span className="flex items-center gap-
        <span></span> Код BFS построения fail
      </span>
      <span className="text-xs bg-indigo-950
        Вершина: #{targetTrainingNode.id} ({t
      </span>
    </h5>
    <div className="bg-slate-950 p-4 rounded-
      <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
        <span>1. let u = trie[r][char]; // Те
        {currentTrainInfo.codeLine === 1 && <
тут</span>}
      </div>
      <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-400 text-indigo-200 font
        <span>2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail?.label}}</span>
        {currentTrainInfo.codeLine === 2 && <

```

```

<div className="bg-slate-900/60 p-5 rounded-x
  <div>
    <h5 className="text-white font-bold mb-2
      <span>✖ </span> Текст для сканирования
    </h5>
    <p className="text-xs text-slate-400 mb-4
      Введи текст, чтобы лосось просканировал
    </p>
    <input
      type="text"
      value={textToScan}
      onChange={handleTextChange}
      maxLength={15}
      placeholder="ВВЕДИТЕ ТЕКСТ..."
      className="w-full bg-slate-800 border b
      cus:outline-none focus:border-blue-500"
    />
  </div>

  <div>
    <div className="mb-4 flex flex-wrap gap-1
      <span className="text-xs text-slate-400
        {textToScan.split('').map((char, idx) =>
          <span
            key={idx}
            className={`w-7 h-8 flex items-cent
              idx === currentIndex
                ? 'bg-blue-600 text-white shadow
                : idx < currentIndex
                ? 'bg-emerald-950/60 text-emera
                : 'bg-slate-800 text-slate-500'
            }`}
          >
            {char}
          </span>
        )}}
      </div>
      {currentIndex >= textToScan.length && (
        <span className="bg-emerald-600 text-

```

```

    </div>
    <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold` :
    <span>3. curr = trie[curr].get(char)
    {activeSearchCodeLine === 3 && <span .
ем по течению</span>}
    </div>
    <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 fon
    <span>4. if (is_terminal[curr]) report
    {activeSearchCodeLine === 4 && <span .
овпадение!</span>}
    </div>
  </div>
</div>

<div>
  <h5 className="text-slate-300 font-bold m
    <AlertCircle className="w-3.5 h-3.5 tex
  </h5>
  <div className="bg-slate-800 p-3.5 rounded
tems-center">
    {searchLog}
  </div>
</div>
</div>
</div>
  )}

```

```

  { /* SVG Анимация водопада */ }

```

```

<div className="bg-gradient-to-b from-blue-950 vi
-hidden shadow-2xl">
  { /* Водопадные фоновые эффекты */ }
  <div className="absolute inset-0 opacity-10 bg-
o-900 to-transparent pointer-events-none"></d
  <div className="absolute top-0 left-1/2 -transl
    <div className="w-8 bg-blue-400 h-full animat
    <div className="w-12 bg-blue-300 h-full anima

```

```

        opacity="0.4"
      />
      <rect x="25" y={level.y - 12} width="14" height="12"
    />
    <text x="32" y={level.y + 4} fill="#94a0b4" font-size="10"
      {level.label}
    </text>
  </g>
)))

{/* Рендер прямых переходов (потоков воды) */}
Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
  const fromNode = WATERFALL_NODES[Number(fromId)];
  return Object.entries(children).map(([childId, child]) => {
    const toNode = WATERFALL_NODES[toId];

    // Подсветка в режиме поиска
    let isHighlighted = activeMode === 'search' ? true :
    уре === 'normal';

    // Подсветка в режиме обучения (когда ищем ответ)
    let isTrainingExamined = activeMode === 'training' ? true : false;
    let isMatchBranch = isTrainingExamined && childId === toId;

    return (
      <g key={`edge-${fromNode.id}-${toNode.id}`}
        <rect x={fromNode.x} y={fromNode.y} width={fromNode.w} height={fromNode.h}
        <rect x={toNode.x} y={toNode.y} width={toNode.w} height={toNode.h}
        <rect x={fromNode.x} y={fromNode.y} width={fromNode.w} height={fromNode.h}
        <rect x={toNode.x} y={toNode.y} width={toNode.w} height={toNode.h}
        stroke={isHighlighted ? '#3b82f6' : '#ccc'}
        strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
        className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
      />
      <circle cx={(fromNode.x + toNode.x) / 2} cy={(fromNode.y + toNode.y) / 2}

```

```

.....
    let isHighlighted = activeMode === 'search'
    e === 'fail';
    let isJustEstablished = activeMode === 'target' ||
    -ссылку

    // Вычисляем изогнутую кривую для красоты
    const dx = targetNode.x - node.x;
    const dy = targetNode.y - node.y;
    const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -1 : 1) * 30;
    const cy = node.y + dy / 2 - 30;

    return (
      <g key={`fail-${node.id}`}>
        <path
          d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`
          fill="none"
          stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
          strokeWidth={isHighlighted || isJustEstablished ? 6 : 2}
          strokeDasharray="6,6"
          className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
          opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
        />
        {isJustEstablished && (
          <text x={cx} y={cy - 10} fill="#10b98f">
            fail[{node.label}] = {targetNode.label}
          </text>
        )}
      </g>
    );
  });
}

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'target' || activeMode === 'trail';
  const hasWords = node.words.length > 0;

```



```

.....
        ИЩЕМ FAIL
    </text>
</g>
}}

{ /* Индикатор словарных слов */
{hasWords && {
    <g transform={`translate(${node.x +
    <circle cx="0" cy="0" r="9" fill=
    <text x="0" y="3" fontSize="10" f
        *
    </text>
    </g>
    }}
</g>
});
}}
}}

{ /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ *
<g>
    { /* Круг-подложка под рыбу */
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-1g"
    />
    { /* Сама рыба */
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
        style={{ transition: 'x 0.8s cubic-bezi

```

```
        }}
      </div>
    );
  };
};
```

РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 66/77: src/components/visualizers/PrefixSumViz.tsx

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ: 1000

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка — наведение на число:
 * • в 1D наводим на P[i] — подсвечивается кусок массива
 * • в 1D наводим на a[i] — видно, в какие префиксы он входит
 * • в 2D наводим на ячейку — подсвечивается прямоугольник
 *   а при выбранном запросе показывается формула включения-исключения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
  [4, 5, 6, 7],
];

const cellBase =
  "flex items-center justify-center rounded-md font-monospace";
```

```
.....
const [range, setRange] = useState<{ l: number; r: num
```

```
const covered =
```

```
  hover?.kind === "pref"
```

```
    ? { from: 0, to: hover.i - 1 }
```

```
    : hover?.kind === "a"
```

```
      ? { from: hover.i, to: hover.i }
```

```
      : null;
```

```
const sum = pref[range.r + 1] - pref[range.l];
```

```
return (
```

```
  <div className="space-y-5">
```

```
    <div className="overflow-x-auto pb-1">
```

```
      <div className="inline-block min-w-full">
```

```
        {/* индексы */}
```

```
        <div className="flex gap-1.5 mb-1 pl-[52px]">
```

```
          {A1.map((_, i) => (
```

```
            <div key={i} className={`${cellBase} !h-5
```

```
              {i}
```

```
            </div>
```

```
          )))
```

```
        </div>
```

```
        {/* массив a */}
```

```
        <div className="flex gap-1.5 items-center mb-1">
```

```
          <div className="w-[46px] shrink-0 text-right">
```

```
            {A1.map((v, i) => {
```

```
              const inCover = covered && i >= covered.f
```

```
              const inRange = i >= range.l && i <= rang
```

```
              return (
```

```
                <div
```

```
                  key={i}
```

```
                  onMouseEnter={() => setHover({ kind:
```

```
                    onMouseLeave={() => setHover(null)}
```

```
                    onClick={() => setRange((r) => (i < r
```

```
                  className={`${cellBase} cursor-pointe
```

```
                    inCover
```

```

        });
      }}}
    </div>
  </div>
</div>

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border i
  {hover?.kind === "pref" ? (
    hover.i === 0 ? (
      <p className="text-slate-300">
        <b className="text-emerald-400">P[0] = 0</b>
        отрезка, начинающегося с нуля.
      </p>
    ) : (
      <p className="text-slate-300">
        <b className="text-emerald-400">
          P[{hover.i}] = {pref[hover.i]}
        </b>{" "}
        – это сумма всего, что набежало от начала
        <span className="font-mono text-indigo-300">
          a[0..{hover.i - 1}] = {A1.slice(0, hove
        </span>
      </p>
    )
  ) : hover?.kind === "a" ? (
    <p className="text-slate-300">
      <b className="text-indigo-400">
        a[{hover.i}] = {A1[hover.i]}
      </b>{" "}
      входит во все префиксы, начиная с{" "}
      <span className="font-mono text-emerald-300">
        подвинуть границу запроса.
      </span>
    </p>
  ) : (
    <p className="text-slate-500">
      Наведите курсор на любое число: у <b classN
      у <b className="text-indigo-400">a</b> – ку

```

```

.....
const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">Исходная матрица
        <div className="inline-block">
          {A2.map((row, i) => (
            <div key={i} className="flex gap-1.5 mb-1">
              {row.map((v, j) => {
                const inRect = i >= rect.r1 && i <= rect.r2 && j >= rect.c1 && j <= rect.c2;
                return (
                  <div
                    key={j}
                    onClick={() => setRect((r) => {
                      i < r.r1 || j < r.c1 ? { r1: i, c1: j, r2: i, c2: j } : r
                    })}
                    className={` ${cellBase} cursor-pointer ${
                      inRect ? "bg-indigo-500 text-white" : "bg-white text-slate-400"
                    }`}
                  >{v}</div>
                );
              })}
            </div>
          ))}
        </div>
      </div>
    </div>
  </div>
);

```

```

.....
        {corner && !isHover && {
            <span className="absolute -top-
ht">
                {corner}
            </span>
        }}
    </div>
    );
    }}}
  </div>
  )))
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border i
  {hover ? {
    <p className="text-slate-300">
      <b className="text-amber-400">
         $P[\{hover.i\}][\{hover.j\}] = \{P[\{hover.i\}][hov$ 
      </b>{" "}
      – сумма всего прямоугольника от левого верх
    </p>
  } : {
    <p className="text-slate-500">Наведите курсор
  }}
</div>

<div className="bg-slate-900 border border-indigo
  <p className="font-mono text-sm sm:text-base te
    сумма = <span className="text-emerald-400">A<
    <span className="text-rose-400">C</span> + <s
    <span className="text-indigo-300 font-bold">{
  </p>
  <p className="text-[11px] text-slate-500 mt-1">
    Вычли два «хвоста» и вернули дважды вычитенный
  </p>
</div>

```

Универсальное пособие • полный исходный код

```
.....  
    [14, 51, 99, 13, 22, 64, 43, 76],  
    [89, 32, 54, 71, 15, 88, 29, 60],  
    [25, 41, 16, 92, 9, 17, 56, 31],  
    [59, 18, 77, 24, 61, 82, 35, 12],  
    [73, 8, 38, 85, 47, 95, 19, 58],  
    [39, 81, 62, 28, 51, 42, 85, 14]  
];  
  
const ST2D: number[][][] = Array.from({length: 8}, ()  
    Array.from({length: 8}, () =>  
        Array.from({length: 4}, () =>  
            Array(4).fill(0)  
        )  
    )  
);  
  
for (let r = 0; r < 8; r++) {  
    for (let c = 0; c < 8; c++) {  
        ST2D[r][c][0][0] = val2D[r][c];  
    }  
}  
  
for (let kx = 0; kx <= 3; kx++) {  
    for (let ky = 0; ky <= 3; ky++) {  
        if (kx === 0 && ky === 0) continue;  
        for (let r = 0; r + (1 << kx) <= 8; r++) {  
            for (let c = 0; c + (1 << ky) <= 8; c++) {  
                if (kx > 0) {  
                    ST2D[r][c][kx][ky] = Math.min(  
                        ST2D[r][c][kx-1][ky],  
                        ST2D[r + (1 << (kx-1))][c][kx-1][ky]  
                    );  
                } else {  
                    ST2D[r][c][kx][ky] = Math.min(  
                        ST2D[r][c][kx][ky-1],  
                        ST2D[r][c + (1 << (ky-1))][kx][ky-1]  
                    );  
                }  
            }  
        }  
    }  
}
```

```

    <Viz2DBuild key="2d_build" />
  ) : (
    <Viz2DQuery key="2d_query" />
  )}
</AnimatePresence>
</div>
);
};

```

```

const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }
}

```

```

const maxStep = computeSteps.length;
const covered = hover ? { from: hover.i, to: hover.i + 1 } : null;

```

```

return (
  <div className="flex flex-col gap-5">
    <div className="flex items-center justify-between">
      <div>
        <h3 className="text-lg font-bold text-indigo-600">
          Построение Sparse Table (Min)
        </h3>
        <p className="text-sm text-slate-400">
          Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + 2^{j-1}][j-1])$ 
        </p>
      </div>
      <div className="flex gap-2">
        <button
          onClick={() => setStep(Math.max(0, step - 1))}
        >

```



```

        );
    }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
    {hover && covered ? (
        <p className="text-slate-300">
            <b className="text-sky-400">
                ST[{hover.i}][{hover.j}] = {stID[hover.i]}
            </b>{" "}
            – это минимум на отрезке{" "}
            <span className="font-mono text-sky-300">
                [{covered.from}, {covered.to}]
            </span>{" "}
            длины  $2^{\{hover.j\}} = \{1 \ll hover.j\} : \{ " " \}$ 
            <span className="font-mono text-slate-400">
                min({arrID.slice(covered.from, covered.to)})
            </span>
        </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор (или тапните) на любое число
        </p>
    )
</div>
</div>

<div className="overflow-x-auto pb-2">
    <div className="inline-block min-w-full bg-slate-50">
        <div className="grid grid-cols-[auto_repeat(4, 1fr)]">
            {/* Headers */}
            <div className="text-slate-500 font-mono text-sm">
                i \ j
            </div>
            <div className="text-center font-bold text-sm">
                 $2^0$  (L=1)
            </div>
            <div className="text-center font-bold text-sm">

```

```

.....
        currentStep.j === j + 1 &&
        currentStep.i === i - (1 << (currentStep.j - 1))
    )
    isSrc2 = true; // wait, no. Src2
    // Let's re-eval exact source:
    // We are asking if THIS cell [i][j]
    if (currentStep.j - 1 === j) {
        if (currentStep.i === i) isSrc1 = true;
        if (currentStep.i + (1 << (currentStep.j - 1)) === i)
            isSrc2 = true;
    }
}

return (
    <div key={j} className="relative flex flex-col items-center justify-center"
    {!isValid ? (
        <div className="w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200"
    ) : (
        <motion.div
            onMouseEnter={() => isVisible = true}
            onMouseLeave={() => setHover(false)}
            onClick={() => isVisible && setHover(true)}
            initial={{ opacity: 0, scale: 0.8 }}
            animate={{
                opacity: isVisible ? 1 : 0,
                scale: isVisible ? 1 : 0.8,
            }}
            className={`w-[clamp(30px,9vw,17px)] font-bold transition-colors duration-200
            hover && hover.i === i && hover.j === j
            ? "bg-sky-500 text-white"
            : isActive
            ? "bg-indigo-500 text-white"
            : isSrc1
            ? "bg-emerald-500 text-white"
            : isSrc2
            ? "bg-amber-500 text-white"
            : isVisible
        `}
    )}

```

```

/* Formula helper text */
<div className="bg-slate-900 border border-slate-800"
  {step > 0 && step <= maxStep ? (
    (() => {
      const c = computeSteps[step - 1];
      const half = 1 << (c.j - 1);
      return (
        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][[{c.j}]]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-bold">
              ST[{c.i}][[{c.j} - 1]]
            </span>
            ,
            <span className="text-amber-400 font-bold">
              ST[{c.i + half}][[{c.j} - 1]]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{st1D[{c.i}][c.j]}
            <span className="text-amber-400">
              {st1D[{c.i + half}][c.j - 1]}
            </span>
          ) ={" "}
            <span className="text-indigo-400 font-bold">
              {st1D[{c.i}][c.j]}
            </span>
          </p>
        );
      })();
    }) : (
      <p className="text-slate-500 italic">
        Нажмите далее, чтобы увидеть, как блоки сво
      </p>
    )
  )
</div>

```



```

.....
    tiveCell.r][activeCell.c + prevL][k-1][k-1]
        </div>
        <div className="flex item
            <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
            <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
        </div>
        <div className="flex item
            <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
            <span className="text
[activeCell.r + prevL][activeCell.c + prevL
        </div>
    </>
    )
    }}())}
</div>

    <div className="pt-3 mt-3 border-t
        ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : (
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
        Наведите курсор на матрицу {L}x{L
        </div>
    )}
</div>
    )}
</div>
</div>
</div>
);

```

```

        </div>
    )
}}}

<div
    className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
    style={{
        top: `${(r1 / 8) * 100}%`,
        left: `${(c1 / 8) * 100}%`,
        marginTop: '8px',
        marginLeft: '8px',
        height: 'calc(' + (H/8*100) +
        width: 'calc(' + (W/8*100) +
    }}
/>

    /* Показываем 4 угла в самой матрице
    <div className="absolute pointer-e
    shadow-[0_0_10px_#f43f5e]" style={{ top: `
    lc(`${(Ly / 8) * 100}% - 16px}`, height: `ca
    <div className="absolute pointer-e
    shadow-[0_0_10px_#3b82f6]" style={{ top: `
    width: `calc(`${(Ly / 8) * 100}% - 16px}`,
    <div className="absolute pointer-e
    ld-400 shadow-[0_0_10px_#10b981]" style={{
    8px}`, width: `calc(`${(Ly / 8) * 100}% - 16
    <div className="absolute pointer-e
    00 shadow-[0_0_10px_#f59e0b]" style={{ top:
    00}% + 8px}`, width: `calc(`${(Ly / 8) * 100
    </div>

    <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
    <label className="flex items-cen
    <span className="text-slate-
    <div className="flex gap-2">
        <input type="number" min={

```

```

.....
    ext-amber-300">1</span>); <span className="
      <span className="text-purple-400">int<
"text-slate-500">// Выс: {Lx}</span><br/>
      <span className="text-purple-400">int<
"text-slate-500">// Шир: {Ly}</span><br/><b

      <span className="text-slate-500">// 2.
      <span className="text-indigo-400">int<
      &nbsp;&nbsp;&nbsp;<span className="text-slate-500">В запроса)</span><br/>
      &nbsp;&nbsp;&nbsp;<span className="text-rose-400">r1</span>][<span className="text-white bg-r

      &nbsp;&nbsp;&nbsp;<span className="text-slate-500">во, чтобы правый край уперся в c2)</span><b
      &nbsp;&nbsp;&nbsp;<span className="text-blue-400">r1</span>][<span className="text-white bg-b

      &nbsp;&nbsp;&nbsp;<span className="text-slate-500">рх, чтобы нижний край уперся в r2)</span><b
      &nbsp;&nbsp;&nbsp;<span className="text-emerald-400">nded">r2 - Lx + 1</span>][<span className="

      &nbsp;&nbsp;&nbsp;<span className="text-slate-500">и влево от c2)</span><br/>
      &nbsp;&nbsp;&nbsp;<span className="text-amber-400">">r2 - Lx + 1</span>][<span className="text
      {'}'}));
    </div>

    <div className="w-full flex justify-cent
      <div className="flex flex-col items-ce
      <h4 className="text-slate-400 font-bo
      Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST{|||||{kx}}||{k
      </h4>
      <div className="grid gap-{2px} p-2.5
Columns: `repeat(${matCols}, 1fr)`>>

```



```
.....
        <span className="text-slate-400 font-medium">
          <span className="text-rose-400">
            <span className="text-blue-400 m
n>
          <span className="text-emerald-400
span>
            <span className="text-amber-400 f
            &nbsp;<span className="text-slate-400 f
            <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
            </div>
          </div>
        </div>
      </div>
    )
  }
```

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 68/77: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```
      </div>
    </main>
  </div>

  <script>
    function setMode(mode) {
      document.getElementById('btn-mode-normal').
        ? 'px-3 py-1 rounded text-sm font-bold i
        : 'px-3 py-1 rounded text-sm font-bold i
```

.....

```
if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
);
```

```
document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
-600 text-white transition-colors' : 'px-3 py-1 rounded transition-colors';
ransition-colors';
```

```
}
```

```
// Initialize theme and mode
setTheme('dark');
```

```
// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
  const mobileMenuBtn = document.getElementById('mobile-menu-btn');
  const mobileDrawer = document.getElementById('mobile-drawer');
  const closeDrawerBtn = document.getElementById('close-drawer-btn');
  const drawerOverlay = document.getElementById('drawer-overlay');
```

```
function toggleDrawer() {
  const isClosed = mobileDrawer.classList.contains('hidden');
  if (isClosed) {
    mobileDrawer.classList.remove('hidden');
    if (drawerOverlay) {
      drawerOverlay.classList.remove('hidden');
      // Timeout allows display:block to take effect
      setTimeout(() => drawerOverlay.classList.remove('hidden'), 100);
    }
    document.body.style.overflow = "hidden";
  } else {
    mobileDrawer.classList.add('hidden');
    if (drawerOverlay) {
      drawerOverlay.classList.add('hidden');
    }
    document.body.style.overflow = "auto";
  }
}
```



```
.....
mjx-container {
  max-width: 100% !important;
}
mjx-container[display="true"] {
  overflow-x: auto;
  overflow-y: hidden;
  max-width: 100%;
  scrollbar-width: none;
  -ms-overflow-style: none;
  -webkit-overflow-scrolling: touch;
}
mjx-container[display="true"]::-webkit-scrollbar
  display: none;
}

@media (max-width: 640px) {
  html { font-size: 13px; }

  .uno-card { width: 40px; height: 60px; font-
  .uno-card::before, .uno-card::after { font-
  .uno-card::before { top: 1px; left: 2px; }
  .uno-card::after { bottom: 1px; right: 2px; }

  .uno-card.mini { width: 28px !important; he
  us: 4px; }
  .uno-card.mini::before, .uno-card.mini::aft

  .uno-equation .text-2xl { font-size: 1.2rem
  .uno-equation.gap-4 { gap: 0.5rem !important

  /* Allow horizontal scroll for equations in
  .formula-scroll { flex-wrap: nowrap !importa
  px; }
}
.uno-inner {
  position: absolute; width: 110%; height: 75%
  border-radius: 50%; transform: rotate(-30de
  box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
```

```
.....
    background-image: repeating-linear-gradient(
    background-attachment: local !important;
    font-family: "Comic Sans MS", "Caveat", "Se
}
body.theme-notebook .bg-slate-900, body.theme-n
    slate-700 {
    background-color: rgba(255, 255, 255, 0.7)
    border-color: #cbd5e1 !important;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme
    ant; }
body.theme-notebook .text-slate-400 { color: #5
body.theme-notebook .border-slate-600, body.the
    : #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "
body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1);
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
    transform-origin: center;
```

```

</marker>
<marker id="arrow-pink" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#f472b0" stroke="none"/>
</marker>
<marker id="arrow-cyan" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4c1" stroke="none"/>
</marker>
<marker id="arrow-indigo" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" stroke="none"/>
</marker>
<marker id="arrow-lime" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="none"/>
</marker>
<marker id="arrow-green" markerWidth="6" markerHeight="6"
  <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="none"/>
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
  <rect x="0" y="0" width="40" height="20" fill="yellow" stroke="black" stroke-width="1"/>
  <rect x="6" y="-5" width="8" height="6" fill="yellow" stroke="black" stroke-width="1"/>
  <rect x="26" y="-5" width="8" height="6" fill="yellow" stroke="black" stroke-width="1"/>
  <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>
<g id="lego-red">
  <rect x="0" y="0" width="40" height="20" fill="red" stroke="black" stroke-width="1"/>
  <rect x="6" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
  <rect x="26" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
  <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>
<g id="lego-rem">
  <rect x="0" y="0" width="40" height="10" fill="red" stroke="black" stroke-width="1"/>
  <rect x="6" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
  <rect x="26" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
  <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>

<!-- МЯСОРУБКА -->
<g id="meat-grinder">

```

```
.....
  nslate-x-full transition-transform duration-300
  w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
  <div class="h-full flex flex-col p-6 overflow-hidden
  lg:rounded-2xl lg:border lg:border-slate-700">
    <div class="flex justify-between items-center">
      <h3 class="font-black text-transparent bg-clip-text
  ider text-sm">
    <div class="flex justify-between items-center">
```

Содержание

```
    </h3>
    <button id="close-drawer-btn" class="text-slate-500
      <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6"
        <path stroke-linecap="round" stroke="black" stroke-width="2"
      </svg>
    </button>
  </div>
  <nav class="space-y-3 text-sm font-medium">
    <div class="group">
      <summary class="list-none cursor-pointer text-slate-500
        <a href="#question-1" class="text-indigo-500 pl-3 font-bold text-blue-300">
          1. Алгебраические структуры
        </a>
      </summary>
      <div class="pl-6 space-y-2 text-sm">
        <a href="#section-1-1" class="text-slate-500">1.1. Группы
        <a href="#section-1-2" class="text-slate-500">1.2. Кольца
        <a href="#section-1-3" class="text-slate-500">1.3. Поля
        <a href="#section-1-4" class="text-slate-500">1.4. Модули
      </div>
    </div>
    <div class="group">
      <summary class="list-none cursor-pointer text-slate-500
        <a href="#question-2" class="text-indigo-500 pl-3 font-bold text-indigo-300">
          2. Комплексные числа
        </a>
```

```
.....
    <details class="group">
      <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-500">
        5. Группа корней из единицы
      </a>
      </summary>
      <div class="pl-6 space-y-2 text-fuchsia-500">
        <a href="#q5-def" class="block hover-fuchsia-500">
        <a href="#q5-group" class="block hover-fuchsia-500">
        <a href="#q5-prim" class="block hover-fuchsia-500">
        <a href="#q5-crit" class="block hover-fuchsia-500">
        <a href="#q5-cyclic" class="block hover-fuchsia-500">
      </div>
    </details>

    <details class="group">
      <summary class="list-none cursor-pointer-lime-500 pl-3 font-bold text-lime-300">
        6. Делимость, НОД и Евклид
      </a>
      </summary>
      <div class="pl-6 space-y-2 text-lime-500">
        <a href="#q6-1" class="block hover-lime-500">
        <a href="#q6-2" class="block hover-lime-500">
        <a href="#q6-3" class="block hover-lime-500">
        <a href="#q6-4" class="block hover-lime-500">
        <a href="#q6-5" class="block hover-lime-500">
        <a href="#q6-6" class="block hover-lime-500">
      </div>
    </details>

    <details class="group">
      <summary class="list-none cursor-pointer-pink-500 pl-3 font-bold text-pink-300">
```



```
.....
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q14-1" class="block">
        <a href="#q14-2" class="block">
        <a href="#q14-3" class="block">
deg)</a>
        <a href="#q14-4" class="block">
ition-colors border border-rose-500/30"> K
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer">
        <a href="#question-15" class="block border-cyan-500 pl-3 font-bold text-cyan-400">
            15. Производная. Критер
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q15-1" class="block">
        <a href="#q15-2" class="block">
        <a href="#q15-3" class="block">
        <a href="#q15-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer">
        <a href="#question-16" class="block border-red-500 pl-3 font-bold text-red-400">
            16. Неприводимые многоч
        </a>
    </summary>
    <div class="pl-6 mt-2 space-y-2">
        <a href="#q16-1" class="block">
        <a href="#q16-2" class="block">
        <a href="#q16-3" class="block">
```

```

.....
        if (detail.open) {
            document.querySelectorA
                if (other !== detail
                    other.removeAtt
                }
            });
        }
    });
});

// Scroll spy logic
document.addEventListener('DOMConten
    const sections = document.query
    ="q5-"], div[id^="q6-"], div[id^="q7-"], di
    v[id^="q13-"], div[id^="q14-"], div[id^="q1
    let isScrolling = false;

    document.querySelectorAll('.gro
        link.addEventListener('click
            isScrolling = true;
            setTimeout(() => isScro
        });
    });

    const observer = new Intersection
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersecting
                visibleId = entry.ta

                // Highlight active
                document.querySelec
                    a.style.fontWeig
                    if (a.href.ends
                        a.style.fon
                }
            }
        });
    });
}

```

```
.....
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a1a1a';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в ' + fileName;
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';
```

```

        modal.appendChild(box);
        document.body.appendChild(modal)
    }

```

```

function downloadMD() {
    let clone = document.querySelector('#clone')

    // Smart markdown parsing based on marked.js

    // Smart markdown parsing based on marked.js
    clone.querySelectorAll('h2').forEach(el => {
    clone.querySelectorAll('h3').forEach(el => {
    clone.querySelectorAll('.analog').forEach(el => {
    clone.querySelectorAll('li').forEach(el => {
    clone.querySelectorAll('svg').forEach(el => {
        let caption = '';
        if (el.nextElementSibling && el.nextElementSibling.classList.contains('img-caption')) {
            caption = el.nextElementSibling.textContent;
        }
        el.textContent = '\n\n[ Изображение: ' + caption + ' ]';
    });
    clone.querySelectorAll('.img-caption').forEach(el => {
        el.remove();
    });

    let text = clone.innerHTML;
    text = text.replace(/\n{3,}/g, '\n\n');

    if (window !== window.top) {
        fallbackModal(text, 'Markdown');
        return;
    }

    const blob = new Blob([text], { type: 'text/markdown' });
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.href = url;
    link.download = 'vyisshaya_algebra.md';
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
}

```

```

.....

body.bg-white {class*="

nt; }

body.bg-white {class*="
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links a
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre

```

```

.....
        </button>
        <a href="algebra.pdf" target="_blank">
- sm font-bold flex items-center justify-center
            <span> </span> PDF
        </a>
        <button onclick="downloadMD()">
- bold flex items-center justify-center gap-2
            <span> </span> Markdown
        </button>
        <button onclick="downloadHTML()">
t-sm font-bold flex items-center justify-center
            <span> </span> HTML
        </button>
    </div>

    <!-- Смена темы -->
    <div>
        <div class="text-xs text-slate-500">
            <div class="flex items-center gap-2">
                <button id="theme-dark" onclick="toggleTheme('dark')">
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
                <button id="theme-light" onclick="toggleTheme('light')">
r justify-center text-amber-50 hover:text-white
">* </button>
                <button id="theme-terminal" onclick="toggleTheme('terminal')">
- center justify-center text-green-400 hover:text-white
терминала"> </button>
            </div>
        </div>
    </div>
</div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12">
- base min-w-0">

```

```

.....
<div id="wtf-container" class="hidden">
  <h2 class="text-3xl font-black text-cen

    <div class="overflow-x-auto w-full pb-4"
      <div class="grid grid-cols-3 gap-4"
        <!-- Column headers -->
        <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
        <div class="bg-orange-900/40 bo
ольцо полиномов</div>
        <div class="bg-cyan-900/40 bord
сные числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tra
        <div class="text-xs text-orange
        <div class="font-bold text-slate
        </a>
        <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-5
        <div class="font-bold text-slate
        </a>

        <!-- Row 2: Базовые операции / Евкл
        <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
        <div class="text-xs text-yellow
        <div class="font-bold text-slate
        </a>
        <a href="#question-10" onclick="set
-1-4 border-orange-500 hover:bg-slate-700 t

```



```

</a>

<!-- Row 5: Корни / Уравнения -->
<div class="hidden md:block"></div>
<a href="#question-13" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
  <div class="text-xs text-orange
  <div class="font-bold text-slate
</a>
<a href="#question-4" onclick="setM
-l-4 border-cyan-500 hover:bg-slate-700 trans
  <div class="text-xs text-cyan-5
  <div class="font-bold text-slate
</a>

<!-- Row 6: Кратность -->
<div class="hidden md:block"></div>
<a href="#question-14" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
  <div class="text-xs text-orange
  <div class="font-bold text-slate
</a>
<div class="hidden md:block"></div>

<!-- Row 7: Производная -->
<div class="hidden md:block"></div>
<a href="#question-15" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
  <div class="text-xs text-orange
  <div class="font-bold text-slate
</a>
<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл
<a href="#question-17" onclick="setM

```

```
.....
        <div class="font-bold text-slate-500">
            </a>
        </div>
    </div>

    <!-- ЛИКБЕЗ SECTION INSERTED HERE -->
    <section id="proof-algorithms" class="mb-6">
        <div class="flex items-center mb-6">
            <div class="bg-red-500/20 p-3 rounded" style="width: 100px; height: 100px; margin-right: 10px; position: relative; border: 1px solid #ccc; border-radius: 5px;">
                <svg class="w-8 h-8 text-red-500" style="position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); pointer-events: none;">
                    <path stroke-linecap="round" stroke-width="2" d="M110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4" />
                </svg>
            </div>
            <h2 class="text-3xl pr-4 border" style="border-left: 2px solid #ccc; padding-left: 10px; margin: 0;">
                </div>

            <div class="prose prose-invert max-w-4xl">
                <p class="text-lg">
                    Хватит тупить. Доказательство
                    есть для того, чтобы жать на кнопки. Вот жес
                </p>

                <h2 class="text-2xl font-bold border" style="border: 1px solid #ccc; padding: 5px; margin-top: 10px;">
                    (Меметика)</h2>

                <!-- Прямое -->
                <div class="bg-slate-800/80 p-6">
                    <h3 class="text-xl text-blue-400">
                        <p class="text-sm text-slate-400">
                            <ul class="list-disc pl-5">
                                <li><span class="text-white">
                                    k$).</li>
                                <li><span class="text-white">
                                    oao.</li>
                                <li><span class="text-white">
                                    oao.</li>
                            </ul>
```

```
.....
        <li><span class="text-w
усок  $k$ , замени его на жареного карася из
        </ul>
    </div>

    <!-- Двойная делимость -->
    <div class="bg-slate-800/80 p-6
        <h3 class="text-xl text-ora
        <p class="text-sm text-slate
        <ul class="list-disc pl-5 sp
            <li><span class="text-w
вверх по уравнениям).</li>
            <li><span class="text-w
рху вниз, показывая, что  $\Delta \vdots r_i$ 
            <li><span class="text-w
х. Таой  $d$  больше.</li>
        </ul>
    </div>

    <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>

    <div class="grid grid-cols-1 md
        <div class="bg-slate-800 p-
            <h4 class="text-red-400
            <p class="text-sm text-
            <p class="text-xs font-
со следующим. Если в конце 0 – пациент мерт
        </div>

        <div class="bg-slate-800 p-
            <h4 class="text-red-400
            <p class="text-sm text-
            <p class="text-xs font-
 $c$ . Пока получается 0 – корень жив. Как то
корня минус один.</p>
        </div>
```

```

.....
        <p class="text-sm text-gray-500">
          x двух, потом последних. <br>
        <code class="text-x">
      </div>
    </li>

    <li class="bg-slate-800/50 p-4">
      <span class="text-2xl">
        <div>
          <h4 class="text-cyan">
            <p class="text-sm text-gray-500">
              есть сложение – есть ноль. Если есть умножение,
            <code class="text-x">
          </div>
        </li>

        <li class="bg-slate-800/50 p-4">
          <span class="text-2xl">
            <div>
              <h4 class="text-cyan">
                <p class="text-sm text-gray-500">
                  авь их драться. Умножение всегда врывається :
                <code class="text-x">
              </div>
            </li>
          </ul>
        </div>
      </section>
    </div>

    <div id="questions-container">
      <!-- ...<div class="text-sm text-gray-500"> ВОПРОС 1 ...

```

Универсальное пособие • полный исходный код

```
.....
fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")]
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\d+)\.\/)?.[1]
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\s>]/i.test(html),
    image: /<img[\s>]/i.test(html),
    interactive: vizIds.has(c.id),
    chars: html.length,
  };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && !
```

Универсальное пособие • полный исходный код

```
.....
* Проверка покрытия глав всплывающими подсказками:
* в каждой ли теме находятся термины из глоссария.
*
*   node scripts/audit-terms.mjs
*/
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossary.ts"],
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".mjs" },
  external: ["react", "react-dom", "motion", "lucide-react"],
});

const { chapters } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));

const esc = (s) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]\]/g, "\\");
const re = new RegExp(
  `(?<![\\p{L}\\p{N}_-])(?=${GLOSSARY_LABELS.map((l) => esc(l.label))})` +
  "giu"
);
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l]));

const bad = [];
const used = new Set();
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/[\s\S]\/>/gi, "")
    .replace(/<[^\>]+>/g, " ");
  const ids = new Set([...text.matchAll(re)].map((m) => m[1]));
  ids.forEach((i) => used.add(i));
  if (ids.size === 0) bad.push(c.title);
  console.log(String(ids.size).padStart(2), "|", c.title);
}
console.log("\nГлав без единой подсказки:", bad.length,
```

```
.....
    size: [A4.width, A4.height],
    margin: 0,
    autoFirstPage: false,
    info: {
      Title: "Универсальное пособие — полный исходный код",
      Author: "CI: rebuild-pdf",
      Subject: "Экспорт всего кода проекта (код -> txt)",
      CreationDate: new Date(),
    },
  });

  const stream = fs.createWriteStream(PDF_PATH);
  doc.pipe(stream);

  for (const file of pages) {
    doc.addPage({ size: [A4.width, A4.height], margin: 0 });
    doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
  }

  doc.end();
  await new Promise((resolve, reject) => {
    stream.on("finish", resolve);
    stream.on("error", reject);
  });

  console.log("[3/3] png -> pdf");
  console.log(`    страниц: ${pages.length}`);
  console.log(`    выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
  console.error("Ошибка сборки PDF:", err.message);
  process.exit(1);
});
```

Универсальное пособие • полный исходный код

```
.....  
  [/^[^/]+$/, "Конфигурация проекта"],  
];  
  
const categoryOf = (rel) => CATEGORIES.find(({rel}) => rel === rel)  
  
/** Порядок разделов в документе. */  
const ORDER = [  
  "Конфигурация проекта",  
  "Ядро приложения",  
  "Контент и данные пособия",  
  "Билеты (учебный контент)",  
  "Интерактивные компоненты и симуляторы",  
  "Визуализаторы (вложенные)",  
  "HTML-разметка (layout)",  
  "Утилиты",  
  "Скрипты сборки и экспорта",  
  "CI / автоматизация",  
  "Прочее",  
];  
  
function listFiles() {  
  let files;  
  try {  
    files = execFileSync("git", ["ls-files", "--cached"], {  
      cwd: ROOT,  
      encoding: "utf8",  
    })  
      .split("\n")  
      .filter(Boolean);  
  } catch {  
    files = [];  
    const walk = (dir) => {  
      for (const e of fs.readdirSync(dir, { withFileTypes: true })) {  
        const abs = path.join(dir, e.name);  
        const rel = path.relative(ROOT, abs).split(path.sep);  
        if (EXCLUDE_DIRS.some((d) => rel[0] === d || rel[0] === d + path.sep))  
          continue;  
        if (e.isDirectory()) walk(abs);  
        else files.push(rel);  
      }  
    };  
  }  
}
```



```
.....

function main() {
  const files = listFiles();
  const chapters = extractChapters(files);
  ensureDir(OUT_DIR);

  const out = [];
  const push = (s = "") => out.push(s);

  const stamp = new Date().toLocaleString("ru-RU", { time: true });
  const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size);

  push(HR);
  push("          УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ");
  push("          ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ И КОД)");
  push(HR);
  push();
  push(`Дата генерации:                ${stamp}`);
  push(`Файлов исходного кода:         ${files.length}`);
  push(`Суммарный объём кода:            ${humanSize(totalBytes)}`);
  push(`Глав/билетов в пособии:          ${chapters.length}`);
  push();

  push(HR);
  push("          ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
  push(HR);
  chapters.forEach((c, i) => {
    push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
  });
  push();

  push(HR);
  push("          ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
  push(HR);
  let current = null;
  files.forEach((rel, i) => {
    const cat = categoryOf(rel);
    if (cat !== current) {
      push(`  ${String(i + 1).padStart(3, " ")}. ${cat}`);
      current = cat;
    }
  });
}
```

```
.....
  console.log("[1/3] код -> txt");
  console.log(`      файлов:  ${files.length}`);
  console.log(`      глав:    ${chapters.length}`);
  console.log(`      строк:   ${txt.split("\n").length}`);
  console.log(`      выход:   ${path.relative(ROOT, TXT)}`);
}

main();

.....
```

ФАЙЛ 75/77: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ: 1.5 КБ

```
.....
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код   -> full_code_all_pages.txt    (collect-code.mjs)
 * txt   -> page-XXXX.png              (render-pages.mjs)
 * png   -> full_code_all_pages.pdf    (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают в бакет) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
.....
```

```
.....
    throw new Error(
      "ImageMagick не найден. Установите его: sudo apt-get
    );
}
```

ФАЙЛ 76/77: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ:

```
.....
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002
 */
```

```
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
  ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";
```

```
const execFileAsync = promisify(execFile);
```

```
/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
```

```
  const expanded = line.replace(/\t/g, "    ").replace(
  if (expanded.length <= PAGE.cols) return [expanded];
  const parts = [];
  const indent = " ".repeat(Math.min((expanded.match(/^
  let rest = expanded;
  let first = true;
```

```

.....
    fs.writeFileSync(txtFile, body + "\n", "utf8");
    return { txtFile, pngFile, num };
  });

const args = (job) => [
  "-density", String(PAGE.density),
  "-page", PAGE.size,
  "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
  "-pointsize", String(PAGE.pointsized),
  `text:${job.txtFile}`,
  "-background", "white",
  "-flatten",
  "-colorspace", "Gray",
  ...(PAGE.monochrome ? ["-monochrome"] : []),
  "-strip",
  "-define", "png:compression-level=9",
  job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

async function worker() {
  while (cursor < jobs.length) {
    const job = jobs[cursor++];
    await execFileAsync(im, args(job));
    fs.rmSync(job.txtFile, { force: true });
    done += 1;
    if (done % 25 === 0 || done === total) {
      process.stdout.write(`отрендерено ${done} страниц\n`);
    }
  }
}

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

Универсальное пособие • полный исходный код

```
.....
push:
  branches:
    - "**"
  paths-ignore:
    # чтобы коммит самого workflow не запускал бесконечный цикл
    - "public/export/**"
    - "**/*.md"
workflow_dispatch:
  inputs:
    density:
      description: "DPI растеризации страниц (по умолчанию 150)"
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"
    steps:
      - name: Checkout
        uses: actions/checkout@v4
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
```

```
.....
    echo ""
    echo "| Артефакт | Размер |"
    echo "| --- | --- |"
    echo "| \`public/export/full_code_all_pages"
    echo "| \`public/export/full_code_all_pages"
    echo "| PNG-страниц | ${ls tmp/export-pages.
} >> "$GITHUB_STEP_SUMMARY"

- name: Upload artifacts (PDF + TXT)
  uses: actions/upload-artifact@v4
  with:
    name: full-code-export
    path: |
      public/export/full_code_all_pages.pdf
      public/export/full_code_all_pages.txt
    if-no-files-found: error
    retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v4
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action.
    git add -- public/export/full_code_all_pages.
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full
    git push origin "HEAD:${GITHUB_REF_NAME}"
```